AIML (IS353IA) — Stripped Reference: Units III–V

Tan, Steinbach, Karpatne & Kumar, *Introduction to Data Mining*, 2nd ed., Global Edition · **Bold number** = page in this PDF · [grey] = original book page

UNIT III — Supervised Learning, Decision Trees, Overfitting, Model Selection

Supervised learning: basic concepts *(§3.1)* **4** [134]
General framework for classification *(§3.2)* **7** [137]
Decision tree classifier *(§3.3)* **9** [139]
- A basic algorithm to build a decision tree (Hunt's) 11 [141]
- Methods for expressing attribute test conditions 14 [144]
- Measures for selecting an attribute test condition (Gini, entropy, gain ratio) 17 [147]
- Algorithm for decision tree induction 26 [156]
- Characteristics of decision tree classifiers 29 [160]

Model overfitting *(§3.4)* **36** [167]
- Reasons for model overfitting 38 [169]

Model selection *(§3.5)* **45** [176]
- Using a validation set 45 [176]
- Incorporating model complexity 46 [177]
- Estimating statistical bounds 51 [182]
- Model selection for decision trees 51 [182]

Model evaluation *(§3.6)* **53** [184]
- Holdout method 54 [185]
- Cross-validation 54 [185]

UNIT IV — Nearest Neighbor, Naïve Bayes, Logistic Regression, Ensembles

Nearest neighbor classifiers *(§6.3)* **115** [410]
- Algorithm 116 [411]
- Characteristics of nearest neighbor classifiers 117 [412]

Naïve Bayes classifier *(§6.4)* **119** [414]
- Basics of probability theory 120 [415]
- Naïve Bayes assumption 125 [420]

Logistic regression *(§6.6)* **135** [445]
- Logistic regression as a generalized linear model 136 [446]
- Learning model parameters 137 [447]
- Characteristics of logistic regression 140 [450]

Ensemble methods *(§6.10)* **142** [498]
- Rationale for ensemble method 143 [499]
- Methods for constructing an ensemble classifier 143 [499]
- Bagging 147 [504]
- Boosting (AdaBoost) 150 [507]
- Random forests 155 [512]

UNIT V — Unsupervised Learning, K-means, Cluster Evaluation

Unsupervised learning: overview *(§5.1)* **61** [310]
- What is cluster analysis? 61 [310]
- Different types of clusterings 62 [311]
- Different types of clusters 64 [313]

K-means *(§5.2)* **67** [316]
- The basic K-means algorithm 68 [317]
- K-means: additional issues 77 [326]
- Bisecting K-means 80 [329]
- K-means and different types of clusters 81 [330]
- Strengths and weaknesses 82 [331]
- K-means as an optimization problem 82 [331]

Cluster evaluation *(§5.5)* **88** [353]
- Overview 88 [353]
- Unsupervised evaluation using cohesion and separation 91 [356]
- Unsupervised evaluation using the proximity matrix 99 [364]
- Determining the correct number of clusters 103 [369]

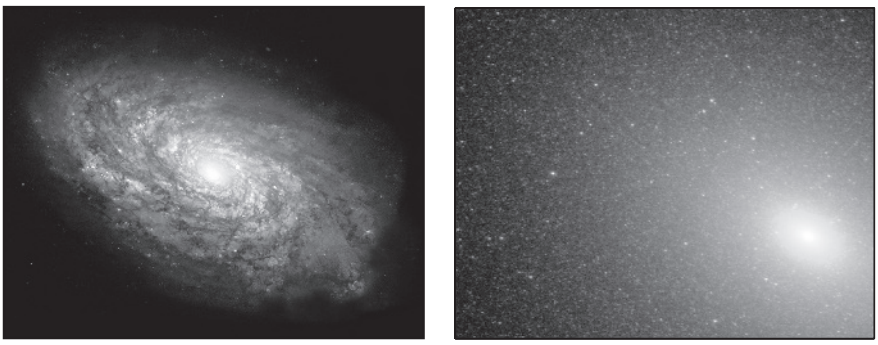
Supervised measures of cluster validity	105 [371]
Assessing the significance of cluster validity measures	110 [376]
Choosing a cluster validity measure	112 [378]

Notes. Pages are copied unchanged in book order (Ch 3 → Ch 5 → Ch 6), so Unit V comes *before* Unit IV in the page sequence; this index is in syllabus order. Chapter opener pages are kept because they introduce each chapter's framing (Ch 6's opener also starts §6.1, not in syllabus). **Skipped:** single page p.159 (§3.3.5 web-robot example, mid-§3.3); §3.7–3.11 (p.189+); §5.3 Hierarchical clustering and §5.4 DBSCAN (p.337–352); single page p.368 (§5.5.4 hierarchical evaluation); §5.6–5.7 notes/ exercises (p.380+); §6.1–6.2 (p.396–409); §6.5 Bayesian networks (p.430–444); §6.7–6.9 ANN, deep learning, SVM (p.452–497); single page p.503 (§6.10.3 bias–variance); §6.10.7 onwards (p.515+). **Shared boundary pages:** p.158, 188, 336, 367, 379, 429, 451, 498, 502 and 514 also carry the start of an unassigned section; p.353, 410, 445 and 504 open with the tail of one. §5.5.6 Clustering tendency (not in syllabus) sits entirely on kept pages 370–371 between two required subsections. Units I–II (Russell & Norvig) are in the companion stripped PDF.

3

Classification: Basic Concepts and Techniques

Humans have an innate ability to classify things into categories, e.g., mundane tasks such as filtering spam email messages or more specialized tasks such as recognizing celestial objects in telescope images (see Figure 3.1). While manual classification often suffices for small and simple data sets with only a few attributes, larger and more complex data sets require an automated solution.



(a) A spiral galaxy. (b) An elliptical galaxy.

Figure 3.1. Classification of galaxies from telescope images taken from the NASA website.

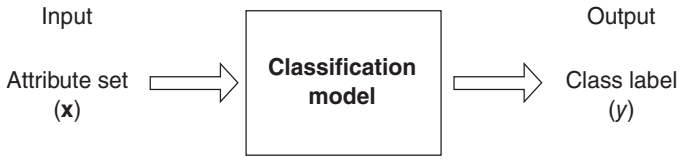


Figure 3.2. A schematic illustration of a classification task.

This chapter introduces the basic concepts of classification and describes some of its key issues such as model overfitting, model selection, and model evaluation. While these topics are illustrated using a classification technique known as decision tree induction, most of the discussion in this chapter is also applicable to other classification techniques, many of which are covered in Chapter 6.

3.1 Basic Concepts

Figure 3.2 illustrates the general idea behind classification. The data for a classification task consists of a collection of instances (records). Each such instance is characterized by the tuple $(\mathbf{x}, y)$, where $\mathbf{x}$ is the set of attribute values that describe the instance and y is the class label of the instance. The attribute set $\mathbf{x}$ can contain attributes of any type, while the class label y must be categorical.

A **classification model** is an abstract representation of the relationship between the attribute set and the class label. As will be seen in the next two chapters, the model can be represented in many ways, e.g., as a tree, a probability table, or simply, a vector of real-valued parameters. More formally, we can express it mathematically as a target function f that takes as input the attribute set $\mathbf{x}$ and produces an output corresponding to the predicted class label. The model is said to classify an instance $(\mathbf{x}, y)$ correctly if $f(\mathbf{x}) = y$.

Table 3.1. Examples of classification tasks.

Task	Attribute set	Class label
Spam filtering	Features extracted from email message header and content	spam or non-spam
Tumor identification	Features extracted from magnetic resonance imaging (MRI) scans	malignant or benign
Galaxy classification	Features extracted from telescope images	elliptical, spiral, or irregular-shaped

Table 3.2. A sample data for the vertebrate classification problem.

Vertebrate Name	Body Temperature	Skin Cover	Gives Birth	Aquatic Creature	Aerial Creature	Has Legs	Hibernates	Class Label
human	warm-blooded	hair	yes	no	no	yes	no	mammal
python	cold-blooded	scales	no	no	no	no	yes	reptile
salmon	cold-blooded	scales	no	yes	no	no	no	fish
whale	warm-blooded	hair	yes	yes	no	no	no	mammal
frog	cold-blooded	none	no	semi	no	yes	yes	amphibian
komodo dragon	cold-blooded	scales	no	no	no	yes	no	reptile
bat	warm-blooded	hair	yes	no	yes	yes	yes	mammal
pigeon	warm-blooded	feathers	no	no	yes	yes	no	bird
cat	warm-blooded	fur	yes	no	no	yes	no	mammal
leopard shark	cold-blooded	scales	yes	yes	no	no	no	fish
turtle	cold-blooded	scales	no	semi	no	yes	no	reptile
penguin	warm-blooded	feathers	no	semi	no	yes	no	bird
porcupine	warm-blooded	quills	yes	no	no	yes	yes	mammal
eel	cold-blooded	scales	no	yes	no	no	no	fish
salamander	cold-blooded	none	no	semi	no	yes	yes	amphibian

Table 3.1 shows examples of attribute sets and class labels for various classification tasks. Spam filtering and tumor identification are examples of binary classification problems, in which each data instance can be categorized into one of two classes. If the number of classes is larger than 2, as in the galaxy classification example, then it is called a multiclass classification problem.

We illustrate the basic concepts of classification in this chapter with the following two examples.

Example 3.1. [Vertebrate Classification] Table 3.2 shows a sample data set for classifying vertebrates into mammals, reptiles, birds, fishes, and amphibians. The attribute set includes characteristics of the vertebrate such as its body temperature, skin cover, and ability to fly. The data set can also be used for a binary classification task such as mammal classification, by grouping the reptiles, birds, fishes, and amphibians into a single category called non-mammals. ■

Example 3.2. [Loan Borrower Classification] Consider the problem of predicting whether a loan borrower will repay the loan or default on the loan payments. The data set used to build the classification model is shown in Table 3.3. The attribute set includes personal information of the borrower such as marital status and annual income, while the class label indicates whether the borrower had defaulted on the loan payments. ■

Table 3.3. A sample data for the loan borrower classification problem.

ID	Home Owner	Marital Status	Annual Income	Defaulted?
1	Yes	Single	125000	No
2	No	Married	100000	No
3	No	Single	70000	No
4	Yes	Married	120000	No
5	No	Divorced	95000	Yes
6	No	Single	60000	No
7	Yes	Divorced	220000	No
8	No	Single	85000	Yes
9	No	Married	75000	No
10	No	Single	90000	Yes

A classification model serves two important roles in data mining. First, it is used as a **predictive model** to classify previously unlabeled instances. A good classification model must provide accurate predictions with a fast response time. Second, it serves as a **descriptive model** to identify the characteristics that distinguish instances from different classes. This is particularly useful for critical applications, such as medical diagnosis, where it is insufficient to have a model that makes a prediction without justifying how it reaches such a decision.

For example, a classification model induced from the vertebrate data set shown in Table 3.2 can be used to predict the class label of the following vertebrate:

Vertebrate Name	Body Temperature	Skin Cover	Gives Birth	Aquatic Creature	Aerial Creature	Has Legs	Hibernates	Class Label
gila monster	cold-blooded	scales	no	no	no	yes	yes	?

In addition, it can be used as a descriptive model to help determine characteristics that define a vertebrate as a mammal, a reptile, a bird, a fish, or an amphibian. For example, the model may identify mammals as warm-blooded vertebrates that give birth to their young.

There are several points worth noting regarding the previous example. First, although all the attributes shown in Table 3.2 are qualitative, there are no restrictions on the type of attributes that can be used as predictor variables. The class label, on the other hand, must be of nominal type. This distinguishes classification from other predictive modeling tasks such as regression, where the predicted value is often quantitative. More information about regression can be found in Appendix D.

Another point worth noting is that not all attributes may be relevant to the classification task. For example, the average length or weight of a

vertebrate may not be useful for classifying mammals, as these attributes can show same value for both mammals and non-mammals. Such an attribute is typically discarded during preprocessing. The remaining attributes might not be able to distinguish the classes by themselves, and thus, must be used in concert with other attributes. For instance, the `Body Temperature` attribute is insufficient to distinguish mammals from other vertebrates. When it is used together with `Gives Birth`, the classification of mammals improves significantly. However, when additional attributes, such as `Skin Cover` are included, the model becomes overly specific and no longer covers all mammals. Finding the optimal combination of attributes that best discriminates instances from different classes is the key challenge in building classification models.

3.2 General Framework for Classification

Classification is the task of assigning labels to unlabeled data instances and a **classifier** is used to perform such a task. A classifier is typically described in terms of a model as illustrated in the previous section. The model is created using a given a set of instances, known as the **training set**, which contains attribute values as well as class labels for each instance. The systematic approach for learning a classification model given a training set is known as a **learning algorithm**. The process of using a learning algorithm to build a classification model from the training data is known as **induction**. This process is also often described as "learning a model" or "building a model." This process of applying a classification model on unseen test instances to predict their class labels is known as **deduction**. Thus, the process of classification involves two steps: applying a learning algorithm to training data to learn a model, and then applying the model to assign labels to unlabeled instances. Figure 3.3 illustrates the general framework for classification.

A **classification technique** refers to a general approach to classification, e.g., the decision tree technique that we will study in this chapter. This classification technique like most others, consists of a family of related models and a number of algorithms for learning these models. In Chapter 6, we will study additional classification techniques, including neural networks and support vector machines.

A couple notes on terminology. First, the terms "classifier" and "model" are often taken to be synonymous. If a classification technique builds a single, global model, then this is fine. However, while every model defines a classifier, not every classifier is defined by a single model. Some classifiers, such as k-nearest neighbor classifiers, do not build an explicit model (Section 6.3), while

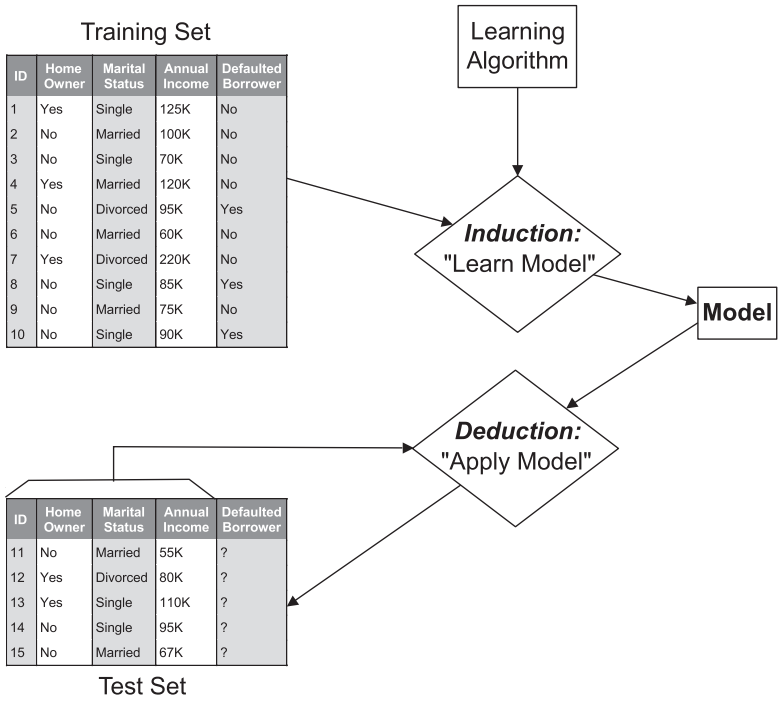


Figure 3.3. General framework for building a classification model.

other classifiers, such as ensemble classifiers, combine the output of a collection of models (Section 6.10). Second, the term "classifier" is often used in a more general sense to refer to a classification technique. Thus, for example, "decision tree classifier" can refer to the decision tree classification technique or a specific classifier built using that technique. Fortunately, the meaning of "classifier" is usually clear from the context.

In the general framework shown in Figure 3.3, the induction and deduction steps should be performed separately. In fact, as will be discussed later in Section 3.6, the training and test sets should be independent of each other to ensure that the induced model can accurately predict the class labels of instances it has never encountered before. Models that deliver such predictive insights are said to have good **generalization performance**. The performance of a model (classifier) can be evaluated by comparing the predicted labels against the true labels of instances. This information can be summarized in a table called a **confusion matrix**. Table 3.4 depicts the confusion matrix for a binary classification problem. Each entry f_{ij} denotes the number of instances from class i predicted to be of class j. For example, f_{01} is the

Table 3.4. Confusion matrix for a binary classification problem.

		Predicted Class	
		Class = 1	Class = 0
Actual Class	Class = 1	f_{11}	f_{10}
	Class = 0	f_{01}	f_{00}

number of instances from class 0 incorrectly predicted as class 1. The number of correct predictions made by the model is $(f_{11} + f_{00})$ and the number of incorrect predictions is $(f_{10} + f_{01})$.

Although a confusion matrix provides the information needed to determine how well a classification model performs, summarizing this information into a single number makes it more convenient to compare the relative performance of different models. This can be done using an **evaluation metric** such as **accuracy**, which is computed in the following way:

$$\text{Accuracy} = \frac{\text{Number of correct predictions}}{\text{Total number of predictions}}. \tag{3.1}$$

For binary classification problems, the accuracy of a model is given by

$$\text{Accuracy} = \frac{f_{11} + f_{00}}{f_{11} + f_{10} + f_{01} + f_{00}}. \tag{3.2}$$

Error rate is another related metric, which is defined as follows for binary classification problems:

$$\text{Error rate} = \frac{\text{Number of wrong predictions}}{\text{Total number of predictions}} = \frac{f_{10} + f_{01}}{f_{11} + f_{10} + f_{01} + f_{00}}. \tag{3.3}$$

The learning algorithms of most classification techniques are designed to learn models that attain the highest accuracy, or equivalently, the lowest error rate when applied to the test set. We will revisit the topic of model evaluation in Section 3.6.

3.3 Decision Tree Classifier

This section introduces a simple classification technique known as the **decision tree** classifier. To illustrate how a decision tree works, consider the classification problem of distinguishing mammals from non-mammals using

the vertebrate data set shown in Table 3.2. Suppose a new species is discovered by scientists. How can we tell whether it is a mammal or a non-mammal? One approach is to pose a series of questions about the characteristics of the species. The first question we may ask is whether the species is cold- or warm-blooded. If it is cold-blooded, then it is definitely not a mammal. Otherwise, it is either a bird or a mammal. In the latter case, we need to ask a follow-up question: Do the females of the species give birth to their young? Those that do give birth are definitely mammals, while those that do not are likely to be non-mammals (with the exception of egg-laying mammals such as the platypus and spiny anteater).

The previous example illustrates how we can solve a classification problem by asking a series of carefully crafted questions about the attributes of the test instance. Each time we receive an answer, we could ask a follow-up question until we can conclusively decide on its class label. The series of questions and their possible answers can be organized into a hierarchical structure called a decision tree. Figure 3.4 shows an example of the decision tree for the mammal classification problem. The tree has three types of nodes:

- A **root node**, with no incoming links and zero or more outgoing links.
- **Internal nodes**, each of which has exactly one incoming link and two or more outgoing links.
- **Leaf** or **terminal** nodes, each of which has exactly one incoming link and no outgoing links.

Every leaf node in the decision tree is associated with a class label. The **non-terminal** nodes, which include the root and internal nodes, contain **attribute test conditions** that are typically defined using a single attribute. Each possible outcome of the attribute test condition is associated with exactly one child of this node. For example, the root node of the tree shown in Figure 3.4 uses the attribute `Body Temperature` to define an attribute test condition that has two outcomes, warm and cold, resulting in two child nodes.

Given a decision tree, classifying a test instance is straightforward. Starting from the root node, we apply its attribute test condition and follow the appropriate branch based on the outcome of the test. This will lead us either to another internal node, for which a new attribute test condition is applied, or to a leaf node. Once a leaf node is reached, we assign the class label associated with the node to the test instance. As an illustration, Figure 3.5 traces the path used to predict the class label of a flamingo. The path terminates at a leaf node labeled as `Non-mammals`.

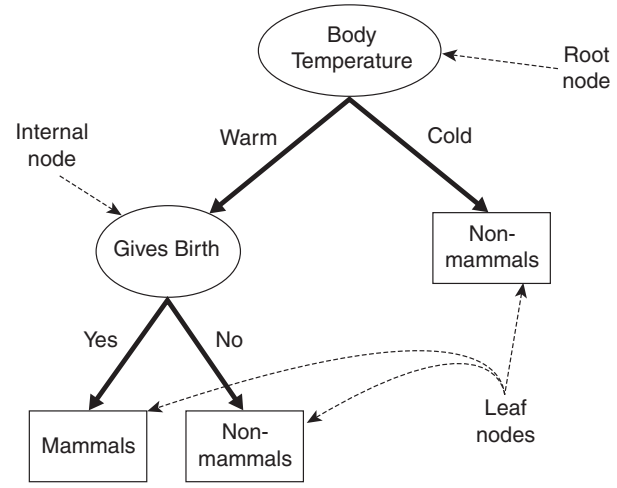


Figure 3.4. A decision tree for the mammal classification problem.

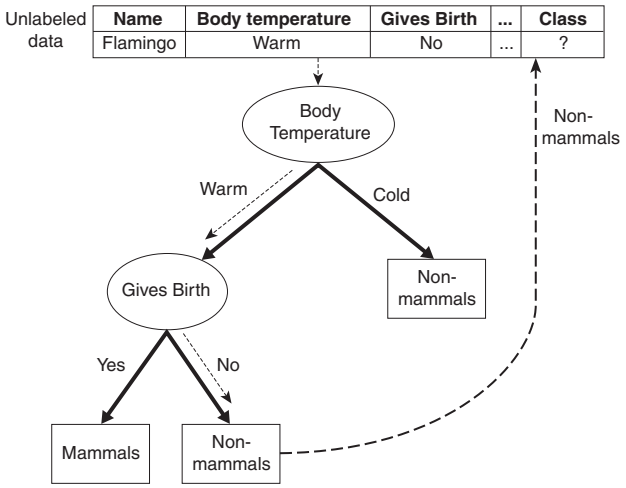


Figure 3.5. Classifying an unlabeled vertebrate. The dashed lines represent the outcomes of applying various attribute test conditions on the unlabeled vertebrate. The vertebrate is eventually assigned to the `Non-mammals` class.

3.3.1 A Basic Algorithm to Build a Decision Tree

Many possible decision trees that can be constructed from a particular data set. While some trees are better than others, finding an optimal one is computationally expensive due to the exponential size of the search space. Efficient algorithms have been developed to induce a reasonably accurate, albeit

suboptimal, decision tree in a reasonable amount of time. These algorithms usually employ a greedy strategy to grow the decision tree in a top-down fashion by making a series of locally optimal decisions about which attribute to use when partitioning the training data. One of the earliest method is **Hunt's algorithm**, which is the basis for many current implementations of decision tree classifiers, including ID3, C4.5, and CART. This subsection presents Hunt's algorithm and describes some of the design issues that must be considered when building a decision tree.

Hunt's Algorithm

In Hunt's algorithm, a decision tree is grown in a recursive fashion. The tree initially contains a single root node that is associated with all the training instances. If a node is associated with instances from more than one class, it is expanded using an attribute test condition that is determined using a **splitting criterion**. A child leaf node is created for each outcome of the attribute test condition and the instances associated with the parent node are distributed to the children based on the test outcomes. This node expansion step can then be recursively applied to each child node, as long as it has labels of more than one class. If all the instances associated with a leaf node have identical class labels, then the node is not expanded any further. Each leaf node is assigned a class label that occurs most frequently in the training instances associated with the node.

To illustrate how the algorithm works, consider the training set shown in Table 3.3 for the loan borrower classification problem. Suppose we apply Hunt's algorithm to fit the training data. The tree initially contains only a single leaf node as shown in Figure 3.6(a). This node is labeled as `Defaulted = No`, since the majority of the borrowers did not default on their loan payments. The training error of this tree is 30% as three out of the ten training instances have the class label `Defaulted = Yes`. The leaf node can therefore be further expanded because it contains training instances from more than one class.

Let `Home Owner` be the attribute chosen to split the training instances. The justification for choosing this attribute as the attribute test condition will be discussed later. The resulting binary split on the `Home Owner` attribute is shown in Figure 3.6(b). All the training instances for which `Home Owner = Yes` are propagated to the left child of the root node and the rest are propagated to the right child. Hunt's algorithm is then recursively applied to each child. The left child becomes a leaf node labeled `Defaulted = No`, since all instances associated with this node have identical class label `Defaulted = No`. The right child has instances from each class label. Hence, we split it

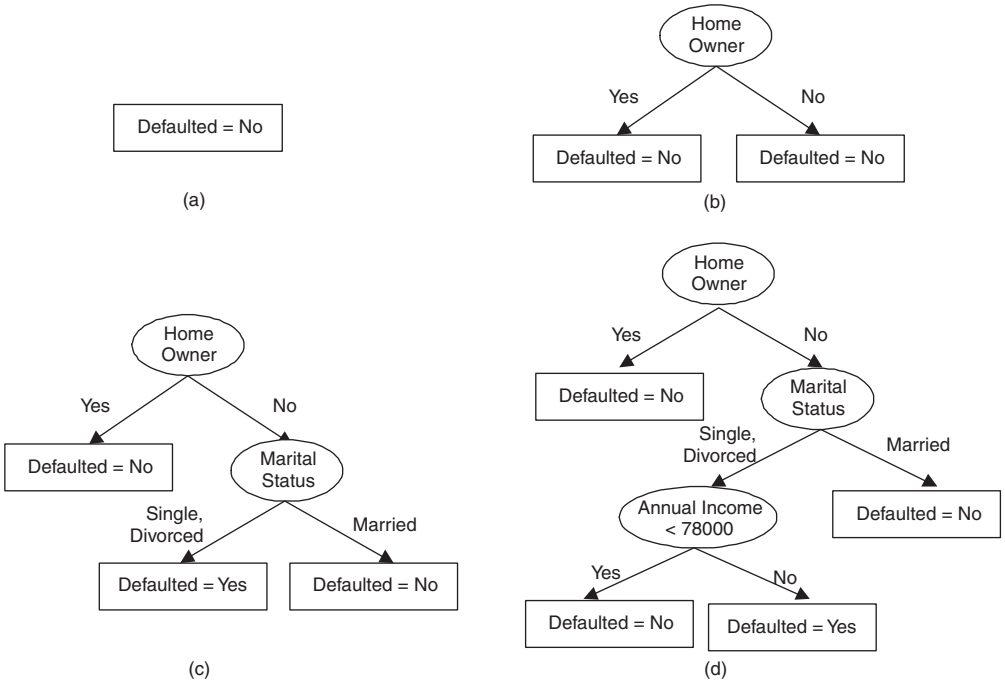


Figure 3.6. Hunt's algorithm for building decision trees.

further. The resulting subtrees after recursively expanding the right child are shown in Figures 3.6(c) and (d).

Hunt's algorithm, as described above, makes some simplifying assumptions that are often not true in practice. In the following, we describe these assumptions and briefly discuss some of the possible ways for handling them.

1. Some of the child nodes created in Hunt's algorithm can be empty if none of the training instances have the particular attribute values. One way to handle this is by declaring each of them as a leaf node with a class label that occurs most frequently among the training instances associated with their parent nodes.

2. If all training instances associated with a node have identical attribute values but different class labels, it is not possible to expand this node any further. One way to handle this case is to declare it a leaf node and assign it the class label that occurs most frequently in the training instances associated with this node.

Design Issues of Decision Tree Induction

Hunt's algorithm is a generic procedure for growing decision trees in a greedy fashion. To implement the algorithm, there are two key design issues that must be addressed.

1. **What is the splitting criterion?** At each recursive step, an attribute must be selected to partition the training instances associated with a node into smaller subsets associated with its child nodes. The splitting criterion determines which attribute is chosen as the test condition and how the training instances should be distributed to the child nodes. This will be discussed in Sections 3.3.2 and 3.3.3.

2. **What is the stopping criterion?** The basic algorithm stops expanding a node only when all the training instances associated with the node have the same class labels or have identical attribute values. Although these conditions are sufficient, there are reasons to stop expanding a node much earlier even if the leaf node contains training instances from more than one class. This process is called early termination and the condition used to determine when a node should be stopped from expanding is called a stopping criterion. The advantages of early termination are discussed in Section 3.4.

3.3.2 Methods for Expressing Attribute Test Conditions

Decision tree induction algorithms must provide a method for expressing an attribute test condition and its corresponding outcomes for different attribute types.

Binary Attributes The test condition for a binary attribute generates two potential outcomes, as shown in Figure 3.7.

Nominal Attributes Since a nominal attribute can have many values, its attribute test condition can be expressed in two ways, as a multiway split or a binary split as shown in Figure 3.8. For a multiway split (Figure 3.8(a)), the number of outcomes depends on the number of distinct values for the corresponding attribute. For example, if an attribute such as marital status has three distinct values—single, married, or divorced—its test condition will produce a three-way split. It is also possible to create a binary split by partitioning all values taken by the nominal attribute into two groups. For example, some decision tree algorithms, such as CART, produce only binary

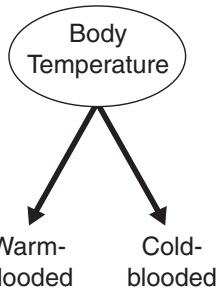


Figure 3.7. Attribute test condition for a binary attribute.

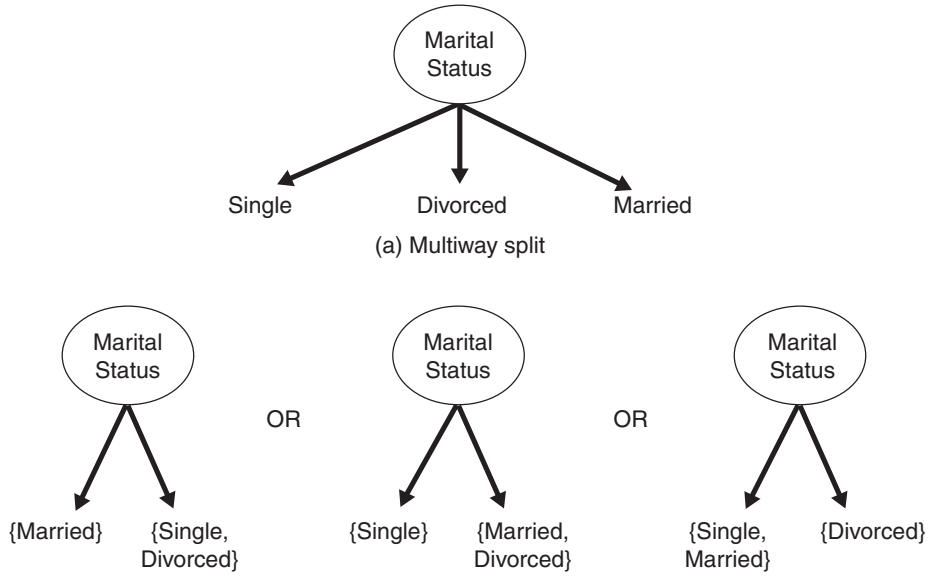


Figure 3.8. Attribute test conditions for nominal attributes.

splits by considering all $2^{k-1} - 1$ ways of creating a binary partition of k attribute values. Figure 3.8(b) illustrates three different ways of grouping the attribute values for marital status into two subsets.

Ordinal Attributes Ordinal attributes can also produce binary or multiway splits. Ordinal attribute values can be grouped as long as the grouping

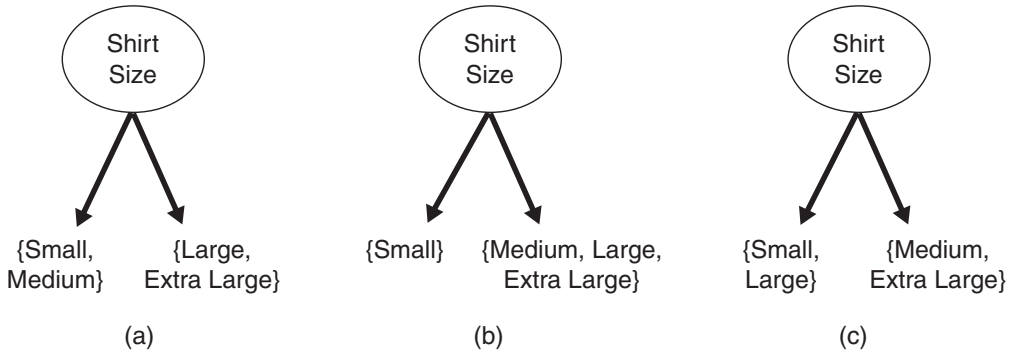


Figure 3.9. Different ways of grouping ordinal attribute values.

does not violate the order property of the attribute values. Figure 3.9 illustrates various ways of splitting training records based on the `Shirt Size` attribute. The groupings shown in Figures 3.9(a) and (b) preserve the order among the attribute values, whereas the grouping shown in Figure 3.9(c) violates this property because it combines the attribute values `Small` and `Large` into the same partition while `Medium` and `Extra Large` are combined into another partition.

Continuous Attributes For continuous attributes, the attribute test condition can be expressed as a comparison test (e.g., $A < v$) producing a binary split, or as a range query of the form $v_i \leq A < v_{i+1}$, for $i = 1, \ldots, k$, producing a multiway split. The difference between these approaches is shown in Figure 3.10. For the binary split, any possible value v between the minimum and maximum attribute values in the training data can be used for constructing the comparison test $A < v$. However, it is sufficient to only consider distinct attribute values in the training set as candidate split positions. For the multiway split, any possible collection of attribute value ranges can be used, as long as they are mutually exclusive and cover the entire range of attribute values between the minimum and maximum values observed in the training set. One approach for constructing multiway splits is to apply the discretization strategies described in Section 2.3.6 on page 83. After discretization, a new ordinal value is assigned to each discretized interval, and the attribute test condition is then defined using this newly constructed ordinal attribute.

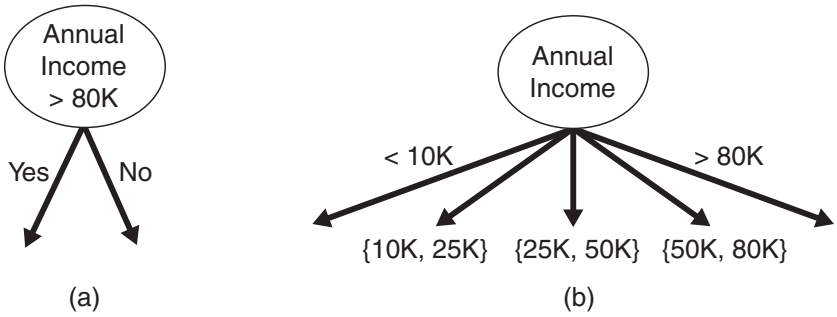


Figure 3.10. Test condition for continuous attributes.

3.3.3 Measures for Selecting an Attribute Test Condition

There are many measures that can be used to determine the goodness of an attribute test condition. These measures try to give preference to attribute test conditions that partition the training instances into *purer* subsets in the child nodes, which mostly have the same class labels. Having purer nodes is useful since a node that has all of its training instances from the same class does not need to be expanded further. In contrast, an impure node containing training instances from multiple classes is likely to require several levels of node expansions, thereby increasing the depth of the tree considerably. Larger trees are less desirable as they are more susceptible to model overfitting, a condition that may degrade the classification performance on unseen instances, as will be discussed in Section 3.4. They are also difficult to interpret and incur more training and test time as compared to smaller trees.

In the following, we present different ways of measuring the impurity of a node and the collective impurity of its child nodes, both of which will be used to identify the best attribute test condition for a node.

Impurity Measure for a Single Node

The impurity of a node measures how dissimilar the class labels are for the data instances belonging to a common node. Following are examples of measures

that can be used to evaluate the impurity of a node t:

$$\text{Entropy} = -\sum_{i=0}^{c-1} p_i(t) \log_2 p_i(t), \tag{3.4}$$

$$\text{Gini index} = 1 - \sum_{i=0}^{c-1} p_i(t)^2, \tag{3.5}$$

$$\text{Classification error} = 1 - \max_i [p_i(t)], \tag{3.6}$$

where $p_i(t)$ is the relative frequency of training instances that belong to class i at node t, c is the total number of classes, and $0 \log_2 0 = 0$ in entropy calculations. All three measures give a zero impurity value if a node contains instances from a single class and maximum impurity if the node has equal proportion of instances from multiple classes.

Figure 3.11 compares the relative magnitude of the impurity measures when applied to binary classification problems. Since there are only two classes, $p_0(t) + p_1(t) = 1$. The horizontal axis p refers to the fraction of instances that belong to one of the two classes. Observe that all three measures attain their maximum value when the class distribution is uniform (i.e., $p_0(t) = p_1(t) = 0.5$) and minimum value when all the instances belong to a single class (i.e., either $p_0(t)$ or $p_1(t)$ equals to 1). The following examples illustrate how the values of the impurity measures vary as we alter the class distribution.

Node N_1	Count
Class=0	0
Class=1	6

Gini $= 1 - (0/6)^2 - (6/6)^2 = 0$
Entropy $= -(0/6) \log_2(0/6) - (6/6) \log_2(6/6) = 0$
Error $= 1 - \max[0/6, 6/6] = 0$

Node N_2	Count
Class=0	1
Class=1	5

Gini $= 1 - (1/6)^2 - (5/6)^2 = 0.278$
Entropy $= -(1/6) \log_2(1/6) - (5/6) \log_2(5/6) = 0.650$
Error $= 1 - \max[1/6, 5/6] = 0.167$

Node N_3	Count
Class=0	3
Class=1	3

Gini $= 1 - (3/6)^2 - (3/6)^2 = 0.5$
Entropy $= -(3/6) \log_2(3/6) - (3/6) \log_2(3/6) = 1$
Error $= 1 - \max[3/6, 3/6] = 0.5$

Based on these calculations, node N_1 has the lowest impurity value, followed by N_2 and N_3. This example, along with Figure 3.11, shows the consistency among the impurity measures, i.e., if a node N_1 has lower entropy than node N_2, then the Gini index and error rate of N_1 will also be lower than that

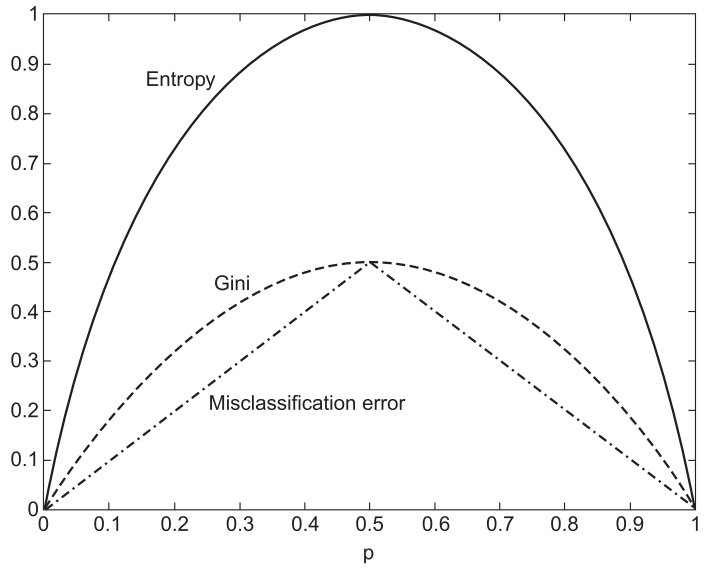


Figure 3.11. Comparison among the impurity measures for binary classification problems.

of N_2. Despite their agreement, the attribute chosen as splitting criterion by the impurity measures can still be different (see Exercise 7 on pages 206–207).

Collective Impurity of Child Nodes

Consider an attribute test condition that splits a node containing N training instances into k children, $\{v_1, v_2, \cdots, v_k\}$, where every child node represents a partition of the data resulting from one of the k outcomes of the attribute test condition. Let $N(v_j)$ be the number of training instances associated with a child node v_j, whose impurity value is $I(v_j)$. Since a training instance in the parent node reaches node v_j for a fraction of $N(v_j)/N$ times, the collective impurity of the child nodes can be computed by taking a weighted sum of the impurities of the child nodes, as follows:

$$I(\text{children}) = \sum_{j=1}^{k} \frac{N(v_j)}{N} I(v_j), \tag{3.7}$$

Example 3.3. [Weighted Entropy] Consider the candidate attribute test condition shown in Figures 3.12(a) and (b) for the loan borrower classification problem. Splitting on the `Home Owner` attribute will generate two child nodes

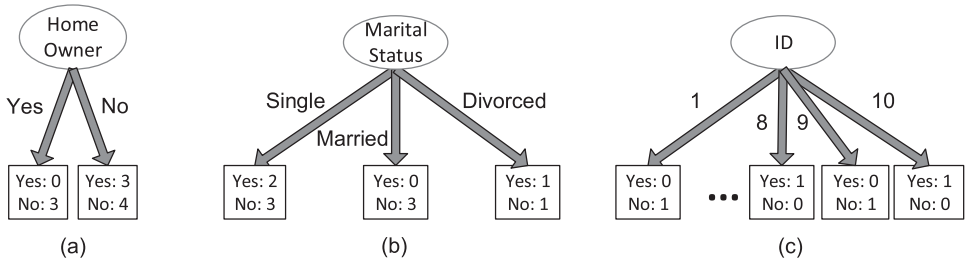


Figure 3.12. Examples of candidate attribute test conditions.

whose weighted entropy can be calculated as follows:

$$I(\texttt{Home Owner = yes}) = -\frac{0}{3}\log_2\frac{0}{3} - \frac{3}{3}\log_2\frac{3}{3} = 0$$
$$I(\texttt{Home Owner = no}) = -\frac{3}{7}\log_2\frac{3}{7} - \frac{4}{7}\log_2\frac{4}{7} = 0.985$$
$$I(\texttt{Home Owner}) = \frac{3}{10}\times 0 + \frac{7}{10}\times 0.985 = 0.690$$

Splitting on `Marital Status`, on the other hand, leads to three child nodes with a weighted entropy given by

$$I(\texttt{Marital Status = Single}) = -\frac{2}{5}\log_2\frac{2}{5} - \frac{3}{5}\log_2\frac{3}{5} = 0.971$$
$$I(\texttt{Marital Status = Married}) = -\frac{0}{3}\log_2\frac{0}{3} - \frac{3}{3}\log_2\frac{3}{3} = 0$$
$$I(\texttt{Marital Status = Divorced}) = -\frac{1}{2}\log_2\frac{1}{2} - \frac{1}{2}\log_2\frac{1}{2} = 1.000$$
$$I(\texttt{Marital Status}) = \frac{5}{10}\times 0.971 + \frac{3}{10}\times 0 + \frac{2}{10}\times 1 = 0.686$$

Thus, `Marital Status` has a lower weighted entropy than `Home Owner`. ■

Identifying the best attribute test condition

To determine the goodness of an attribute test condition, we need to compare the degree of impurity of the parent node (before splitting) with the weighted degree of impurity of the child nodes (after splitting). The larger their difference, the better the test condition. This difference, Δ, also termed as the **gain** in purity of an attribute test condition, can be defined as follows:

$$\Delta = I(\text{parent}) - I(\text{children}), \tag{3.8}$$

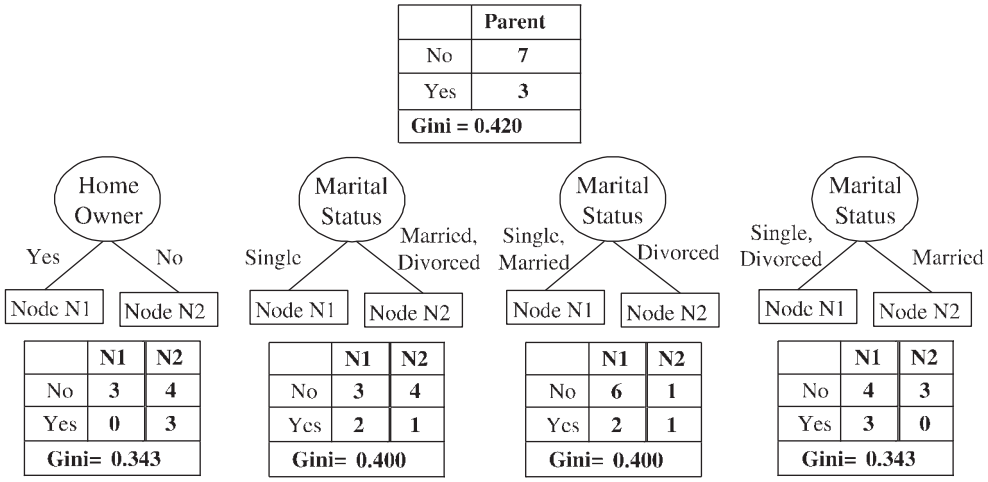


Figure 3.13. Splitting criteria for the loan borrower classification problem using Gini index.

where $I(\text{parent})$ is the impurity of a node before splitting and $I(\text{children})$ is the weighted impurity measure after splitting. It can be shown that the gain is non-negative since $I(\text{parent}) \geq I(\text{children})$ for any reasonable measure such as those presented above. The higher the gain, the purer are the classes in the child nodes relative to the parent node. The splitting criterion in the decision tree learning algorithm selects the attribute test condition that shows the maximum gain. Note that maximizing the gain at a given node is equivalent to minimizing the weighted impurity measure of its children since $I(\text{parent})$ is the same for all candidate attribute test conditions. Finally, when entropy is used as the impurity measure, the difference in entropy is commonly known as **information gain**, Δ_{info}.

In the following, we present illustrative approaches for identifying the best attribute test condition given qualitative or quantitative attributes.

Splitting of Qualitative Attributes

Consider the first two candidate splits shown in Figure 3.12 involving qualitative attributes `Home Owner` and `Marital Status`. The initial class distribution at the parent node is $(0.3, 0.7)$, since there are 3 instances of class `Yes` and 7 instances of class `No` in the training data. Thus,

$$I(\text{parent}) = -\frac{3}{10}\log_2\frac{3}{10} - \frac{7}{10}\log_2\frac{7}{10} = 0.881$$

The information gains for `Home Owner` and `Marital Status` are each given by

$$\begin{aligned} \Delta_{\text{info}}(\texttt{Home Owner}) &= 0.881 - 0.690 = 0.191 \\ \Delta_{\text{info}}(\texttt{Marital Status}) &= 0.881 - 0.686 = 0.195 \end{aligned}$$

The information gain for `Marital Status` is thus higher due to its lower weighted entropy, which will thus be considered for splitting.

Binary Splitting of Qualitative Attributes

Consider building a decision tree using only binary splits and the Gini index as the impurity measure. Figure 3.13 shows examples of four candidate splitting criteria for the `Home Owner` and `Marital Status` attributes. Since there are 3 borrowers in the training set who defaulted and 7 others who repaid their loan (see Table in Figure 3.13), the Gini index of the parent node before splitting is

$$1 - \left(\frac{3}{10}\right)^2 - \left(\frac{7}{10}\right)^2 = 0.420.$$

If `Home Owner` is chosen as the splitting attribute, the Gini index for the child nodes $\texttt{N}_1$ and $\texttt{N}_2$ are 0 and 0.490, respectively. The weighted average Gini index for the children is

$$(3/10) \times 0 + (7/10) \times 0.490 = 0.343,$$

where the weights represent the proportion of training instances assigned to each child. The gain using `Home Owner` as splitting attribute is 0.420 - 0.343 = 0.077. Similarly, we can apply a binary split on the `Marital Status` attribute. However, since `Marital Status` is a nominal attribute with three outcomes, there are three possible ways to group the attribute values into a binary split. The weighted average Gini index of the children for each candidate binary split is shown in Figure 3.13. Based on these results, `Home Owner` and the last binary split using `Marital Status` are clearly the best candidates, since they both produce the lowest weighted average Gini index. Binary splits can also be used for ordinal attributes, if the binary partitioning of the attribute values does not violate the ordering property of the values.

Binary Splitting of Quantitative Attributes

Consider the problem of identifying the best binary split $\texttt{Annual Income} \leq \tau$ for the preceding loan approval classification problem. As discussed previously,

Class	No	No	No	Yes	Yes	Yes	No	No	No	No
	Annual Income (in '000s)									
Sorted Values →	60	70	75	85	90	95	100	120	125	220
Split Positions →	55	65	72.5	80	87.5	92.5	97.5	110	122.5	172.5	230
	<= >	<= >	<= >	<= >	<= >	<= >	<= >	<= >	<= >	<= >	<= >
Yes	0 3	0 3	0 3	0 3	1 2	2 1	3 0	3 0	3 0	3 0	3 0
No	0 7	1 6	2 5	3 4	3 4	3 4	3 4	4 3	5 2	6 1	7 0
Gini	0.420	0.400	0.375	0.343	0.417	0.400	***0.300***	0.343	0.375	0.400	0.420

Figure 3.14. Splitting continuous attributes.

even though τ can take any value between the minimum and maximum values of annual income in the training set, it is sufficient to only consider the annual income values observed in the training set as candidate split positions. For each candidate τ, the training set is scanned once to count the number of borrowers with annual income less than or greater than τ along with their class proportions. We can then compute the Gini index at each candidate split position and choose the τ that produces the lowest value. Computing the Gini index at each candidate split position requires $O(N)$ operations, where N is the number of training instances. Since there are at most N possible candidates, the overall complexity of this brute-force method is $O(N^2)$. It is possible to reduce the complexity of this problem to $O(N \log N)$ by using a method described as follows (see illustration in Figure 3.14). In this method, we first sort the training instances based on their annual income, a one-time cost that requires $O(N \log N)$ operations. The candidate split positions are given by the midpoints between every two adjacent sorted values: \$55,000, \$65,000, \$72,500, and so on. For the first candidate, since none of the instances has an annual income less than or equal to \$55,000, the Gini index for the child node with `Annual Income` $<$ \$55,000 is equal to zero. In contrast, there are 3 training instances of class `Yes` and 7 instances of class `No` with annual income greater than \$55,000. The Gini index for this node is 0.420. The weighted average Gini index for the first candidate split position, $\tau = \$55,000$, is equal to $0 \times 0 + 1 \times 0.420 = 0.420$.

For the next candidate, $\tau = \$65,000$, the class distribution of its child nodes can be obtained with a simple update of the distribution for the previous candidate. This is because, as τ increases from \$55,000 to \$65,000, there is only one training instance affected by the change. By examining the class label of the affected training instance, the new class distribution is obtained. For example, as τ increases to \$65,000, there is only one borrower in the training

set, with an annual income of $60,000, affected by this change. Since the class label for the borrower is `No`, the count for class `No` increases from 0 to 1 (for `Annual Income` $\leq$ $65,000) and decreases from 7 to 6 (for `Annual Income` $>$ $65,000), as shown in Figure 3.14. The distribution for the `Yes` class remains unaffected. The updated Gini index for this candidate split position is 0.400.

This procedure is repeated until the Gini index for all candidates are found. The best split position corresponds to the one that produces the lowest Gini index, which occurs at $\tau = \$97,500$. Since the Gini index at each candidate split position can be computed in $O(1)$ time, the complexity of finding the best split position is $O(N)$ once all the values are kept sorted, a one-time operation that takes $O(N \log N)$ time. The overall complexity of this method is thus $O(N \log N)$, which is much smaller than the $O(N^2)$ time taken by the brute-force method. The amount of computation can be further reduced by considering only candidate split positions located between two adjacent sorted instances with different class labels. For example, we do not need to consider candidate split positions located between $60,000 and $75,000 because all three instances with annual income in this range ($60,000, $70,000, and $75,000) have the same class labels. Choosing a split position within this range only increases the degree of impurity, compared to a split position located outside this range. Therefore, the candidate split positions at $\tau = \$65,000$ and $\tau = \$72,500$ can be ignored. Similarly, we do not need to consider the candidate split positions at $87,500, $92,500, $110,000, $122,500, and $172,500 because they are located between two adjacent instances with the same labels. This strategy reduces the number of candidate split positions to consider from 9 to 2 (excluding the two boundary cases $\tau = \$55,000$ and $\tau = \$230,000$).

Gain Ratio

One potential limitation of impurity measures such as entropy and Gini index is that they tend to favor qualitative attributes with large number of distinct values. Figure 3.12 shows three candidate attributes for partitioning the data set given in Table 3.3. As previously mentioned, the attribute `Marital Status` is a better choice than the attribute `Home Owner`, because it provides a larger information gain. However, if we compare them against `Customer ID`, the latter produces the purest partitions with the maximum information gain, since the weighted entropy and Gini index is equal to zero for its children. Yet, `Customer ID` is not a good attribute for splitting because it has a unique value for each instance. Even though a test condition involving `Customer ID` will accurately classify every instance in the training data, we cannot use such a test condition on new test instances with `Customer ID` values that haven't

been seen before during training. This example suggests having a low impurity value alone is insufficient to find a good attribute test condition for a node. As we will see later in Section 3.4, having more number of child nodes can make a decision tree more complex and consequently more susceptible to overfitting. Hence, the number of children produced by the splitting attribute should also be taken into consideration while deciding the best attribute test condition.

There are two ways to overcome this problem. One way is to generate only binary decision trees, thus avoiding the difficulty of handling attributes with varying number of partitions. This strategy is employed by decision tree classifiers such as CART. Another way is to modify the splitting criterion to take into account the number of partitions produced by the attribute. For example, in the C4.5 decision tree algorithm, a measure known as **gain ratio** is used to compensate for attributes that produce a large number of child nodes. This measure is computed as follows:

$$\text{Gain ratio} = \frac{\Delta_{\text{info}}}{\text{Split Info}} = \frac{\text{Entropy(Parent)} - \sum_{i=1}^{k} \frac{N(v_i)}{N}\text{Entropy}(v_i)}{-\sum_{i=1}^{k} \frac{N(v_i)}{N} \log_2 \frac{N(v_i)}{N}} \quad (3.9)$$

where $N(v_i)$ is the number of instances assigned to node v_i and k is the total number of splits. The split information measures the entropy of splitting a node into its child nodes and evaluates if the split results in a larger number of equally-sized child nodes or not. For example, if every partition has the same number of instances, then $\forall i : N(v_i)/N = 1/k$ and the split information would be equal to $\log_2 k$. Thus, if an attribute produces a large number of splits, its split information is also large, which in turn, reduces the gain ratio.

Example 3.4. [Gain Ratio] Consider the data set given in Exercise 3 on page 205. We want to select the best attribute test condition among the following three attributes: `Gender`, `Car Type`, and `Customer ID`. The entropy before splitting is

$$\text{Entropy(parent)} = -\frac{10}{20}\log_2\frac{10}{20} - \frac{10}{20}\log_2\frac{10}{20} = 1.$$

If `Gender` is used as attribute test condition:

$$\begin{aligned} \text{Entropy(children)} &= \frac{10}{20}\left[-\frac{6}{10}\log_2\frac{6}{10} - \frac{4}{10}\log_2\frac{4}{10}\right] \times 2 = 0.971 \\ \text{Gain Ratio} &= \frac{1 - 0.971}{-\frac{10}{20}\log_2\frac{10}{20} - \frac{10}{20}\log_2\frac{10}{20}} = \frac{0.029}{1} = 0.029 \end{aligned}$$

If `Car Type` is used as attribute test condition:

$$\begin{aligned}\text{Entropy(children)} &= \frac{4}{20}\left[-\frac{1}{4}\log_2\frac{1}{4}-\frac{3}{4}\log_2\frac{3}{4}\right]+\frac{8}{20}\times 0 \\ &\quad +\frac{8}{20}\left[-\frac{1}{8}\log_2\frac{1}{8}-\frac{7}{8}\log_2\frac{7}{8}\right]=0.380 \\ \text{Gain Ratio} &= \frac{1-0.380}{-\frac{4}{20}\log_2\frac{4}{20}-\frac{8}{20}\log_2\frac{8}{20}-\frac{8}{20}\log_2\frac{8}{20}}=\frac{0.620}{1.52}=0.41\end{aligned}$$

Finally, if `Customer ID` is used as attribute test condition:

$$\begin{aligned}\text{Entropy(children)} &= \frac{1}{20}\left[-\frac{1}{1}\log_2\frac{1}{1}-\frac{0}{1}\log_2\frac{0}{1}\right]\times 20=0 \\ \text{Gain Ratio} &= \frac{1-0}{-\frac{1}{20}\log_2\frac{1}{20}\times 20}=\frac{1}{4.32}=0.23\end{aligned}$$

Thus, even though `Customer ID` has the highest information gain, its gain ratio is lower than `Car Type` since it produces a larger number of splits. ■

3.3.4 Algorithm for Decision Tree Induction

Algorithm 3.1 presents a pseudocode for decision tree induction algorithm. The input to this algorithm is a set of training instances E along with the attribute set F. The algorithm works by recursively selecting the best attribute to split the data (Step 7) and expanding the nodes of the tree (Steps 11 and 12) until the stopping criterion is met (Step 1). The details of this algorithm are explained below.

1. The `createNode`() function extends the decision tree by creating a new node. A node in the decision tree either has a test condition, denoted as *node.test_cond*, or a class label, denoted as *node.label*.

2. The `find_best_split`() function determines the attribute test condition for partitioning the training instances associated with a node. The splitting attribute chosen depends on the impurity measure used. The popular measures include entropy and the Gini index.

3. The `Classify`() function determines the class label to be assigned to a leaf node. For each leaf node t, let $p(i|t)$ denote the fraction of training instances from class i associated with the node t. The label assigned

Algorithm 3.1 A skeleton decision tree induction algorithm.

```
TreeGrowth (E, F)
 1: if stopping_cond(E,F) = true then
 2:    leaf = createNode().
 3:    leaf.label = Classify(E).
 4:    return leaf.
 5: else
 6:    root = createNode().
 7:    root.test_cond = find_best_split(E, F).
 8:    let V = {v|v is a possible outcome of root.test_cond }.
 9:    for each v ∈ V do
10:       E_v = {e | root.test_cond(e) = v and e ∈ E}.
11:       child = TreeGrowth(E_v, F).
12:       add child as descendent of root and label the edge (root → child) as v.
13:    end for
14: end if
15: return root.
```

to the leaf node is typically the one that occurs most frequently in the training instances that are associated with this node.

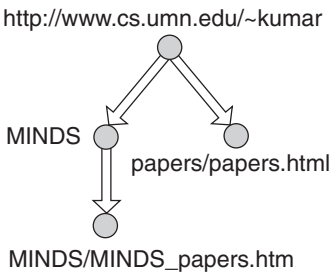
$$leaf.label = \underset{i}{\text{argmax}}\ p(i|t), \tag{3.10}$$

where the argmax operator returns the class i that maximizes $p(i|t)$. Besides providing the information needed to determine the class label of a leaf node, $p(i|t)$ can also be used as a rough estimate of the probability that an instance assigned to the leaf node t belongs to class i. Sections 6.11.2 and 6.11.4 in the next chapter describe how such probability estimates can be used to determine the performance of a decision tree under different cost functions.

4. The `stopping_cond`() function is used to terminate the tree-growing process by checking whether all the instances have identical class label or attribute values. Since decision tree classifiers employ a top-down, recursive partitioning approach for building a model, the number of training instances associated with a node decreases as the depth of the tree increases. As a result, a leaf node may contain too few training instances to make a statistically significant decision about its class label. This is known as the **data fragmentation** problem. One way to avoid this problem is to disallow splitting of a node when the number of instances associated with the node fall below a certain threshold. A

Session	IP Address	Timestamp	Request Method	Requested Web Page	Protocol	Status	Number of Bytes	Referrer	User Agent
1	160.11.11.11	08/Aug/2004 10:15:21	GET	http://www.cs.umn.edu/~kumar	HTTP/1.1	200	6424		Mozilla/4.0 (compatible; MSIE 6.0; Windows NT 5.0)
1	160.11.11.11	08/Aug/2004 10:15:34	GET	http://www.cs.umn.edu/~kumar/MINDS	HTTP/1.1	200	41378	http://www.cs.umn.edu/~kumar	Mozilla/4.0 (compatible; MSIE 6.0; Windows NT 5.0)
1	160.11.11.11	08/Aug/2004 10:15:41	GET	http://www.cs.umn.edu/~kumar/MINDS/MINDS_papers.htm	HTTP/1.1	200	1018516	http://www.cs.umn.edu/~kumar/MINDS	Mozilla/4.0 (compatible; MSIE 6.0; Windows NT 5.0)
1	160.11.11.11	08/Aug/2004 10:16:11	GET	http://www.cs.umn.edu/~kumar/papers/papers.html	HTTP/1.1	200	7463	http://www.cs.umn.edu/~kumar	Mozilla/4.0 (compatible; MSIE 6.0; Windows NT 5.0)
2	35.9.2.2	08/Aug/2004 10:16:15	GET	http://www.cs.umn.edu/~steinbac	HTTP/1.0	200	3149		Mozilla/5.0 (Windows; U; Windows NT 5.1; en-US; rv:1.7) Gecko/20040616

(a) Example of a Web server log.



(b) Graph of a Web session.

Attribute Name	Description
totalPages	Total number of pages retrieved in a Web session
ImagePages	Total number of image pages retrieved in a Web session
TotalTime	Total amount of time spent by Web site visitor
RepeatedAccess	The same page requested more than once in a Web session
ErrorRequest	Errors in requesting for Web pages
GET	Percentage of requests made using GET method
POST	Percentage of requests made using POST method
HEAD	Percentage of requests made using HEAD method
Breadth	Breadth of Web traversal
Depth	Depth of Web traversal
MultiIP	Session with multiple IP addresses
MultiAgent	Session with multiple user agents

(c) Derived attributes for Web robot detection.

Figure 3.15. Input data for web robot detection.

more systematic way to control the size of a decision tree (number of leaf nodes) will be discussed in Section 3.5.4.

3.3.5 Example Application: Web Robot Detection

Consider the task of distinguishing the access patterns of web robots from those generated by human users. A web robot (also known as a web crawler) is a software program that automatically retrieves files from one or more websites by following the hyperlinks extracted from an initial set of seed URLs. These programs have been deployed for various purposes, from gathering web pages on behalf of search engines to more malicious activities such as spamming and committing click frauds in online advertisements.

The web robot detection problem can be cast as a binary classification task. The input data for the classification task is a web server log, a sample of which is shown in Figure 3.15(a). Each line in the log file corresponds to a

shown in Figure 3.16, which has an error rate equal to 3.8% on the training set and 5.3% on the test set. In addition to its low error rate, the tree also reveals some interesting properties that can help discriminate web robots from human users:

1. Accesses by web robots tend to be broad but shallow, whereas accesses by human users tend to be more focused (narrow but deep).

2. Web robots seldom retrieve the image pages associated with a web page.

3. Sessions due to web robots tend to be long and contain a large number of requested pages.

4. Web robots are more likely to make repeated requests for the same web page than human users since the web pages retrieved by human users are often cached by the browser.

3.3.6 Characteristics of Decision Tree Classifiers

The following is a summary of the important characteristics of decision tree induction algorithms.

1. **Applicability:** Decision trees are a nonparametric approach for building classification models. This approach does not require any prior assumption about the probability distribution governing the class and attributes of the data, and thus, is applicable to a wide variety of data sets. It is also applicable to both categorical and continuous data without requiring the attributes to be transformed into a common representation via binarization, normalization, or standardization. Unlike some binary classifiers described in Chapter 6, it can also deal with multiclass problems without the need to decompose them into multiple binary classification tasks. Another appealing feature of decision tree classifiers is that the induced trees, especially the shorter ones, are relatively easy to interpret. The accuracies of the trees are also quite comparable to other classification techniques for many simple data sets.

2. **Expressiveness:** A decision tree provides a universal representation for discrete-valued functions. In other words, it can encode any function of discrete-valued attributes. This is because every discrete-valued function can be represented as an assignment table, where every unique combination of discrete attributes is assigned a class label. Since every

```
Decision Tree:
depth = 1:
|  breadth> 7 :  class 1
|  breadth<= 7:
|  |  breadth <= 3:
|  |  |  ImagePages> 0.375:  class 0
|  |  |  ImagePages<= 0.375:
|  |  |  |  totalPages<= 6:  class 1
|  |  |  |  totalPages> 6:
|  |  |  |  |  breadth <= 1:  class 1
|  |  |  |  |  breadth > 1:  class 0
|  |  width > 3:
|  |  |  MultiIP = 0:
|  |  |  |  ImagePages<= 0.1333:  class 1
|  |  |  |  ImagePages> 0.1333:
|  |  |  |  breadth <= 6:  class 0
|  |  |  |  breadth > 6:  class 1
|  |  |  MultiIP = 1:
|  |  |  |  TotalTime <= 361:  class 0
|  |  |  |  TotalTime > 361:  class 1
depth> 1:
|  MultiAgent = 0:
|  |  depth > 2:  class 0
|  |  depth < 2:
|  |  |  MultiIP = 1:  class 0
|  |  |  MultiIP = 0:
|  |  |  |  breadth <= 6:  class 0
|  |  |  |  breadth > 6:
|  |  |  |  |  RepeatedAccess <= 0.322:  class 0
|  |  |  |  |  RepeatedAccess > 0.322:  class 1
|  MultiAgent = 1:
|  |  totalPages <= 81:  class 0
|  |  totalPages > 81:  class 1
```

Figure 3.16. Decision tree model for web robot detection.

combination of attributes can be represented as a leaf in the decision tree, we can always find a decision tree whose label assignments at the leaf nodes matches with the assignment table of the original function. Decision trees can also help in providing compact representations of functions when some of the unique combinations of attributes can be represented by the same leaf node. For example, Figure 3.17 shows the assignment table of the Boolean function $(A \wedge B) \vee (C \wedge D)$ involving four binary attributes, resulting in a total of $2^4 = 16$ possible assignments. The tree shown in Figure 3.17 shows a compressed encoding of this assignment table. Instead of requiring a fully-grown tree with 16 leaf nodes, it is possible to encode the function using a simpler tree with only 7 leaf nodes. Nevertheless, not all decision trees for discrete-valued attributes can be simplified. One notable example is the parity function,

A	B	C	D	class
0	0	0	0	0
0	0	0	1	0
0	0	1	0	0
0	0	1	1	1
0	1	0	0	0
0	1	0	1	0
0	1	1	0	0
0	1	1	1	1
1	0	0	0	0
1	0	0	1	0
1	0	1	0	0
1	0	1	1	1
1	1	0	0	1
1	1	0	1	1
1	1	1	0	1
1	1	1	1	1

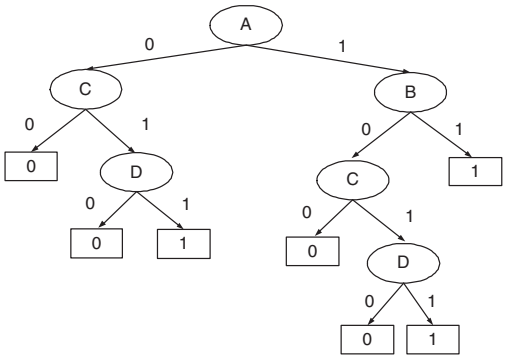


Figure 3.17. Decision tree for the Boolean function $(A \wedge B) \vee (C \wedge D)$.

whose value is 1 when there is an even number of true values among its Boolean attributes, and 0 otherwise. Accurate modeling of such a function requires a full decision tree with 2^d nodes, where d is the number of Boolean attributes (see Exercise 2 on page 205).

3. **Computational Efficiency:** Since the number of possible decision trees can be very large, many decision tree algorithms employ a heuristic-based approach to guide their search in the vast hypothesis space. For example, the algorithm presented in Section 3.3.4 uses a greedy, top-down, recursive partitioning strategy for growing a decision tree. For many data sets, such techniques quickly construct a reasonably good decision tree even when the training set size is very large. Furthermore, once a decision tree has been built, classifying a test record is extremely fast, with a worst-case complexity of $O(w)$, where w is the maximum depth of the tree.

4. **Handling Missing Values:** A decision tree classifier can handle missing attribute values in a number of ways, both in the training and the test sets. When there are missing values in the test set, the classifier must decide which branch to follow if the value of a splitting node attribute is missing for a given test instance. One approach, known as the **probabilistic split method**, which is employed by the C4.5 decision tree classifier, distributes the data instance to every child of the splitting node according to the probability that the missing attribute has a particular value. In contrast, the CART algorithm uses the **surrogate split method**, where the instance whose splitting attribute value is

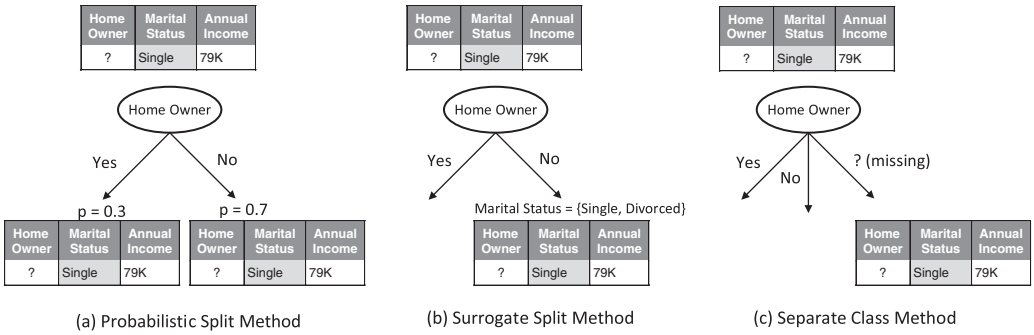


Figure 3.18. Methods for handling missing attribute values in decision tree classifier.

missing is assigned to one of the child nodes based on the value of another non-missing surrogate attribute whose splits most resemble the partitions made by the missing attribute. Another approach, known as the **separate class method** is used by the CHAID algorithm, where the missing value is treated as a separate categorical value distinct from other values of the splitting attribute. Figure 3.18 shows an example of the three different ways for handling missing values in a decision tree classifier. Other strategies for dealing with missing values are based on data preprocessing, where the instance with missing value is either imputed with the mode (for categorical attribute) or mean (for continuous attribute) value or discarded before the classifier is trained.

During training, if an attribute v has missing values in some of the training instances associated with a node, we need a way to measure the gain in purity if v is used for splitting. One simple way is to exclude instances with missing values of v in the counting of instances associated with every child node, generated for every possible outcome of v. Further, if v is chosen as the attribute test condition at a node, training instances with missing values of v can be propagated to the child nodes using any of the methods described above for handling missing values in test instances.

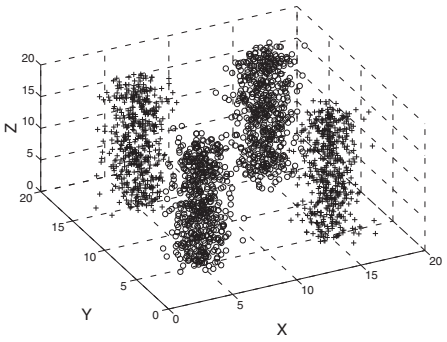
5. **Handling Interactions among Attributes:** Attributes are considered interacting if they are able to distinguish between classes when used together, but individually they provide little or no information. Due to the greedy nature of the splitting criteria in decision trees, such attributes could be passed over in favor of other attributes that are not as useful. This could result in more complex decision trees than necessary.

Hence, decision trees can perform poorly when there are interactions among attributes.

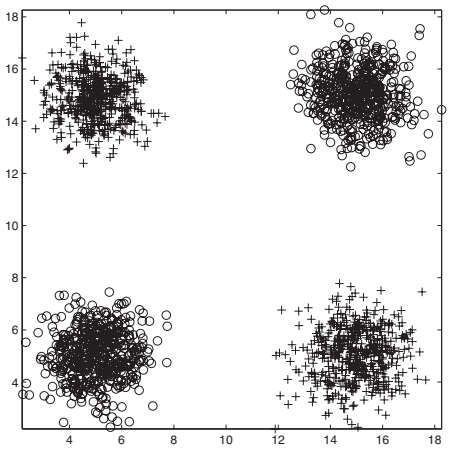
To illustrate this point, consider the three-dimensional data shown in Figure 3.19(a), which contains 2000 data points from one of two classes, denoted as + and ∘ in the diagram. Figure 3.19(b) shows the distribution of the two classes in the two-dimensional space involving attributes X and Y, which is a noisy version of the XOR Boolean function. We can see that even though the two classes are well-separated in this two-dimensional space, neither of the two attributes contain sufficient information to distinguish between the two classes when used alone. For example, the entropies of the following attribute test conditions: $X \leq 10$ and $Y \leq 10$, are close to 1, indicating that neither X nor Y provide any reduction in the impurity measure when used individually. X and Y thus represent a case of interaction among attributes. The data set also contains a third attribute, Z, in which both classes are distributed uniformly, as shown in Figures 3.19(c) and 3.19(d), and hence, the entropy of any split involving Z is close to 1. As a result, Z is as likely to be chosen for splitting as the interacting but useful attributes, X and Y. For further illustration of this issue, readers are referred to Example 3.7 in Section 3.4.1 and Exercise 8 at the end of this chapter.

6. **Handling Irrelevant Attributes:** An attribute is irrelevant if it is not useful for the classification task. Since irrelevant attributes are poorly associated with the target class labels, they will provide little or no gain in purity and thus will be passed over by other more relevant features. Hence, the presence of a small number of irrelevant attributes will not impact the decision tree construction process. However, not all attributes that provide little to no gain are irrelevant (see Figure 3.19). Hence, if the classification problem is complex (e.g., involving interactions among attributes) and there are a large number of irrelevant attributes, then some of these attributes may be accidentally chosen during the tree-growing process, since they may provide a better gain than a relevant attribute just by random chance. Feature selection techniques can help to improve the accuracy of decision trees by eliminating the irrelevant attributes during preprocessing. We will investigate the issue of too many irrelevant attributes in Section 3.4.1.

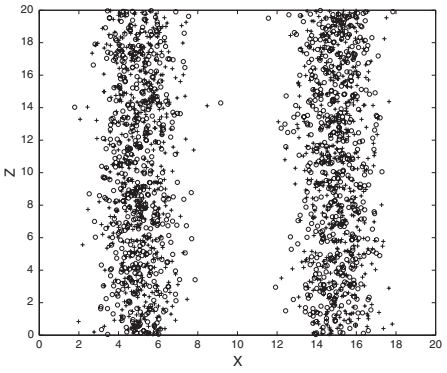
7. **Handling Redundant Attributes:** An attribute is redundant if it is strongly correlated with another attribute in the data. Since redundant



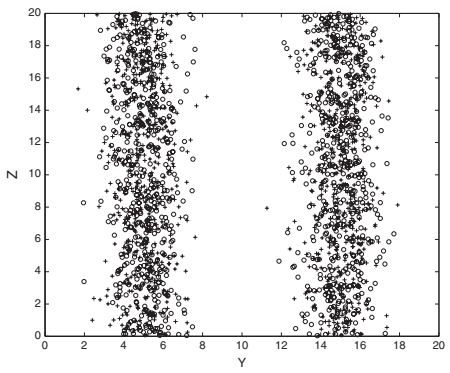
(a) Three-dimensional data with attributes X, Y, and Z.



(b) X and Y.



(c) X and Z.



(d) Y and Z.

Figure 3.19. Example of a XOR data involving X and Y, along with an irrelevant attribute Z.

attributes show similar gains in purity if they are selected for splitting, only one of them will be selected as an attribute test condition in the decision tree algorithm. Decision trees can thus handle the presence of redundant attributes.

8. **Using Rectilinear Splits:** The test conditions described so far in this chapter involve using only a single attribute at a time. As a consequence,

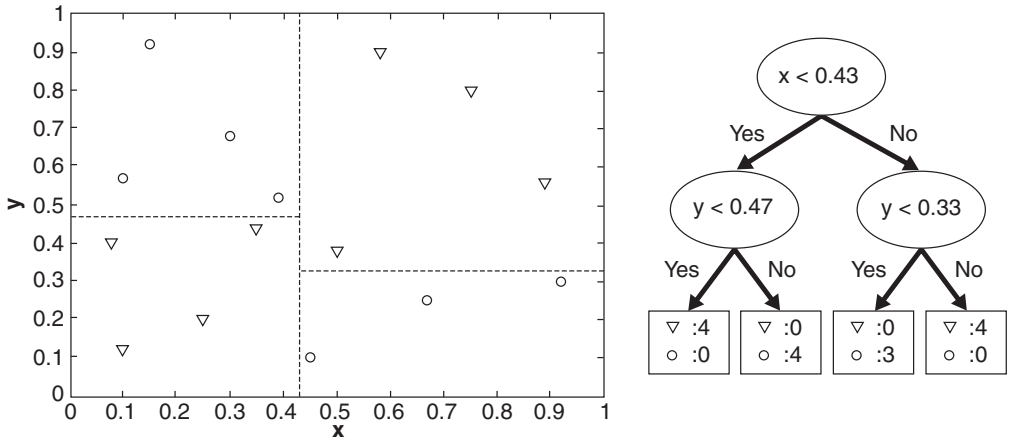


Figure 3.20. Example of a decision tree and its decision boundaries for a two-dimensional data set.

the tree-growing procedure can be viewed as the process of partitioning the attribute space into disjoint regions until each region contains records of the same class. The border between two neighboring regions of different classes is known as a **decision boundary**. Figure 3.20 shows the decision tree as well as the decision boundary for a binary classification problem. Since the test condition involves only a single attribute, the decision boundaries are rectilinear; i.e., parallel to the coordinate axes. This limits the expressiveness of decision trees in representing decision boundaries of data sets with continuous attributes. Figure 3.21 shows a two-dimensional data set involving binary classes that cannot be perfectly classified by a decision tree whose attribute test conditions are defined based on single attributes. The binary classes in the data set are generated from two skewed Gaussian distributions, centered at (8,8) and (12,12), respectively. The true decision boundary is represented by the diagonal dashed line, whereas the rectilinear decision boundary produced by the decision tree classifier is shown by the thick solid line.

In contrast, an **oblique decision tree** may overcome this limitation by allowing the test condition to be specified using more than one attribute. For example, the binary classification data shown in Figure 3.21 can be easily represented by an oblique decision tree with a single root node with test condition

$$x + y < 20.$$

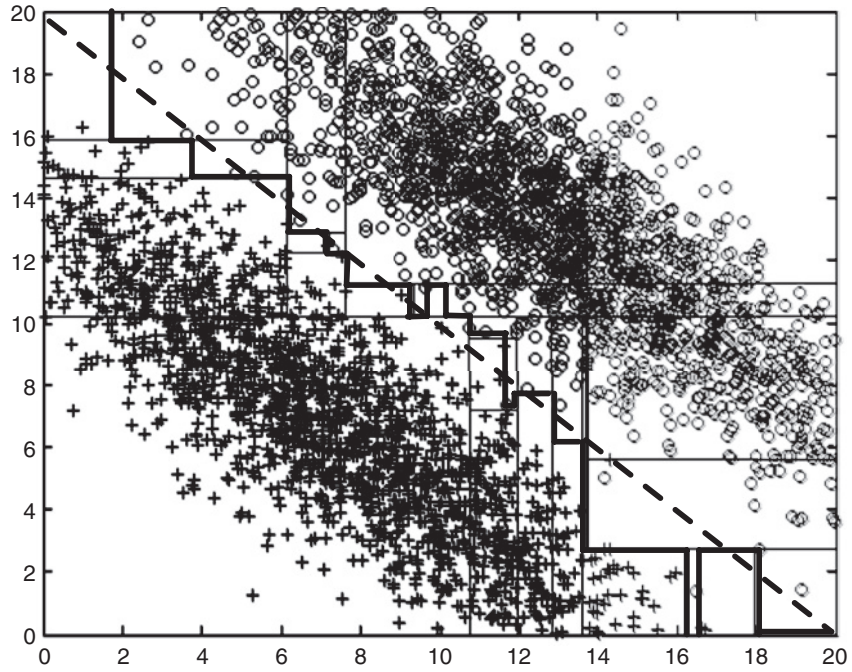


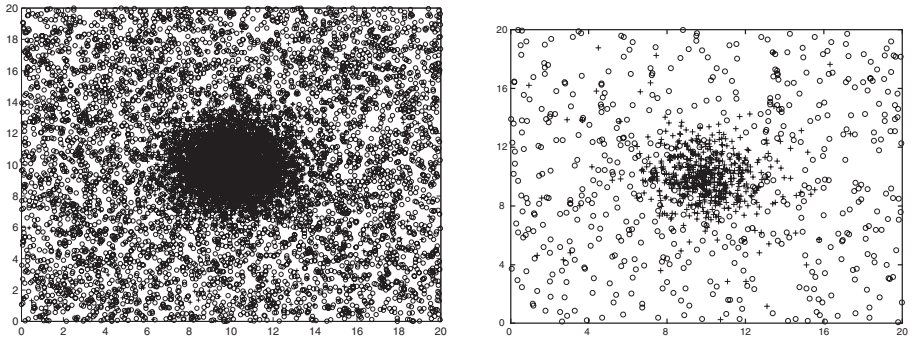
Figure 3.21. Example of data set that cannot be partitioned optimally using a decision tree with single attribute test conditions. The true decision boundary is shown by the dashed line.

Although an oblique decision tree is more expressive and can produce more compact trees, finding the optimal test condition is computationally more expensive.

9. **Choice of Impurity Measure:** It should be noted that the choice of impurity measure often has little effect on the performance of decision tree classifiers since many of the impurity measures are quite consistent with each other, as shown in Figure 3.11 on page 149. Instead, the strategy used to prune the tree has a greater impact on the final tree than the choice of impurity measure.

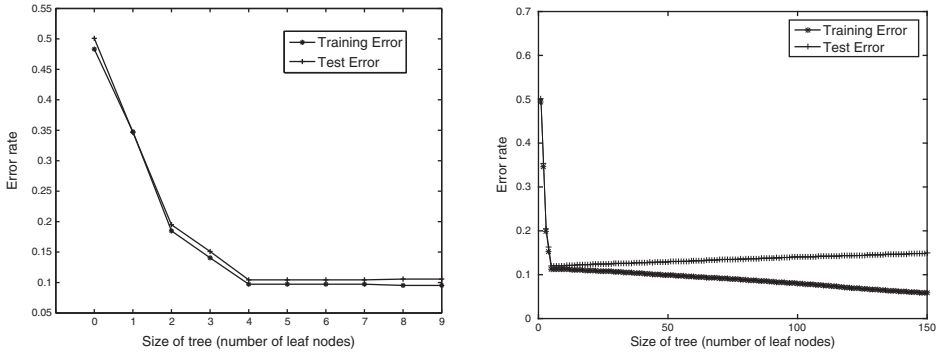
3.4 Model Overfitting

Methods presented so far try to learn classification models that show the lowest error on the training set. However, as we will show in the following example, even if a model fits well over the training data, it can still show poor generalization performance, a phenomenon known as model overfitting.



(a) Example of a 2-D data. (b) Training set using 10% data.

Figure 3.22. Examples of training and test sets of a two-dimensional classification problem.



(a) Varying tree size from 1 to 8. (b) Varying tree size from 1 to 150.

Figure 3.23. Effect of varying tree size (number of leaf nodes) on training and test errors.

Example 3.5. [Overfitting and Underfitting of Decision Trees] Consider the two-dimensional data set shown in Figure 3.22(a). The data set contains instances that belong to two separate classes, represented as + and o, respectively, where each class has 5400 instances. All instances belonging to the o class were generated from a uniform distribution. For the + class, 5000 instances were generated from a Gaussian distribution centered at (10,10) with unit variance, while the remaining 400 instances were sampled from the same uniform distribution as the o class. We can see from Figure 3.22(a) that

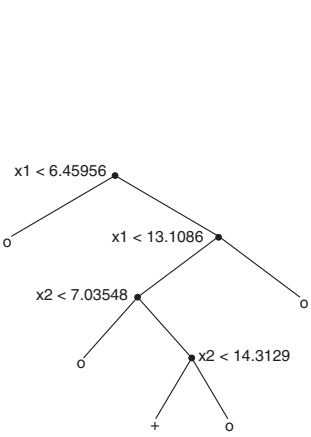
the + class can be largely distinguished from the o class by drawing a circle of appropriate size centered at (10,10). To learn a classifier using this two-dimensional data set, we randomly sampled 10% of the data for training and used the remaining 90% for testing. The training set, shown in Figure 3.22(b), looks quite representative of the overall data. We used Gini index as the impurity measure to construct decision trees of increasing sizes (number of leaf nodes), by recursively expanding a node into child nodes till every leaf node was pure, as described in Section 3.3.4.

Figure 3.23(a) shows changes in the training and test error rates as the size of the tree varies from 1 to 8. Both error rates are initially large when the tree has only one or two leaf nodes. This situation is known as **model underfitting**. Underfitting occurs when the learned decision tree is too simplistic, and thus, incapable of fully representing the true relationship between the attributes and the class labels. As we increase the tree size from 1 to 8, we can observe two effects. First, both the error rates decrease since larger trees are able to represent more complex decision boundaries. Second, the training and test error rates are quite close to each other, which indicates that the performance on the training set is fairly representative of the generalization performance. As we further increase the size of the tree from 8 to 150, the training error continues to steadily decrease till it eventually reaches zero, as shown in Figure 3.23(b). However, in a striking contrast, the test error rate ceases to decrease any further beyond a certain tree size, and then it begins to increase. The training error rate thus grossly under-estimates the test error rate once the tree becomes too large. Further, the gap between the training and test error rates keeps on widening as we increase the tree size. This behavior, which may seem counter-intuitive at first, can be attributed to the phenomena of **model overfitting**.

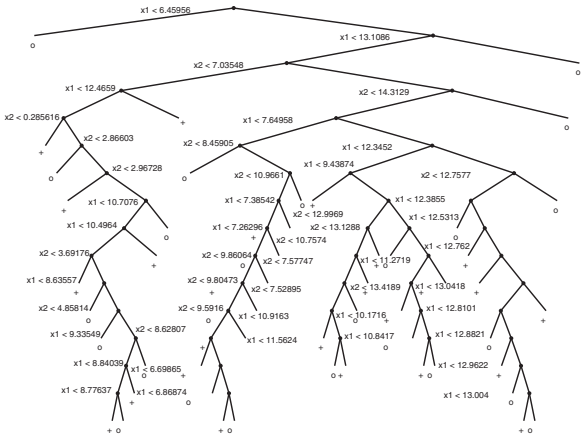
■

3.4.1 Reasons for Model Overfitting

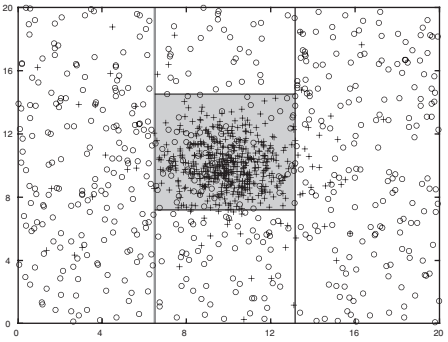
Model overfitting is the phenomena where, in the pursuit of minimizing the training error rate, an overly complex model is selected that captures specific patterns in the training data but fails to learn the *true* nature of relationships between attributes and class labels in the overall data. To illustrate this, Figure 3.24 shows decision trees and their corresponding decision boundaries (shaded rectangles represent regions assigned to the + class) for two trees of sizes 5 and 50. We can see that the decision tree of size 5 appears quite simple and its decision boundaries provide a reasonable approximation to the ideal decision boundary, which in this case corresponds to a circle centered around



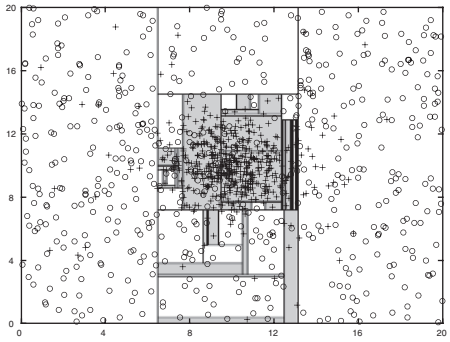
(a) Decision tree with 5 leaf nodes.



(b) Decision tree with 50 leaf nodes.



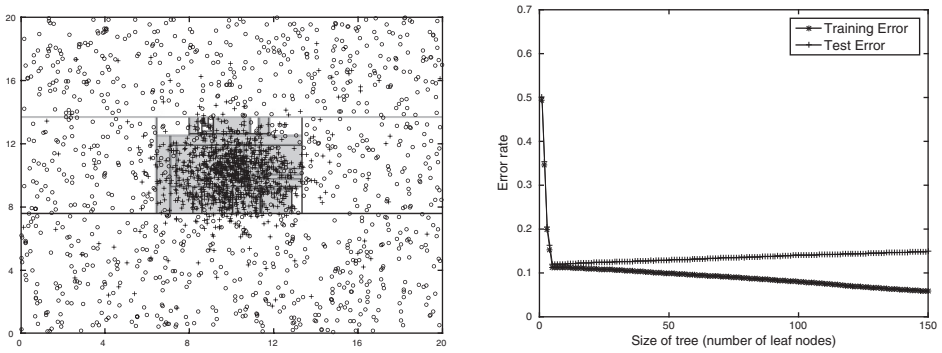
(c) Decision boundary for tree with 5 leaf nodes.



(d) Decision boundary for tree with 50 leaf nodes.

Figure 3.24. Decision trees with different model complexities.

the Gaussian distribution at (10, 10). Although its training and test error rates are non-zero, they are very close to each other, which indicates that the patterns learned in the training set should generalize well over the test set. On the other hand, the decision tree of size 50 appears much more complex than the tree of size 5, with complicated decision boundaries. For example, some of its shaded rectangles (assigned the + class) attempt to cover narrow regions in the input space that contain only one or two + training instances. Note that



(a) Decision boundary for tree with 50 leaf nodes using 20% data for training.

(b) Training and test error rates using 20% data for training.

Figure 3.25. Performance of decision trees using 20% data for training (twice the original training size).

the prevalence of + instances in such regions is highly specific to the training set, as these regions are mostly dominated by − instances in the overall data. Hence, in an attempt to perfectly fit the training data, the decision tree of size 50 starts fine tuning itself to specific patterns in the training data, leading to poor performance on an independently chosen test set.

There are a number of factors that influence model overfitting. In the following, we provide brief descriptions of two of the major factors: limited training size and high model complexity. Though they are not exhaustive, the interplay between them can help explain most of the common model overfitting phenomena in real-world applications.

Limited Training Size

Note that a training set consisting of a finite number of instances can only provide a limited representation of the overall data. Hence, it is possible that the patterns learned from a training set do not fully represent the true patterns in the overall data, leading to model overfitting. In general, as we increase the size of a training set (number of training instances), the patterns learned from the training set start resembling the true patterns in the overall data. Hence, the effect of overfitting can be reduced by increasing the training size, as illustrated in the following example.

Example 3.6. [Effect of Training Size] Suppose that we use twice the number of training instances than what we had used in the experiments conducted in Example 3.5. Specifically, we use 20% data for training and use the remainder for testing. Figure 3.25(b) shows the training and test error rates as the size of the tree is varied from 1 to 150. There are two major differences in the trends shown in this figure and those shown in Figure 3.23(b) (using only 10% of the data for training). First, even though the training error rate decreases with increasing tree size in both figures, its rate of decrease is much smaller when we use twice the training size. Second, for a given tree size, the gap between the training and test error rates is much smaller when we use twice the training size. These differences suggest that the patterns learned using 20% of data for training are more generalizable than those learned using 10% of data for training.

Figure 3.25(a) shows the decision boundaries for the tree of size 50, learned using 20% of data for training. In contrast to the tree of the same size learned using 10% data for training (see Figure 3.24(d)), we can see that the decision tree is not capturing specific patterns of noisy + instances in the training set. Instead, the high model complexity of 50 leaf nodes is being effectively used to learn the boundaries of the + instances centered at (10, 10).

■

High Model Complexity

Generally, a more complex model has a better ability to represent complex patterns in the data. For example, decision trees with larger number of leaf nodes can represent more complex decision boundaries than decision trees with fewer leaf nodes. However, an overly complex model also has a tendency to learn specific patterns in the training set that do not generalize well over unseen instances. Models with high complexity should thus be judiciously used to avoid overfitting.

One measure of model complexity is the number of "parameters" that need to be inferred from the training set. For example, in the case of decision tree induction, the attribute test conditions at internal nodes correspond to the parameters of the model that need to be inferred from the training set. A decision tree with larger number of attribute test conditions (and consequently more leaf nodes) thus involves more "parameters" and hence is more complex.

Given a class of models with a certain number of parameters, a learning algorithm attempts to select the best combination of parameter values that maximizes an evaluation metric (e.g., accuracy) over the training set. If the number of parameter value combinations (and hence the complexity) is large,

the learning algorithm has to select the best combination from a large number of possibilities, using a limited training set. In such cases, there is a high chance for the learning algorithm to pick a *spurious* combination of parameters that maximizes the evaluation metric just by random chance. This is similar to the **multiple comparisons problem** (also referred as multiple testing problem) in statistics.

As an illustration of the multiple comparisons problem, consider the task of predicting whether the stock market will rise or fall in the next ten trading days. If a stock analyst simply makes random guesses, the probability that her prediction is correct on any trading day is 0.5. However, the probability that she will predict correctly at least nine out of ten times is

$$\frac{\binom{10}{9} + \binom{10}{10}}{2^{10}} = 0.0107,$$

which is extremely low.

Suppose we are interested in choosing an investment advisor from a pool of 200 stock analysts. Our strategy is to select the analyst who makes the most number of correct predictions in the next ten trading days. The flaw in this strategy is that even if all the analysts make their predictions in a random fashion, the probability that at least one of them makes at least nine correct predictions is

$$1 - (1 - 0.0107)^{200} = 0.8847,$$

which is very high. Although each analyst has a low probability of predicting at least nine times correctly, considered together, we have a high probability of finding at least one analyst who can do so. However, there is no guarantee in the future that such an analyst will continue to make accurate predictions by random guessing.

How does the multiple comparisons problem relate to model overfitting? In the context of learning a classification model, each combination of parameter values corresponds to an analyst, while the number of training instances corresponds to the number of days. Analogous to the task of selecting the best analyst who makes the most accurate predictions on consecutive days, the task of a learning algorithm is to select the best combination of parameters that results in the highest accuracy on the training set. If the number of parameter combinations is large but the training size is small, it is highly likely for the learning algorithm to choose a spurious parameter combination that provides high training accuracy just by random chance. In the following example, we illustrate the phenomena of overfitting due to multiple comparisons in the context of decision tree induction.

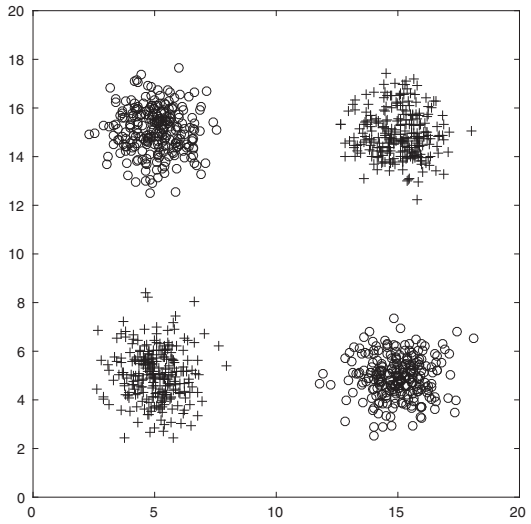
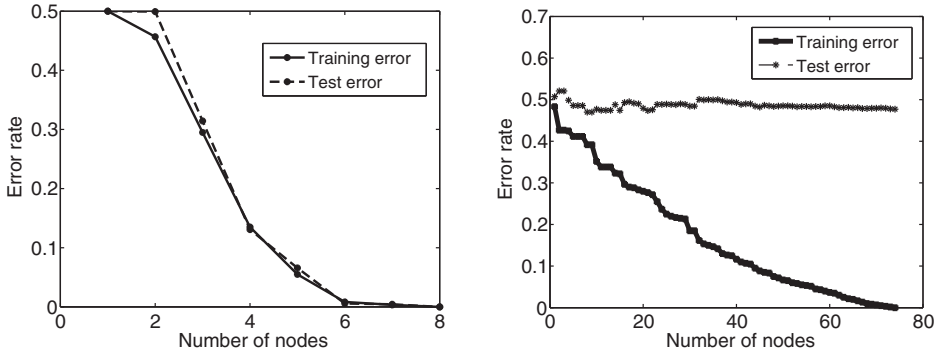


Figure 3.26. Example of a two-dimensional (X-Y) data set.

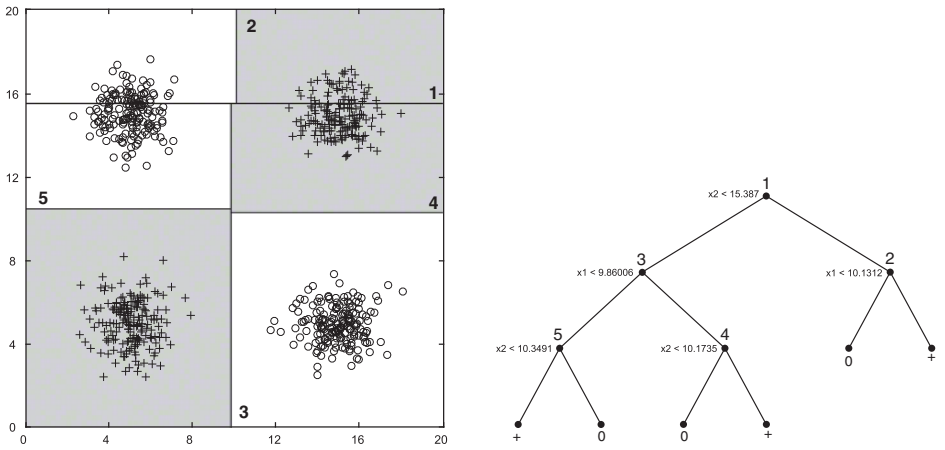


(a) Using X and Y attributes only.

(b) After adding 100 irrelevant attributes.

Figure 3.27. Training and test error rates illustrating the effect of multiple comparisons problem on model overfitting.

Example 3.7. [Multiple Comparisons and Overfitting] Consider the two-dimensional data set shown in Figure 3.26 containing 500 + and 500 o instances, which is similar to the data shown in Figure 3.19. In this data set, the distributions of both classes are well-separated in the two-dimensional (X-Y) attribute space, but none of the two attributes (X or Y) are sufficiently informative to be used alone for separating the two classes. Hence, splitting



(a) Decision boundary for tree with 6 leaf nodes.

(b) Decision tree with 6 leaf nodes.

Figure 3.28. Decision tree with 6 leaf nodes using X and Y as attributes. Splits have been numbered from 1 to 5 in order of other occurrence in the tree.

the data set based on any value of an X or Y attribute will provide close to zero reduction in an impurity measure. However, if X and Y attributes are used together in the splitting criterion (e.g., splitting X at 10 *and* Y at 10), the two classes can be effectively separated.

Figure 3.27(a) shows the training and test error rates for learning decision trees of varying sizes, when 30% of the data is used for training and the remainder of the data for testing. We can see that the two classes can be separated using a small number of leaf nodes. Figure 3.28 shows the decision boundaries for the tree with six leaf nodes, where the splits have been numbered according to their order of appearance in the tree. Note that the even though splits 1 and 3 provide trivial gains, their consequent splits (2, 4, and 5) provide large gains, resulting in effective discrimination of the two classes.

Assume we add 100 irrelevant attributes to the two-dimensional X-Y data. Learning a decision tree from this resultant data will be challenging because the number of candidate attributes to choose for splitting at every internal node will increase from two to 102. With such a large number of candidate attribute test conditions to choose from, it is quite likely that spurious attribute test conditions will be selected at internal nodes because of the multiple comparisons problem. Figure 3.27(b) shows the training and test error rates after adding 100 irrelevant attributes to the training set. We can see that the

test error rate remains close to 0.5 even after using 50 leaf nodes, while the training error rate keeps on declining and eventually becomes 0.

■

3.5 Model Selection

There are many possible classification models with varying levels of model complexity that can be used to capture patterns in the training data. Among these possibilities, we want to select the model that shows lowest generalization error rate. The process of selecting a model with the right level of complexity, which is expected to generalize well over unseen test instances, is known as **model selection**. As described in the previous section, the training error rate cannot be reliably used as the sole criterion for model selection. In the following, we present three generic approaches to estimate the generalization performance of a model that can be used for model selection. We conclude this section by presenting specific strategies for using these approaches in the context of decision tree induction.

3.5.1 Using a Validation Set

Note that we can always estimate the generalization error rate of a model by using "out-of-sample" estimates, i.e. by evaluating the model on a separate **validation set** that is not used for training the model. The error rate on the validation set, termed as the validation error rate, is a better indicator of generalization performance than the training error rate, since the validation set has not been used for training the model. The validation error rate can be used for model selection as follows.

Given a training set $D.train$, we can partition $D.train$ into two smaller subsets, $D.tr$ and $D.val$, such that $D.tr$ is used for training while $D.val$ is used as the validation set. For example, two-thirds of $D.train$ can be reserved as $D.tr$ for training, while the remaining one-third is used as $D.val$ for computing validation error rate. For any choice of classification model m that is trained on $D.tr$, we can estimate its validation error rate on $D.val$, $err_{val}(m)$. The model that shows the lowest value of $err_{val}(m)$ can then be selected as the preferred choice of model.

The use of validation set provides a generic approach for model selection. However, one limitation of this approach is that it is sensitive to the sizes of $D.tr$ and $D.val$, obtained by partitioning $D.train$. If the size of $D.tr$ is too small, it may result in the learning of a poor classification model with sub-standard performance, since a smaller training set will be less representative

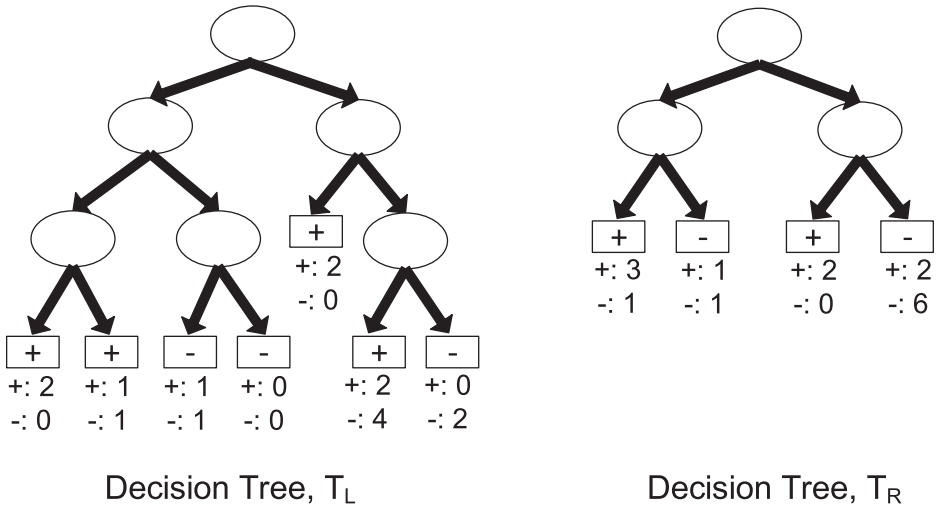


Figure 3.29. Class distribution of validation data for the two decision trees shown in Figure 3.30.

of the overall data. On the other hand, if the size of $D.val$ is too small, the validation error rate might not be reliable for selecting models, as it would be computed over a small number of instances.

Example 3.8. [Validation Error] In the following example, we illustrate one possible approach for using a validation set in decision tree induction. Figure 3.29 shows the predicted labels at the leaf nodes of the decision trees generated in Figure 3.30. The counts given beneath the leaf nodes represent the proportion of data instances in the validation set that reach each of the nodes. Based on the predicted labels of the nodes, the validation error rate for the left tree is $err_{val}(T_L) = 6/16 = 0.375$, while the validation error rate for the right tree is $err_{val}(T_R) = 4/16 = 0.25$. Based on their validation error rates, the right tree is preferred over the left one. ■

3.5.2 Incorporating Model Complexity

Since the chance for model overfitting increases as the model becomes more complex, a model selection approach should not only consider the training error rate but also the model complexity. This strategy is inspired by a well-known principle known as **Occam's razor** or the **principle of parsimony**, which suggests that given two models with the same errors, the simpler model is preferred over the more complex model. A generic approach to account

for model complexity while estimating generalization performance is formally described as follows.

Given a training set $D.train$, let us consider learning a classification model m that belongs to a certain class of models, $\mathcal{M}$. For example, if $\mathcal{M}$ represents the set of all possible decision trees, then m can correspond to a specific decision tree learned from the training set. We are interested in estimating the generalization error rate of m, $gen.error(m)$. As discussed previously, the training error rate of m, $train.error(m, D.train)$, can under-estimate $gen.error(m)$ when the model complexity is high. Hence, we represent $gen.error(m)$ as a function of not just the training error rate but also the model complexity of $\mathcal{M}$, $complexity(\mathcal{M})$, as follows:

$$gen.error(m) = train.error(m, D.train) + \alpha \times complexity(\mathcal{M}), \quad (3.11)$$

where α is a hyper-parameter that strikes a balance between minimizing training error and reducing model complexity. A higher value of α gives more emphasis to the model complexity in the estimation of generalization performance. To choose the right value of α, we can make use of the validation set in a similar way as described in 3.5.1. For example, we can iterate through a range of values of α and for every possible value, we can learn a model on a subset of the training set, $D.tr$, and compute its validation error rate on a separate subset, $D.val$. We can then select the value of α that provides the lowest validation error rate.

Equation 3.11 provides one possible approach for incorporating model complexity into the estimate of generalization performance. This approach is at the heart of a number of techniques for estimating generalization performance, such as the structural risk minimization principle, the Akaike's Information Criterion (AIC), and the Bayesian Information Criterion (BIC). The structural risk minimization principle serves as the building block for learning support vector machines, which will be discussed later in Chapter 6. For more details on AIC and BIC, see the Bibliographic Notes.

In the following, we present two different approaches for estimating the complexity of a model, $complexity(\mathcal{M})$. While the former is specific to decision trees, the latter is more generic and can be used with any class of models.

Estimating the Complexity of Decision Trees

In the context of decision trees, the complexity of a decision tree can be estimated as the ratio of the number of leaf nodes to the number of training instances. Let k be the number of leaf nodes and N_{train} be the number of

training instances. The complexity of a decision tree can then be described as k/N_{train}. This reflects the intuition that for a larger training size, we can learn a decision tree with larger number of leaf nodes without it becoming overly complex. The generalization error rate of a decision tree T can then be computed using Equation 3.11 as follows:

$$err_{gen}(T) = err(T) + \Omega \times \frac{k}{N_{train}},$$

where $err(T)$ is the training error of the decision tree and Ω is a hyper-parameter that makes a trade-off between reducing training error and minimizing model complexity, similar to the use of α in Equation 3.11. Ω can be viewed as the relative cost of adding a leaf node relative to incurring a training error. In the literature on decision tree induction, the above approach for estimating generalization error rate is also termed as the **pessimistic error estimate**. It is called pessimistic as it assumes the generalization error rate to be worse than the training error rate (by adding a penalty term for model complexity). On the other hand, simply using the training error rate as an estimate of the generalization error rate is called the **optimistic error estimate** or the **resubstitution estimate**.

Example 3.9. [Generalization Error Estimates] Consider the two binary decision trees, T_L and T_R, shown in Figure 3.30. Both trees are generated from the same training data and T_L is generated by expanding three leaf nodes of T_R. The counts shown in the leaf nodes of the trees represent the class

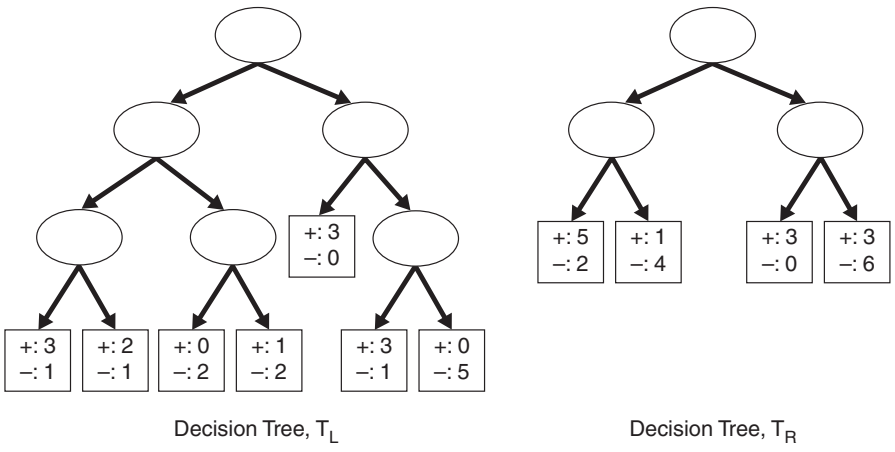


Figure 3.30. Example of two decision trees generated from the same training data.

distribution of the training instances. If each leaf node is labeled according to the majority class of training instances that reach the node, the training error rate for the left tree will be $err(T_L) = 4/24 = 0.167$, while the training error rate for the right tree will be $err(T_R) = 6/24 = 0.25$. Based on their training error rates alone, T_L would preferred over T_R, even though T_L is more complex (contains larger number of leaf nodes) than T_R.

Now, assume that the cost associated with each leaf node is $\Omega = 0.5$. Then, the generalization error estimate for T_L will be

$$err_{gen}(T_L) = \frac{4}{24} + 0.5 \times \frac{7}{24} = \frac{7.5}{24} = 0.3125$$

and the generalization error estimate for T_R will be

$$err_{gen}(T_R) = \frac{6}{24} + 0.5 \times \frac{4}{24} = \frac{8}{24} = 0.3333.$$

Since T_L has a lower generalization error rate, it will still be preferred over T_R. Note that $\Omega = 0.5$ implies that a node should always be expanded into its two child nodes if it improves the prediction of at least one training instance, since expanding a node is less costly than misclassifying a training instance. On the other hand, if $\Omega = 1$, then the generalization error rate for T_L is $err_{gen}(T_L) = 11/24 = 0.458$ and for T_R is $err_{gen}(T_R) = 10/24 = 0.417$. In this case, T_R will be preferred over T_L because it has a lower generalization error rate. This example illustrates that different choices of Ω can change our preference of decision trees based on their generalization error estimates. However, for a given choice of Ω, the pessimistic error estimate provides an approach for modeling the generalization performance on unseen test instances. The value of Ω can be selected with the help of a validation set. ■

Minimum Description Length Principle

Another way to incorporate model complexity is based on an information-theoretic approach known as the minimum description length or MDL principle. To illustrate this approach, consider the example shown in Figure 3.31. In this example, both person `A` and person `B` are given a set of instances with known attribute values $\mathbf{x}$. Assume person `A` knows the class label y for every instance, while person `B` has no such information. `A` would like to share the class information with `B` by sending a message containing the labels. The message would contain $\Theta(N)$ bits of information, where N is the number of instances.

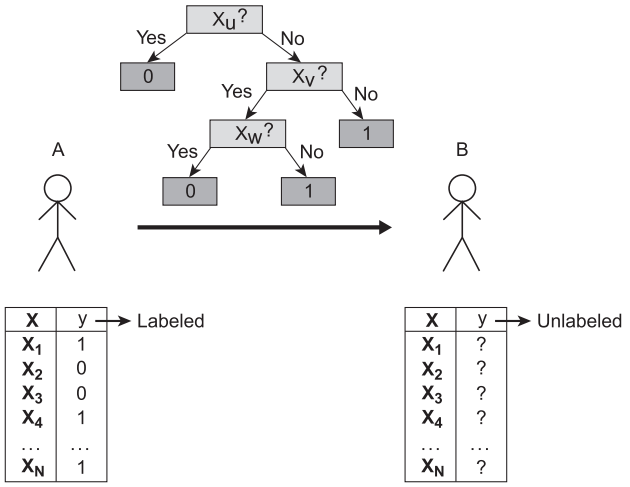


Figure 3.31. An illustration of the minimum description length principle.

Alternatively, instead of sending the class labels explicitly, `A` can build a classification model from the instances and transmit it to `B`. `B` can then apply the model to determine the class labels of the instances. If the model is 100% accurate, then the cost for transmission is equal to the number of bits required to encode the model. Otherwise, `A` must also transmit information about which instances are misclassified by the model so that `B` can reproduce the same class labels. Thus, the overall transmission cost, which is equal to the total description length of the message, is

$$Cost(model, data) = Cost(data|model) + \alpha \times Cost(model), \tag{3.12}$$

where the first term on the right-hand side is the number of bits needed to encode the misclassified instances, while the second term is the number of bits required to encode the model. There is also a hyper-parameter α that trades-off the relative costs of the misclassified instances and the model. Notice the familiarity between this equation and the generic equation for generalization error rate presented in Equation 3.11. A good model must have a total description length less than the number of bits required to encode the entire sequence of class labels. Furthermore, given two competing models, the model with lower total description length is preferred. An example showing how to compute the total description length of a decision tree is given in Exercise 11 on page 209.

3.5.3 Estimating Statistical Bounds

Instead of using Equation 3.11 to estimate the generalization error rate of a model, an alternative way is to apply a statistical correction to the training error rate of the model that is indicative of its model complexity. This can be done if the probability distribution of training error is available or can be assumed. For example, the number of errors committed by a leaf node in a decision tree can be assumed to follow a binomial distribution. We can thus compute an upper bound limit to the observed training error rate that can be used for model selection, as illustrated in the following example.

Example 3.10. [Statistical Bounds on Training Error] Consider the left-most branch of the binary decision trees shown in Figure 3.30. Observe that the left-most leaf node of T_R has been expanded into two child nodes in T_L. Before splitting, the training error rate of the node is $2/7 = 0.286$. By approximating a binomial distribution with a normal distribution, the following upper bound of the training error rate e can be derived:

$$e_{upper}(N, e, \alpha) = \frac{e + \frac{z^2_{\alpha/2}}{2N} + z_{\alpha/2}\sqrt{\frac{e(1-e)}{N} + \frac{z^2_{\alpha/2}}{4N^2}}}{1 + \frac{z^2_{\alpha/2}}{N}}, \tag{3.13}$$

where α is the confidence level, $z_{\alpha/2}$ is the standardized value from a standard normal distribution, and N is the total number of training instances used to compute e. By replacing $\alpha = 25\%$, $N = 7$, and $e = 2/7$, the upper bound for the error rate is $e_{upper}(7, 2/7, 0.25) = 0.503$, which corresponds to $7 \times 0.503 = 3.521$ errors. If we expand the node into its child nodes as shown in T_L, the training error rates for the child nodes are $1/4 = 0.250$ and $1/3 = 0.333$, respectively. Using Equation (3.13), the upper bounds of these error rates are $e_{upper}(4, 1/4, 0.25) = 0.537$ and $e_{upper}(3, 1/3, 0.25) = 0.650$, respectively. The overall training error of the child nodes is $4 \times 0.537 + 3 \times 0.650 = 4.098$, which is larger than the estimated error for the corresponding node in T_R, suggesting that it should not be split. ■

3.5.4 Model Selection for Decision Trees

Building on the generic approaches presented above, we present two commonly used model selection strategies for decision tree induction.

Prepruning (Early Stopping Rule) In this approach, the tree-growing algorithm is halted before generating a fully grown tree that perfectly fits

the entire training data. To do this, a more restrictive stopping condition must be used; e.g., stop expanding a leaf node when the observed gain in the generalization error estimate falls below a certain threshold. This estimate of the generalization error rate can be computed using any of the approaches presented in the preceding three subsections, e.g., by using pessimistic error estimates, by using validation error estimates, or by using statistical bounds. The advantage of prepruning is that it avoids the computations associated with generating overly complex subtrees that overfit the training data. However, one major drawback of this method is that, even if no significant gain is obtained using one of the existing splitting criterion, subsequent splitting may result in better subtrees. Such subtrees would not be reached if prepruning is used because of the greedy nature of decision tree induction.

Post-pruning In this approach, the decision tree is initially grown to its maximum size. This is followed by a tree-pruning step, which proceeds to trim the fully grown tree in a bottom-up fashion. Trimming can be done by replacing a subtree with (1) a new leaf node whose class label is determined from the majority class of instances affiliated with the subtree (approach known as **subtree replacement**), or (2) the most frequently used branch of the subtree (approach known as **subtree raising**). The tree-pruning step terminates when no further improvement in the generalization error estimate is observed beyond a certain threshold. Again, the estimates of generalization error rate can be computed using any of the approaches presented in the previous three subsections. Post-pruning tends to give better results than prepruning because it makes pruning decisions based on a fully grown tree, unlike prepruning, which can suffer from premature termination of the tree-growing process. However, for post-pruning, the additional computations needed to grow the full tree may be wasted when the subtree is pruned.

Figure 3.32 illustrates the simplified decision tree model for the web robot detection example given in Section 3.3.5. Notice that the subtree rooted at `depth` $= 1$ has been replaced by one of its branches corresponding to `breadth` $<= 7$, `width` > 3, and `MultiP` $= 1$, using subtree raising. On the other hand, the subtree corresponding to `depth` > 1 and `MultiAgent` $= 0$ has been replaced by a leaf node assigned to class 0, using subtree replacement. The subtree for `depth` > 1 and `MultiAgent` $= 1$ remains intact.

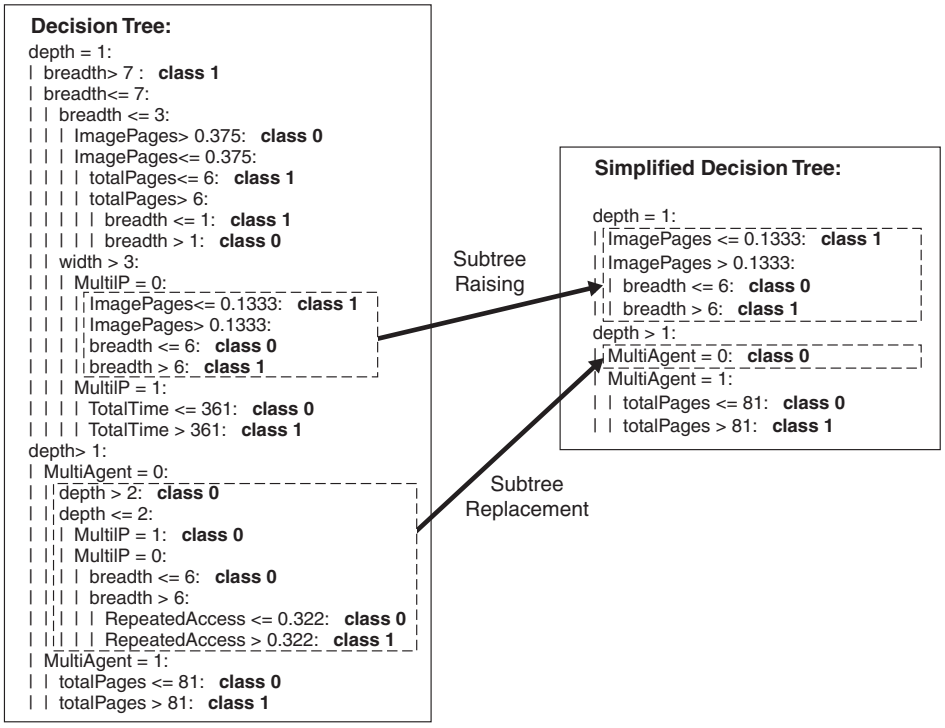


Figure 3.32. Post-pruning of the decision tree for web robot detection.

3.6 Model Evaluation

The previous section discussed several approaches for model selection that can be used to learn a classification model from a training set $D.train$. Here we discuss methods for estimating its generalization performance, i.e. its performance on unseen instances outside of $D.train$. This process is known as **model evaluation**.

Note that model selection approaches discussed in Section 3.5 also compute an estimate of the generalization performance using the training set $D.train$. However, these estimates are *biased* indicators of the performance on unseen instances, since they were used to guide the selection of classification model. For example, if we use the validation error rate for model selection (as described in Section 3.5.1), the resulting model would be deliberately chosen to minimize the errors on the validation set. The validation error rate may thus under-estimate the true generalization error rate, and hence cannot be reliably used for model evaluation.

A correct approach for model evaluation would be to assess the performance of a learned model on a labeled test set has not been used at any stage of model selection. This can be achieved by partitioning the entire set of labeled instances D, into two disjoint subsets, $D.train$, which is used for model selection and $D.test$, which is used for computing the test error rate, err_{test}. In the following, we present two different approaches for partitioning D into $D.train$ and $D.test$, and computing the test error rate, err_{test}.

3.6.1 Holdout Method

The most basic technique for partitioning a labeled data set is the holdout method, where the labeled set D is randomly partitioned into two disjoint sets, called the training set $D.train$ and the test set $D.test$. A classification model is then induced from $D.train$ using the model selection approaches presented in Section 3.5, and its error rate on $D.test$, err_{test}, is used as an estimate of the generalization error rate. The proportion of data reserved for training and for testing is typically at the discretion of the analysts, e.g., two-thirds for training and one-third for testing.

Similar to the trade-off faced while partitioning $D.train$ into $D.tr$ and $D.val$ in Section 3.5.1, choosing the right fraction of labeled data to be used for training and testing is non-trivial. If the size of $D.train$ is small, the learned classification model may be improperly learned using an insufficient number of training instances, resulting in a biased estimation of generalization performance. On the other hand, if the size of $D.test$ is small, err_{test} may be less reliable as it would be computed over a small number of test instances. Moreover, err_{test} can have a high variance as we change the random partitioning of D into $D.train$ and $D.test$.

The holdout method can be repeated several times to obtain a distribution of the test error rates, an approach known as **random subsampling** or repeated holdout method. This method produces a distribution of the error rates that can be used to understand the variance of err_{test}.

3.6.2 Cross-Validation

Cross-validation is a widely-used model evaluation method that aims to make effective use of all labeled instances in D for both training and testing. To illustrate this method, suppose that we are given a labeled set that we have randomly partitioned into three equal-sized subsets, S_1, S_2, and S_3, as shown in Figure 3.33. For the first run, we train a model using subsets S_2 and S_3 (shown as empty blocks) and test the model on subset S_1. The test error rate

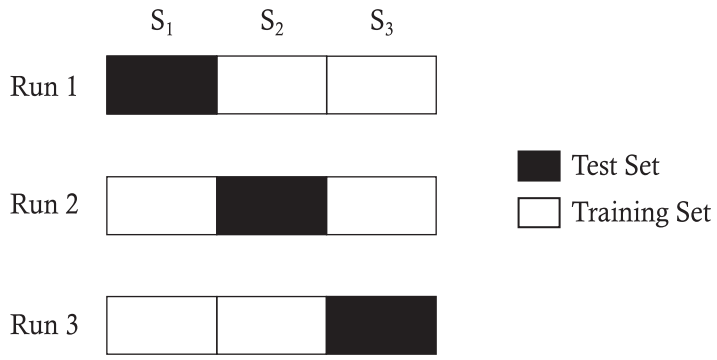


Figure 3.33. Example demonstrating the technique of 3-fold cross-validation.

on S_1, denoted as $err(S_1)$, is thus computed in the first run. Similarly, for the second run, we use S_1 and S_3 as the training set and S_2 as the test set, to compute the test error rate, $err(S_2)$, on S_2. Finally, we use S_1 and S_3 for training in the third run, while S_3 is used for testing, thus resulting in the test error rate $err(S_3)$ for S_3. The overall test error rate is obtained by summing up the number of errors committed in each test subset across all runs and dividing it by the total number of instances. This approach is called three-fold cross-validation.

The k-fold cross-validation method generalizes this approach by segmenting the labeled data D (of size N) into k equal-sized partitions (or folds). During the ith run, one of the partitions of D is chosen as $D.test(i)$ for testing, while the rest of the partitions are used as $D.train(i)$ for training. A model $m(i)$ is learned using $D.train(i)$ and applied on $D.test(i)$ to obtain the sum of test errors, $err_{sum}(i)$. This procedure is repeated k times. The total test error rate, err_{test}, is then computed as

$$err_{test} = \frac{\sum_{i=1}^{k} err_{sum}(i)}{N}. \tag{3.14}$$

Every instance in the data is thus used for testing exactly once and for training exactly $(k-1)$ times. Also, every run uses $(k-1)/k$ fraction of the data for training and $1/k$ fraction for testing.

The right choice of k in k-fold cross-validation depends on a number of characteristics of the problem. A small value of k will result in a smaller training set at every run, which will result in a larger estimate of generalization error rate than what is expected of a model trained over the entire labeled set. On the other hand, a high value of k results in a larger training set at

every run, which reduces the bias in the estimate of generalization error rate. In the extreme case, when $k = N$, every run uses exactly one data instance for testing and the remainder of the data for testing. This special case of k-fold cross-validation is called the **leave-one-out** approach. This approach has the advantage of utilizing as much data as possible for training. However, leave-one-out can produce quite misleading results in some special scenarios, as illustrated in Exercise 12. Furthermore, leave-one-out can be computationally expensive for large data sets as the cross-validation procedure needs to be repeated N times. For most practical applications, the choice of k between 5 and 10 provides a reasonable approach for estimating the generalization error rate, because each fold is able to make use of 80% to 90% of the labeled data for training.

The k-fold cross-validation method, as described above, produces a single estimate of the generalization error rate, without providing any information about the variance of the estimate. To obtain this information, we can run k-fold cross-validation for every possible partitioning of the data into k partitions, and obtain a distribution of test error rates computed for every such partitioning. The average test error rate across all possible partitionings serves as a more robust estimate of generalization error rate. This approach of estimating the generalization error rate and its variance is known as the **complete cross-validation** approach. Even though such an estimate is quite robust, it is usually too expensive to consider all possible partitionings of a large data set into k partitions. A more practical solution is to repeat the cross-validation approach multiple times, using a different random partitioning of the data into k partitions at every time, and use the average test error rate as the estimate of generalization error rate. Note that since there is only one possible partitioning for the leave-one-out approach, it is not possible to estimate the variance of generalization error rate, which is another limitation of this method.

The k-fold cross-validation does not guarantee that the fraction of positive and negative instances in every partition of the data is equal to the fraction observed in the overall data. A simple solution to this problem is to perform a stratified sampling of the positive and negative instances into k partitions, an approach called **stratified cross-validation**.

In k-fold cross-validation, a different model is learned at every run and the performance of every one of the k models on their respective test folds is then aggregated to compute the overall test error rate, err_{test}. Hence, err_{test} does not reflect the generalization error rate of any of the k models. Instead, it reflects the *expected* generalization error rate of the *model selection approach*, when applied on a training set of the same size as one of the training

folds ($N(k-1)/k$). This is different than the err_{test} computed in the holdout method, which exactly corresponds to the specific model learned over $D.train$. Hence, although effectively utilizing every data instance in D for training and testing, the err_{test} computed in the cross-validation method does not represent the performance of a single model learned over a specific $D.train$.

Nonetheless, in practice, err_{test} is typically used as an estimate of the generalization error of a model built on D. One motivation for this is that when the size of the training folds is closer to the size of the overall data (when k is large), then err_{test} resembles the expected performance of a model learned over a data set of the same size as D. For example, when k is 10, every training fold is 90% of the overall data. The err_{test} then should approach the expected performance of a model learned over 90% of the overall data, which will be close to the expected performance of a model learned over D.

3.7 Presence of Hyper-parameters

Hyper-parameters are parameters of learning algorithms that need to be determined before learning the classification model. For instance, consider the hyper-parameter α that appeared in Equation 3.11, which is repeated here for convenience. This equation was used for estimating the generalization error for a model selection approach that used an explicit representations of model complexity. (See Section 3.5.2.)

$$gen.error(m) = train.error(m, D.train) + \alpha \times complexity(\mathcal{M})$$

For other examples of hyper-parameters, see Chapter 6.

Unlike regular model parameters, such as the test conditions in the internal nodes of a decision tree, hyper-parameters such as α do not appear in the final classification model that is used to classify unlabeled instances. However, the values of hyper-parameters need to be determined during model selection—a process known as **hyper-parameter selection**—and must be taken into account during model evaluation. Fortunately, both tasks can be effectively accomplished via slight modifications of the cross-validation approach described in the previous section.

3.7.1 Hyper-parameter Selection

In Section 3.5.2, a validation set was used to select α and this approach is generally applicable for hyper-parameter section. Let p be the hyper-parameter that needs to be selected from a finite range of values, $\mathcal{P} = \{p_1, p_2, \ldots p_n\}$.

5

Cluster Analysis: Basic Concepts and Algorithms

Cluster analysis divides data into groups (clusters) that are meaningful, useful, or both. If meaningful groups are the goal, then the clusters should capture the natural structure of the data. In some cases, however, cluster analysis is used for data summarization in order to reduce the size of the data. Whether for understanding or utility, cluster analysis has long played an important role in a wide variety of fields: psychology and other social sciences, biology, statistics, pattern recognition, information retrieval, machine learning, and data mining.

There have been many applications of cluster analysis to practical problems. We provide some specific examples, organized by whether the purpose of the clustering is understanding or utility.

Clustering for Understanding Classes, or conceptually meaningful groups of objects that share common characteristics, play an important role in how people analyze and describe the world. Indeed, human beings are skilled at dividing objects into groups (clustering) and assigning particular objects to these groups (classification). For example, even relatively young children can quickly label the objects in a photograph. In the context of understanding data, clusters are potential classes and cluster analysis is the study of techniques for automatically finding classes. The following are some examples:

- **Biology.** Biologists have spent many years creating a taxonomy (hierarchical classification) of all living things: kingdom, phylum, class, order, family, genus, and species. Thus, it is perhaps not surprising that

much of the early work in cluster analysis sought to create a discipline of mathematical taxonomy that could automatically find such classification structures. More recently, biologists have applied clustering to analyze the large amounts of genetic information that are now available. For example, clustering has been used to find groups of genes that have similar functions.

- **Information Retrieval.** The World Wide Web consists of billions of web pages, and the results of a query to a search engine can return thousands of pages. Clustering can be used to group these search results into a small number of clusters, each of which captures a particular aspect of the query. For instance, a query of "movie" might return web pages grouped into categories such as reviews, trailers, stars, and theaters. Each category (cluster) can be broken into subcategories (subclusters), producing a hierarchical structure that further assists a user's exploration of the query results.

- **Climate.** Understanding the Earth's climate requires finding patterns in the atmosphere and ocean. To that end, cluster analysis has been applied to find patterns in atmospheric pressure and ocean temperature that have a significant impact on climate.

- **Psychology and Medicine.** An illness or condition frequently has a number of variations, and cluster analysis can be used to identify these different subcategories. For example, clustering has been used to identify different types of depression. Cluster analysis can also be used to detect patterns in the spatial or temporal distribution of a disease.

- **Business.** Businesses collect large amounts of information about current and potential customers. Clustering can be used to segment customers into a small number of groups for additional analysis and marketing activities.

Clustering for Utility Cluster analysis provides an abstraction from individual data objects to the clusters in which those data objects reside. Additionally, some clustering techniques characterize each cluster in terms of a cluster prototype; i.e., a data object that is representative of the objects in the cluster. These cluster prototypes can be used as the basis for a number of additional data analysis or data processing techniques. Therefore, in the context of utility, cluster analysis is the study of techniques for finding the most representative cluster prototypes.

- **Summarization.** Many data analysis techniques, such as regression or principal component analysis, have a time or space complexity of $O(m^2)$ or higher (where m is the number of objects), and thus, are not practical for large data sets. However, instead of applying the algorithm to the entire data set, it can be applied to a reduced data set consisting only of cluster prototypes. Depending on the type of analysis, the number of prototypes, and the accuracy with which the prototypes represent the data, the results can be comparable to those that would have been obtained if all the data could have been used.

- **Compression.** Cluster prototypes can also be used for data compression. In particular, a table is created that consists of the prototypes for each cluster; i.e., each prototype is assigned an integer value that is its position (index) in the table. Each object is represented by the index of the prototype associated with its cluster. This type of compression is known as **vector quantization** and is often applied to image, sound, and video data, where (1) many of the data objects are highly similar to one another, (2) some loss of information is acceptable, and (3) a substantial reduction in the data size is desired.

- **Efficiently Finding Nearest Neighbors.** Finding nearest neighbors can require computing the pairwise distance between all points. Often clusters and their cluster prototypes can be found much more efficiently. If objects are relatively close to the prototype of their cluster, then we can use the prototypes to reduce the number of distance computations that are necessary to find the nearest neighbors of an object. Intuitively, if two cluster prototypes are far apart, then the objects in the corresponding clusters cannot be nearest neighbors of each other. Consequently, to find an object's nearest neighbors, it is necessary to compute only the distance to objects in nearby clusters, where the nearness of two clusters is measured by the distance between their prototypes. This idea is made more precise in Exercise 30 of Chapter 2, which is on pages 131–132.

This chapter provides an introduction to cluster analysis. We begin with a high-level overview of clustering, including a discussion of the various approaches to dividing objects into sets of clusters and the different types of clusters. We then describe three specific clustering techniques that represent broad categories of algorithms and illustrate a variety of concepts: K-means, agglomerative hierarchical clustering, and DBSCAN. The final section of this chapter is devoted to cluster validity—methods for evaluating the goodness

of the clusters produced by a clustering algorithm. More advanced clustering concepts and algorithms will be discussed in Chapter 8. Whenever possible, we discuss the strengths and weaknesses of different schemes. In addition, the Bibliographic Notes provide references to relevant books and papers that explore cluster analysis in greater depth.

5.1 Overview

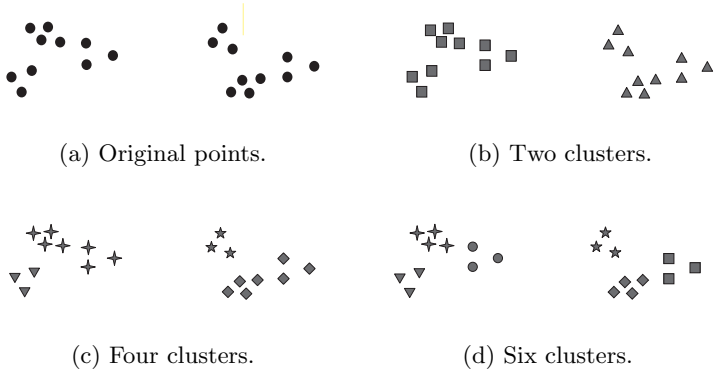
Before discussing specific clustering techniques, we provide some necessary background. First, we further define cluster analysis, illustrating why it is difficult and explaining its relationship to other techniques that group data. Then we explore two important topics: (1) different ways to group a set of objects into a set of clusters, and (2) types of clusters.

5.1.1 What Is Cluster Analysis?

Cluster analysis groups data objects based on information found only in the data that describes the objects and their relationships. The goal is that the objects within a group be similar (or related) to one another and different from (or unrelated to) the objects in other groups. The greater the similarity (or homogeneity) within a group and the greater the difference between groups, the better or more distinct the clustering.

In many applications, the notion of a cluster is not well defined. To better understand the difficulty of deciding what constitutes a cluster, consider Figure 5.1, which shows 20 points and three different ways of dividing them into clusters. The shapes of the markers indicate cluster membership. Figures 5.1(b) and 5.1(d) divide the data into two and six parts, respectively. However, the apparent division of each of the two larger clusters into three subclusters may simply be an artifact of the human visual system. Also, it may not be unreasonable to say that the points form four clusters, as shown in Figure 5.1(c). This figure illustrates that the definition of a cluster is imprecise and that the best definition depends on the nature of data and the desired results.

Cluster analysis is related to other techniques that are used to divide data objects into groups. For instance, clustering can be regarded as a form of classification in that it creates a labeling of objects with class (cluster) labels. However, it derives these labels only from the data. In contrast, classification in the sense of Chapter 3 is **supervised classification**; i.e., new, unlabeled objects are assigned a class label using a model developed from objects with known class labels. For this reason, cluster analysis is sometimes referred to as **unsupervised classification**. When the term classification is used



(a) Original points. (b) Two clusters.

(c) Four clusters. (d) Six clusters.

Figure 5.1. Three different ways of clustering the same set of points.

without any qualification within data mining, it typically refers to supervised classification.

Also, while the terms **segmentation** and **partitioning** are sometimes used as synonyms for clustering, these terms are frequently used for approaches outside the traditional bounds of cluster analysis. For example, the term partitioning is often used in connection with techniques that divide graphs into subgraphs and that are not strongly connected to clustering. Segmentation often refers to the division of data into groups using simple techniques; e.g., an image can be split into segments based only on pixel intensity and color, or people can be divided into groups based on their income. Nonetheless, some work in graph partitioning and in image and segmentation is related to cluster analysis.

5.1.2 Different Types of Clusterings

An entire collection of clusters is commonly referred to as a **clustering**, and in this section, we distinguish various types of clusterings: hierarchical (nested) versus partitional (unnested), exclusive versus overlapping versus fuzzy, and complete versus partial.

Hierarchical versus Partitional The most commonly discussed distinction among different types of clusterings is whether the set of clusters is nested or unnested, or in more traditional terminology, hierarchical or partitional. A **partitional clustering** is simply a division of the set of data objects into non-overlapping subsets (clusters) such that each data object is in exactly one

subset. Taken individually, each collection of clusters in Figures 5.1 (b–d) is a partitional clustering.

If we permit clusters to have subclusters, then we obtain a **hierarchical clustering**, which is a set of nested clusters that are organized as a tree. Each node (cluster) in the tree (except for the leaf nodes) is the union of its children (subclusters), and the root of the tree is the cluster containing all the objects. Often, but not always, the leaves of the tree are singleton clusters of individual data objects. If we allow clusters to be nested, then one interpretation of Figure 5.1(a) is that it has two subclusters (Figure 5.1(b)), each of which, in turn, has three subclusters (Figure 5.1(d)). The clusters shown in Figures 5.1 (a–d), when taken in that order, also form a hierarchical (nested) clustering with, respectively, 1, 2, 4, and 6 clusters on each level. Finally, note that a hierarchical clustering can be viewed as a sequence of partitional clusterings and a partitional clustering can be obtained by taking any member of that sequence; i.e., by cutting the hierarchical tree at a particular level.

Exclusive versus Overlapping versus Fuzzy The clusterings shown in Figure 5.1 are all **exclusive**, as they assign each object to a single cluster. There are many situations in which a point could reasonably be placed in more than one cluster, and these situations are better addressed by non-exclusive clustering. In the most general sense, an **overlapping** or **non-exclusive clustering** is used to reflect the fact that an object can *simultaneously* belong to more than one group (class). For instance, a person at a university can be both an enrolled student and an employee of the university. A non-exclusive clustering is also often used when, for example, an object is "between" two or more clusters and could reasonably be assigned to any of these clusters. Imagine a point halfway between two of the clusters of Figure 5.1. Rather than make a somewhat arbitrary assignment of the object to a single cluster, it is placed in all of the "equally good" clusters.

In a **fuzzy clustering** (Section 8.2.1), every object belongs to every cluster with a membership weight that is between 0 (absolutely doesn't belong) and 1 (absolutely belongs). In other words, clusters are treated as fuzzy sets. (Mathematically, a fuzzy set is one in which an object belongs to every set with a weight that is between 0 and 1. In fuzzy clustering, we often impose the additional constraint that the sum of the weights for each object must equal 1.) Similarly, probabilistic clustering techniques (Section 8.2.2) compute the probability with which each point belongs to each cluster, and these probabilities must also sum to 1. Because the membership weights or probabilities for any object sum to 1, a fuzzy or probabilistic clustering does

not address true multiclass situations, such as the case of a student employee, where an object belongs to multiple classes. Instead, these approaches are most appropriate for avoiding the arbitrariness of assigning an object to only one cluster when it is close to several. In practice, a fuzzy or probabilistic clustering is often converted to an exclusive clustering by assigning each object to the cluster in which its membership weight or probability is highest.

Complete versus Partial A **complete clustering** assigns every object to a cluster, whereas a **partial clustering** does not. The motivation for a partial clustering is that some objects in a data set may not belong to well-defined groups. Many times objects in the data set represent noise, outliers, or "uninteresting background." For example, some newspaper stories share a common theme, such as global warming, while other stories are more generic or one-of-a-kind. Thus, to find the important topics in last month's stories, we often want to search only for clusters of documents that are tightly related by a common theme. In other cases, a complete clustering of the objects is desired. For example, an application that uses clustering to organize documents for browsing needs to guarantee that all documents can be browsed.

5.1.3 Different Types of Clusters

Clustering aims to find useful groups of objects (clusters), where usefulness is defined by the goals of the data analysis. Not surprisingly, several different notions of a cluster prove useful in practice. In order to visually illustrate the differences among these types of clusters, we use two-dimensional points, as shown in Figure 5.2, as our data objects. We stress, however, that the types of clusters described here are equally valid for other kinds of data.

Well-Separated A cluster is a set of objects in which each object is closer (or more similar) to every other object in the cluster than to any object not in the cluster. Sometimes a threshold is used to specify that all the objects in a cluster must be sufficiently close (or similar) to one another. This idealistic definition of a cluster is satisfied only when the data contains natural clusters that are quite far from each other. Figure 5.2(a) gives an example of well-separated clusters that consists of two groups of points in a two-dimensional space. The distance between any two points in different groups is larger than the distance between any two points within a group. Well-separated clusters do not need to be globular, but can have any shape.

Prototype-Based A cluster is a set of objects in which each object is closer (more similar) to the prototype that defines the cluster than to the prototype of any other cluster. For data with continuous attributes, the prototype of a cluster is often a centroid, i.e., the average (mean) of all the points in the cluster. When a centroid is not meaningful, such as when the data has categorical attributes, the prototype is often a medoid, i.e., the most representative point of a cluster. For many types of data, the prototype can be regarded as the most central point, and in such instances, we commonly refer to prototype-based clusters as **center-based clusters**. Not surprisingly, such clusters tend to be globular. Figure 5.2(b) shows an example of center-based clusters.

Graph-Based If the data is represented as a graph, where the nodes are objects and the links represent connections among objects (see Section 2.1.2), then a cluster can be defined as a **connected component**; i.e., a group of objects that are connected to one another, but that have no connection to objects outside the group. An important example of graph-based clusters is a **contiguity-based cluster**, where two objects are connected only if they are within a specified distance of each other. This implies that each object in a contiguity-based cluster is closer to some other object in the cluster than to any point in a different cluster. Figure 5.2(c) shows an example of such clusters for two-dimensional points. This definition of a cluster is useful when clusters are irregular or intertwined. However, this approach can have trouble when noise is present since, as illustrated by the two spherical clusters of Figure 5.2(c), a small bridge of points can merge two distinct clusters.

Other types of graph-based clusters are also possible. One such approach (Section 5.3.2) defines a cluster as a **clique**; i.e., a set of nodes in a graph that are completely connected to each other. Specifically, if we add connections between objects in the order of their distance from one another, a cluster is formed when a set of objects forms a clique. Like prototype-based clusters, such clusters tend to be globular.

Density-Based A cluster is a dense region of objects that is surrounded by a region of low density. Figure 5.2(d) shows some density-based clusters for data created by adding noise to the data of Figure 5.2(c). The two circular clusters are not merged, as in Figure 5.2(c), because the bridge between them fades into the noise. Likewise, the curve that is present in Figure 5.2(c) also fades into the noise and does not form a cluster in Figure 5.2(d). A density-based definition of a cluster is often employed when the clusters are irregular or

intertwined, and when noise and outliers are present. By contrast, a contiguity-based definition of a cluster would not work well for the data of Figure 5.2(d) because the noise would tend to form bridges between clusters.

Shared-Property (Conceptual Clusters) More generally, we can define a cluster as a set of objects that share some property. This definition encompasses all the previous definitions of a cluster; e.g., objects in a center-based cluster share the property that they are all closest to the same centroid or medoid. However, the shared-property approach also includes new types of clusters. Consider the clusters shown in Figure 5.2(e). A triangular area (cluster) is adjacent to a rectangular one, and there are two intertwined circles (clusters). In both cases, a clustering algorithm would need a very specific concept of a cluster to successfully detect these clusters. The process of finding such clusters is called conceptual clustering. However, too sophisticated a notion of a cluster would take us into the area of pattern recognition, and thus, we only consider simpler types of clusters in this book.

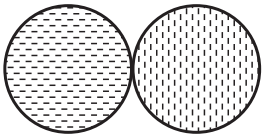
Road Map

In this chapter, we use the following three simple, but important techniques to introduce many of the concepts involved in cluster analysis.

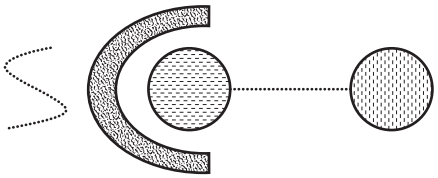
- **K-means**. This is a prototype-based, partitional clustering technique that attempts to find a user-specified number of clusters (K), which are represented by their centroids.
- **Agglomerative Hierarchical Clustering.** This clustering approach refers to a collection of closely related clustering techniques that produce a hierarchical clustering by starting with each point as a singleton cluster and then repeatedly merging the two closest clusters until a single, all-encompassing cluster remains. Some of these techniques have a natural interpretation in terms of graph-based clustering, while others have an interpretation in terms of a prototype-based approach.
- **DBSCAN**. This is a density-based clustering algorithm that produces a partitional clustering, in which the number of clusters is automatically determined by the algorithm. Points in low-density regions are classified as noise and omitted; thus, DBSCAN does not produce a complete clustering.



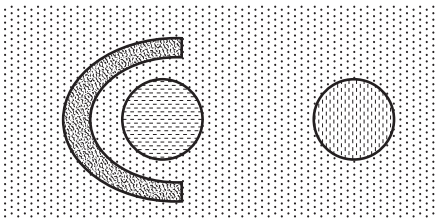
(a) Well-separated clusters. Each point is closer to all of the points in its cluster than to any point in another cluster.



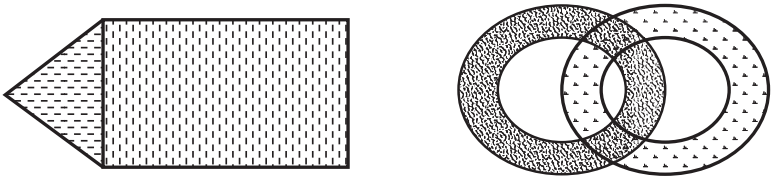
(b) Center-based clusters. Each point is closer to the center of its cluster than to the center of any other cluster.



(c) Contiguity-based clusters. Each point is closer to at least one point in its cluster than to any point in another cluster.



(d) Density-based clusters. Clusters are regions of high density separated by regions of low density.



(e) Conceptual clusters. Points in a cluster share some general property that derives from the entire set of points. (Points in the intersection of the circles belong to both.)

Figure 5.2. Different types of clusters as illustrated by sets of two-dimensional points.

5.2 K-means

Prototype-based clustering techniques create a one-level partitioning of the data objects. There are a number of such techniques, but two of the most prominent are K-means and K-medoid. K-means defines a prototype in terms of a centroid, which is usually the mean of a group of points, and is typically

applied to objects in a continuous n-dimensional space. K-medoid defines a prototype in terms of a medoid, which is the most representative point for a group of points, and can be applied to a wide range of data since it requires only a proximity measure for a pair of objects. While a centroid almost never corresponds to an actual data point, a medoid, by its definition, must be an actual data point. In this section, we will focus solely on K-means, which is one of the oldest and most widely-used clustering algorithms.

5.2.1 The Basic K-means Algorithm

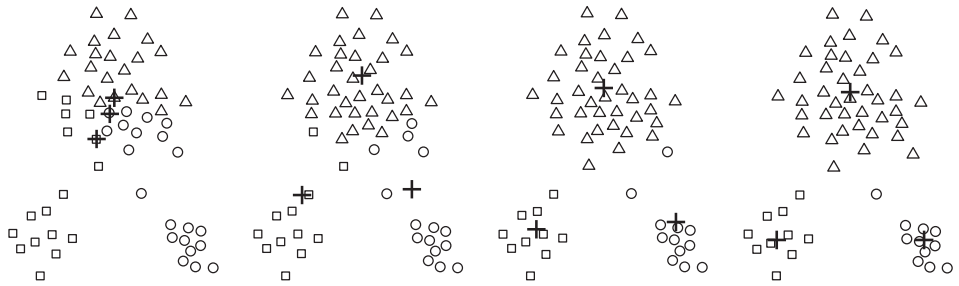
The K-means clustering technique is simple, and we begin with a description of the basic algorithm. We first choose K initial centroids, where K is a user-specified parameter, namely, the number of clusters desired. Each point is then assigned to the closest centroid, and each collection of points assigned to a centroid is a cluster. The centroid of each cluster is then updated based on the points assigned to the cluster. We repeat the assignment and update steps until no point changes clusters, or equivalently, until the centroids remain the same.

K-means is formally described by Algorithm 5.1. The operation of K-means is illustrated in Figure 5.3, which shows how, starting from three centroids, the final clusters are found in four assignment-update steps. In these and other figures displaying K-means clustering, each subfigure shows (1) the centroids at the start of the iteration and (2) the assignment of the points to those centroids. The centroids are indicated by the "+" symbol; all points belonging to the same cluster have the same marker shape.

Algorithm 5.1 Basic K-means algorithm.

1: Select K points as initial centroids.
2: **repeat**
3: Form K clusters by assigning each point to its closest centroid.
4: Recompute the centroid of each cluster.
5: **until** Centroids do not change.

In the first step, shown in Figure 5.3(a), points are assigned to the initial centroids, which are all in the larger group of points. For this example, we use the mean as the centroid. After points are assigned to a centroid, the centroid is then updated. Again, the figure for each step shows the centroid at the beginning of the step and the assignment of points to those centroids. In the second step, points are assigned to the updated centroids, and the centroids



(a) Iteration 1. (b) Iteration 2. (c) Iteration 3. (d) Iteration 4.

Figure 5.3. Using the K-means algorithm to find three clusters in sample data.

are updated again. In steps 2, 3, and 4, which are shown in Figures 5.3 (b), (c), and (d), respectively, two of the centroids move to the two small groups of points at the bottom of the figures. When the K-means algorithm terminates in Figure 5.3(d), because no more changes occur, the centroids have identified the natural groupings of points.

For a number of combinations of proximity functions and types of centroids, K-means always converges to a solution; i.e., K-means reaches a state in which no points are shifting from one cluster to another, and hence, the centroids don't change. Because most of the convergence occurs in the early steps, however, the condition on line 5 of Algorithm 5.1 is often replaced by a weaker condition, e.g., repeat until only 1% of the points change clusters.

We consider each of the steps in the basic K-means algorithm in more detail and then provide an analysis of the algorithm's space and time complexity.

Assigning Points to the Closest Centroid

To assign a point to the closest centroid, we need a proximity measure that quantifies the notion of "closest" for the specific data under consideration. Euclidean (L_2) distance is often used for data points in Euclidean space, while cosine similarity is more appropriate for documents. However, several types of proximity measures can be appropriate for a given type of data. For example, Manhattan (L_1) distance can be used for Euclidean data, while the Jaccard measure is often employed for documents.

Usually, the similarity measures used for K-means are relatively simple since the algorithm repeatedly calculates the similarity of each point to each centroid. In some cases, however, such as when the data is in low-dimensional

Table 5.1. Table of notation.

Symbol	Description
$\mathbf{x}$	An object.
C_i	The ith cluster.
$\mathbf{c}_i$	The centroid of cluster C_i.
$\mathbf{c}$	The centroid of all points.
m_i	The number of objects in the ith cluster.
m	The number of objects in the data set.
K	The number of clusters.

Euclidean space, it is possible to avoid computing many of the similarities, thus significantly speeding up the K-means algorithm. Bisecting K-means (described in Section 5.2.3) is another approach that speeds up K-means by reducing the number of similarities computed.

Centroids and Objective Functions

Step 4 of the K-means algorithm was stated rather generally as "recompute the centroid of each cluster," since the centroid can vary, depending on the proximity measure for the data and the goal of the clustering. The goal of the clustering is typically expressed by an objective function that depends on the proximities of the points to one another or to the cluster centroids; e.g., minimize the squared distance of each point to its closest centroid. We illustrate this with two examples. However, the key point is this: after we have specified a proximity measure and an objective function, the centroid that we should choose can often be determined mathematically. We provide mathematical details in Section 5.2.6, and provide a non-mathematical discussion of this observation here.

Data in Euclidean Space Consider data whose proximity measure is Euclidean distance. For our objective function, which measures the quality of a clustering, we use the **sum of the squared error (SSE)**, which is also known as scatter. In other words, we calculate the error of each data point, i.e., its Euclidean distance to the closest centroid, and then compute the total sum of the squared errors. Given two different sets of clusters that are produced by two different runs of K-means, we prefer the one with the smallest squared error since this means that the prototypes (centroids) of this clustering are a better representation of the points in their cluster. Using the notation in Table 5.1, the SSE is formally defined as follows:

$$\text{SSE} = \sum_{i=1}^{K} \sum_{\mathbf{x} \in C_i} dist(\mathbf{c}_i, \mathbf{x})^2 \tag{5.1}$$

where *dist* is the standard Euclidean (L_2) distance between two objects in Euclidean space.

Given these assumptions, it can be shown (see Section 5.2.6) that the centroid that minimizes the SSE of the cluster is the mean. Using the notation in Table 5.1, the centroid (mean) of the ith cluster is defined by Equation 5.2.

$$\mathbf{c}_i = \frac{1}{m_i} \sum_{\mathbf{x} \in C_i} \mathbf{x} \tag{5.2}$$

To illustrate, the centroid of a cluster containing the three two-dimensional points, (1,1), (2,3), and (6,2), is $((1+2+6)/3, ((1+3+2)/3) = (3, 2)$.

Steps 3 and 4 of the K-means algorithm directly attempt to minimize the SSE (or more generally, the objective function). Step 3 forms clusters by assigning points to their nearest centroid, which minimizes the SSE for the given set of centroids. Step 4 recomputes the centroids so as to further minimize the SSE. However, the actions of K-means in Steps 3 and 4 are guaranteed to only find a local minimum with respect to the SSE because they are based on optimizing the SSE for specific choices of the centroids and clusters, rather than for all possible choices. We will later see an example in which this leads to a suboptimal clustering.

Document Data To illustrate that K-means is not restricted to data in Euclidean space, we consider document data and the cosine similarity measure. Here we assume that the document data is represented as a document-term matrix as described on page 57. Our objective is to maximize the similarity of the documents in a cluster to the cluster centroid; this quantity is known as the **cohesion** of the cluster. For this objective it can be shown that the cluster centroid is, as for Euclidean data, the mean. The analogous quantity to the total SSE is the total cohesion, which is given by Equation 5.3.

$$\text{Total Cohesion} = \sum_{i=1}^{K} \sum_{\mathbf{x} \in C_i} cosine(\mathbf{x}, \mathbf{c}_i) \tag{5.3}$$

The General Case There are a number of choices for the proximity function, centroid, and objective function that can be used in the basic K-means

Table 5.2. K-means: Common choices for proximity, centroids, and objective functions.

Proximity Function	Centroid	Objective Function
Manhattan (L_1)	median	Minimize sum of the L_1 distance of an object to its cluster centroid
Squared Euclidean (L_2^2)	mean	Minimize sum of the squared L_2 distance of an object to its cluster centroid
cosine	mean	Maximize sum of the cosine similarity of an object to its cluster centroid
Bregman divergence	mean	Minimize sum of the Bregman divergence of an object to its cluster centroid

algorithm and that are guaranteed to converge. Table 5.2 shows some possible choices, including the two that we have just discussed. Notice that for Manhattan (L_1) distance and the objective of minimizing the sum of the distances, the appropriate centroid is the median of the points in a cluster.

The last entry in the table, Bregman divergence (Section 2.4.8), is actually a class of proximity measures that includes the squared Euclidean distance, L_2^2, the Mahalanobis distance, and cosine similarity. The importance of Bregman divergence functions is that any such function can be used as the basis of a K-means style clustering algorithm with the mean as the centroid. Specifically, if we use a Bregman divergence as our proximity function, then the resulting clustering algorithm has the usual properties of K-means with respect to convergence, local minima, etc. Furthermore, the properties of such a clustering algorithm can be developed for all possible Bregman divergences. For example, K-means algorithms that use cosine similarity or squared Euclidean distance are particular instances of a general clustering algorithm based on Bregman divergences.

For the rest of our K-means discussion, we use two-dimensional data since it is easy to explain K-means and its properties for this type of data. But, as suggested by the last few paragraphs, K-means is a general clustering algorithm and can be used with a wide variety of data types, such as documents and time series.

Choosing Initial Centroids

When random initialization of centroids is used, different runs of K-means typically produce different total SSEs. We illustrate this with the set of two-dimensional points shown in Figure 5.3, which has three natural clusters of points. Figure 5.4(a) shows a clustering solution that is the global minimum of



(a) Optimal clustering. (b) Suboptimal clustering.

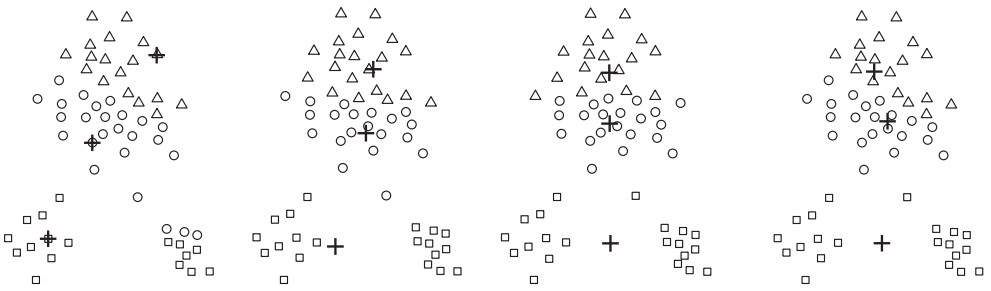
Figure 5.4. Three optimal and non-optimal clusters.

the SSE for three clusters, while Figure 5.4(b) shows a suboptimal clustering that is only a local minimum.

Choosing the proper initial centroids is the key step of the basic K-means procedure. A common approach is to choose the initial centroids randomly, but the resulting clusters are often poor.

Example 5.1 (Poor Initial Centroids)**.** Randomly selected initial centroids can be poor. We provide an example of this using the same data set used in Figures 5.3 and 5.4. Figures 5.3 and 5.5 show the clusters that result from two particular choices of initial centroids. (For both figures, the positions of the cluster centroids in the various iterations are indicated by crosses.) In Figure 5.3, even though all the initial centroids are from one natural cluster, the minimum SSE clustering is still found. In Figure 5.5, however, even though the initial centroids seem to be better distributed, we obtain a suboptimal clustering, with higher squared error. ■

Example 5.2 (Limits of Random Initialization)**.** One technique that is commonly used to address the problem of choosing initial centroids is to perform multiple runs, each with a different set of randomly chosen initial centroids, and then select the set of clusters with the minimum SSE. While simple, this strategy might not work very well, depending on the data set and the number of clusters sought. We demonstrate this using the sample data set shown in Figure 5.6(a). The data consists of two pairs of clusters, where the clusters in each (top-bottom) pair are closer to each other than to the clusters in the other pair. Figure 5.6 (b–d) shows that if we start with two initial centroids per pair of clusters, then even when both centroids are in a single cluster, the centroids



(a) Iteration 1. (b) Iteration 2. (c) Iteration 3. (d) Iteration 4.

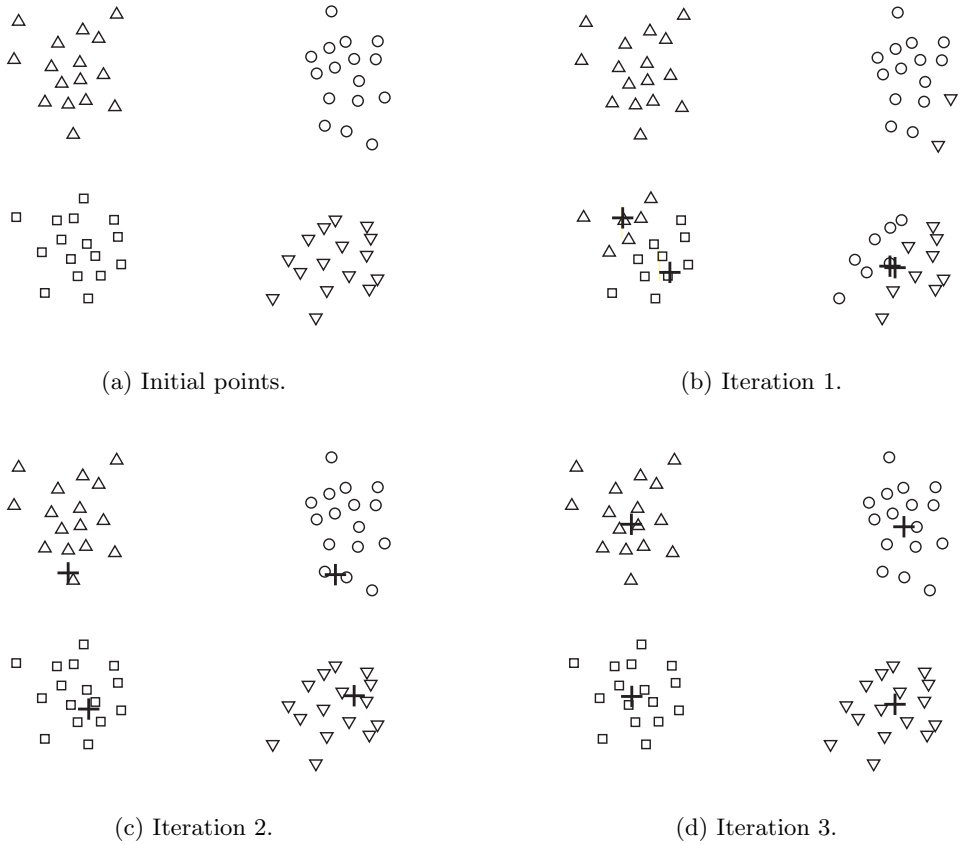
Figure 5.5. Poor starting centroids for K-means.

will redistribute themselves so that the "true" clusters are found. However, Figure 5.7 shows that if a pair of clusters has only one initial centroid and the other pair has three, then two of the true clusters will be combined and one true cluster will be split.

Note that an optimal clustering will be obtained as long as two initial centroids fall anywhere in a pair of clusters, since the centroids will redistribute themselves, one to each cluster. Unfortunately, as the number of clusters becomes larger, it is increasingly likely that at least one pair of clusters will have only one initial centroid—see Exercise 4 on page 385. In this case, because the pairs of clusters are farther apart than clusters within a pair, the K-means algorithm will not redistribute the centroids between pairs of clusters, and thus, only a local minimum will be achieved. ■

Because of the problems with using randomly selected initial centroids, which even repeated runs might not overcome, other techniques are often employed for initialization. One effective approach is to take a sample of points and cluster them using a hierarchical clustering technique. K clusters are extracted from the hierarchical clustering, and the centroids of those clusters are used as the initial centroids. This approach often works well, but is practical only if (1) the sample is relatively small, e.g., a few hundred to a few thousand (hierarchical clustering is expensive), and (2) K is relatively small compared to the sample size.

The following procedure is another approach to selecting initial centroids. Select the first point at random or take the centroid of all points. Then, for each successive initial centroid, select the point that is farthest from any of the initial centroids already selected. In this way, we obtain a set of initial centroids

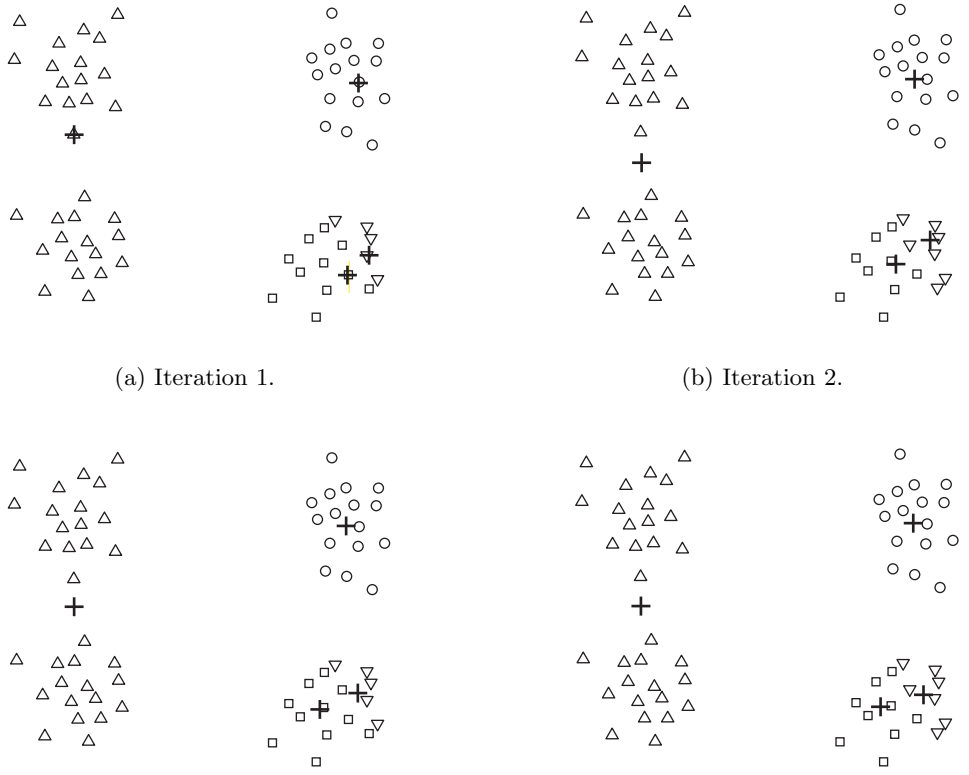


(a) Initial points. (b) Iteration 1.

(c) Iteration 2. (d) Iteration 3.

Figure 5.6. Two pairs of clusters with a pair of initial centroids within each pair of clusters.

that is guaranteed to be not only randomly selected but also well separated. Unfortunately, such an approach can select outliers, rather than points in dense regions (clusters), which can lead to a situation where many clusters have just one point—an outlier—which reduces the number of centroids for forming clusters for the majority of points. Also, it is expensive to compute the farthest point from the current set of initial centroids. To overcome these problems, this approach is often applied to a sample of the points. Because outliers are rare, they tend not to show up in a random sample. In contrast, points from every dense region are likely to be included unless the sample size is very small. Also, the computation involved in finding the initial centroids is greatly reduced because the sample size is typically much smaller than the number of points.



(a) Iteration 1.

(b) Iteration 2.

(c) Iteration 3.

(d) Iteration 4.

Figure 5.7. Two pairs of clusters with more or fewer than two initial centroids within a pair of clusters.

K-means++ More recently, a new approach for initializing K-means, called K-means++, has been developed. This procedure is guaranteed to find a K-means clustering solution that is optimal to within a factor of $O \log(k)$, which in practice translates into noticeably better clustering results in terms of lower SSE. This technique is similar to the idea just discussed of picking the first centroid at random and then picking each remaining centroid as the point as far from the remaining centroids as possible. Specifically, K-means++ picks centroids incrementally until k centroids have been picked. At every such step, each point has a probability of being picked as the new centroid that is proportional to the square of its distance to its closest centroid. It might seem that this approach might tend to choose outliers for centroids, but because outliers are rare, by definition, this is unlikely.

The details of K-means++ initialization are given by Algorithm 5.2. The rest of the algorithm is the same as ordinary K-means.

Algorithm 5.2 K-means++ initialization algorithm.

1: For the first centroid, pick one of the points at random.
2: **for** $i = 1$ to *number of trials* **do**
3: Compute the distance, $d(x)$, of each point to its closest centroid.
4: Assign each point a probability proportional to each point's $d(x)^2$.
5: Pick new centroid from the remaining points using the weighted probabilities.
6: **end for**

Later, we will discuss two other approaches that are also useful for producing better-quality (lower SSE) clusterings: using a variant of K-means that is less susceptible to initialization problems (bisecting K-means) and using postprocessing to "fix up" the set of clusters produced. K-means++ could be combined with either approach.

Time and Space Complexity

The space requirements for K-means are modest because only the data points and centroids are stored. Specifically, the storage required is $O((m + K)n)$, where m is the number of points and n is the number of attributes. The time requirements for K-means are also modest—basically linear in the number of data points. In particular, the time required is $O(I \times K \times m \times n)$, where I is the number of iterations required for convergence. As mentioned, I is often small and can usually be safely bounded, as most changes typically occur in the first few iterations. Therefore, K-means is linear in m, the number of points, and is efficient as well as simple provided that K, the number of clusters, is significantly less than m.

5.2.2 K-means: Additional Issues

Handling Empty Clusters

One of the problems with the basic K-means algorithm is that empty clusters can be obtained if no points are allocated to a cluster during the assignment step. If this happens, then a strategy is needed to choose a replacement centroid, since otherwise, the squared error will be larger than necessary. One approach is to choose the point that is farthest away from any current centroid. If nothing else, this eliminates the point that currently contributes most to the

total squared error. (A K-means++ approach could be used as well.) Another approach is to choose the replacement centroid at random from the cluster that has the highest SSE. This will typically split the cluster and reduce the overall SSE of the clustering. If there are several empty clusters, then this process can be repeated several times.

Outliers

When the squared error criterion is used, outliers can unduly influence the clusters that are found. In particular, when outliers are present, the resulting cluster centroids (prototypes) are typically not as representative as they otherwise would be and thus, the SSE will be higher. Because of this, it is often useful to discover outliers and eliminate them beforehand. It is important, however, to appreciate that there are certain clustering applications for which outliers should not be eliminated. When clustering is used for data compression, every point must be clustered, and in some cases, such as financial analysis, apparent outliers, e.g., unusually profitable customers, can be the most interesting points.

An obvious issue is how to identify outliers. A number of techniques for identifying outliers will be discussed in Chapter 9. If we use approaches that remove outliers before clustering, we avoid clustering points that will not cluster well. Alternatively, outliers can also be identified in a postprocessing step. For instance, we can keep track of the SSE contributed by each point, and eliminate those points with unusually high contributions, especially over multiple runs. Also, we often want to eliminate small clusters because they frequently represent groups of outliers.

Reducing the SSE with Postprocessing

An obvious way to reduce the SSE is to find more clusters, i.e., to use a larger K. However, in many cases, we would like to improve the SSE, but don't want to increase the number of clusters. This is often possible because K-means typically converges to a local minimum. Various techniques are used to "fix up" the resulting clusters in order to produce a clustering that has lower SSE. The strategy is to focus on individual clusters since the total SSE is simply the sum of the SSE contributed by each cluster. (We will use the terms *total SSE* and *cluster SSE*, respectively, to avoid any potential confusion.) We can change the total SSE by performing various operations on the clusters, such as splitting or merging clusters. One commonly used approach is to employ alternate cluster splitting and merging phases. During a splitting phase, clusters are divided,

while during a merging phase, clusters are combined. In this way, it is often possible to escape local SSE minima and still produce a clustering solution with the desired number of clusters. The following are some techniques used in the splitting and merging phases.

Two strategies that decrease the total SSE by increasing the number of clusters are the following:

Split a cluster: The cluster with the largest SSE is usually chosen, but we could also split the cluster with the largest standard deviation for one particular attribute.

Introduce a new cluster centroid: Often the point that is farthest from any cluster center is chosen. We can easily determine this if we keep track of the SSE contributed by each point. Another approach is to choose randomly from all points or from the points with the highest SSE with respect to their closest centroids.

Two strategies that decrease the number of clusters, while trying to minimize the increase in total SSE, are the following:

Disperse a cluster: This is accomplished by removing the centroid that corresponds to the cluster and reassigning the points to other clusters. Ideally, the cluster that is dispersed should be the one that increases the total SSE the least.

Merge two clusters: The clusters with the closest centroids are typically chosen, although another, perhaps better, approach is to merge the two clusters that result in the smallest increase in total SSE. These two merging strategies are the same ones that are used in the hierarchical clustering techniques known as the centroid method and Ward's method, respectively. Both methods are discussed in Section 5.3.

Updating Centroids Incrementally

Instead of updating cluster centroids after all points have been assigned to a cluster, the centroids can be updated incrementally, after each assignment of a point to a cluster. Notice that this requires either zero or two updates to cluster centroids at each step, since a point either moves to a new cluster (two updates) or stays in its current cluster (zero updates). Using an incremental update strategy guarantees that empty clusters are not produced because all

clusters start with a single point, and if a cluster ever has only one point, then that point will always be reassigned to the same cluster.

In addition, if incremental updating is used, the relative weight of the point being added can be adjusted; e.g., the weight of points is often decreased as the clustering proceeds. While this can result in better accuracy and faster convergence, it can be difficult to make a good choice for the relative weight, especially in a wide variety of situations. These update issues are similar to those involved in updating weights for artificial neural networks.

Yet another benefit of incremental updates has to do with using objectives other than "minimize SSE." Suppose that we are given an arbitrary objective function to measure the goodness of a set of clusters. When we process an individual point, we can compute the value of the objective function for each possible cluster assignment, and then choose the one that optimizes the objective. Specific examples of alternative objective functions are given in Section 5.5.2.

On the negative side, updating centroids incrementally introduces an order dependency. In other words, the clusters produced usually depend on the order in which the points are processed. Although this can be addressed by randomizing the order in which the points are processed, the basic K-means approach of updating the centroids after all points have been assigned to clusters has no order dependency. Also, incremental updates are slightly more expensive. However, K-means converges rather quickly, and therefore, the number of points switching clusters quickly becomes relatively small.

5.2.3 Bisecting K-means

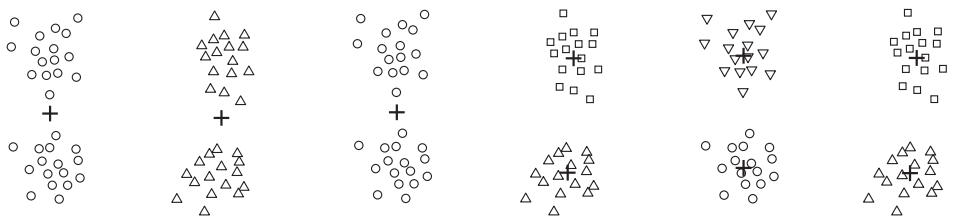
The bisecting K-means algorithm is a straightforward extension of the basic K-means algorithm that is based on a simple idea: to obtain K clusters, split the set of all points into two clusters, select one of these clusters to split, and so on, until K clusters have been produced. The details of bisecting K-means are given by Algorithm 5.3.

There are a number of different ways to choose which cluster to split. We can choose the largest cluster at each step, choose the one with the largest SSE, or use a criterion based on both size and SSE. Different choices result in different clusters.

Because we are using the K-means algorithm "locally," i.e., to bisect individual clusters, the final set of clusters does not represent a clustering that is a local minimum with respect to the total SSE. Thus, we often refine the resulting clusters by using their cluster centroids as the initial centroids for the standard K-means algorithm.

Algorithm 5.3 Bisecting K-means algorithm.

```
1: Initialize the list of clusters to contain the cluster consisting of all points.
2: repeat
3:    Remove a cluster from the list of clusters.
4:    {Perform several "trial" bisections of the chosen cluster.}
5:    for i = 1 to number of trials do
6:       Bisect the selected cluster using basic K-means.
7:    end for
8:    Select the two clusters from the bisection with the lowest total SSE.
9:    Add these two clusters to the list of clusters.
10: until The list of clusters contains K clusters.
```



(a) Iteration 1. (b) Iteration 2. (c) Iteration 3.

Figure 5.8. Bisecting K-means on the four clusters example.

Example 5.3 (Bisecting K-means and Initialization)**.** To illustrate that bisecting K-means is less susceptible to initialization problems, we show, in Figure 5.8, how bisecting K-means finds four clusters in the data set originally shown in Figure 5.6(a). In iteration 1, two pairs of clusters are found; in iteration 2, the rightmost pair of clusters is split; and in iteration 3, the leftmost pair of clusters is split. Bisecting K-means has less trouble with initialization because it performs several trial bisections and takes the one with the lowest SSE, and because there are only two centroids at each step. ■

Finally, by recording the sequence of clusterings produced as K-means bisects clusters, we can also use bisecting K-means to produce a hierarchical clustering.

5.2.4 K-means and Different Types of Clusters

K-means and its variations have a number of limitations with respect to finding different types of clusters. In particular, K-means has difficulty detecting the

"natural" clusters, when clusters have non-spherical shapes or widely different sizes or densities. This is illustrated by Figures 5.9, 5.10, and 5.11. In Figure 5.9, K-means cannot find the three natural clusters because one of the clusters is much larger than the other two, and hence, the larger cluster is broken, while one of the smaller clusters is combined with a portion of the larger cluster. In Figure 5.10, K-means fails to find the three natural clusters because the two smaller clusters are much denser than the larger cluster. Finally, in Figure 5.11, K-means finds two clusters that mix portions of the two natural clusters because the shape of the natural clusters is not globular.

The difficulty in these three situations is that the K-means objective function is a mismatch for the kinds of clusters we are trying to find because it is minimized by globular clusters of equal size and density or by clusters that are well separated. However, these limitations can be overcome, in some sense, if the user is willing to accept a clustering that breaks the natural clusters into a number of subclusters. Figure 5.12 shows what happens to the three previous data sets if we find six clusters instead of two or three. Each smaller cluster is pure in the sense that it contains only points from one of the natural clusters.

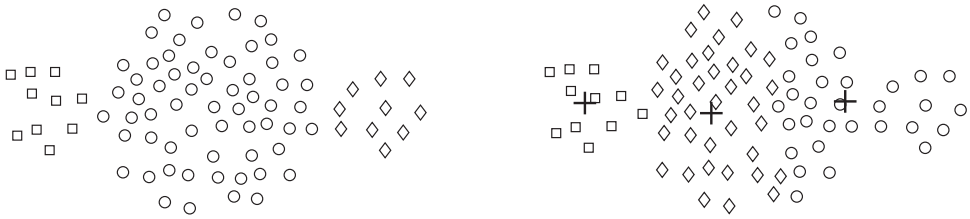
5.2.5 Strengths and Weaknesses

K-means is simple and can be used for a wide variety of data types. It is also quite efficient, even though multiple runs are often performed. Some variants, including bisecting K-means, are even more efficient, and are less susceptible to initialization problems. K-means is not suitable for all types of data, however. It cannot handle non-globular clusters or clusters of different sizes and densities, although it can typically find pure subclusters if a large enough number of clusters is specified. K-means also has trouble clustering data that contains outliers. Outlier detection and removal can help significantly in such situations. Finally, K-means is restricted to data for which there is a notion of a center (centroid). A related technique, K-medoid clustering, does not have this restriction, but is more expensive.

5.2.6 K-means as an Optimization Problem

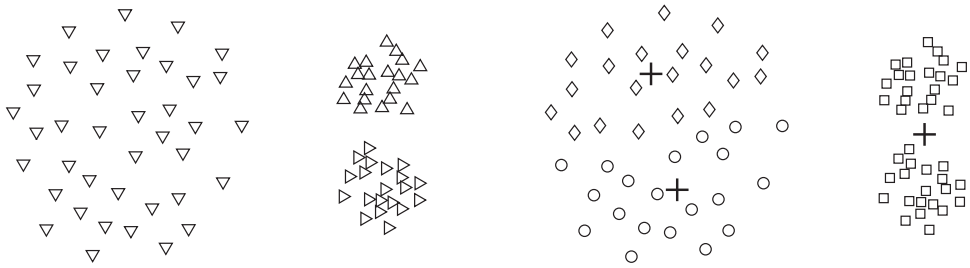
Here, we delve into the mathematics behind K-means. This section, which can be skipped without loss of continuity, requires knowledge of calculus through partial derivatives. Familiarity with optimization techniques, especially those based on gradient descent, can also be helpful.

As mentioned earlier, given an objective function such as "minimize SSE," clustering can be treated as an optimization problem. One way to solve this



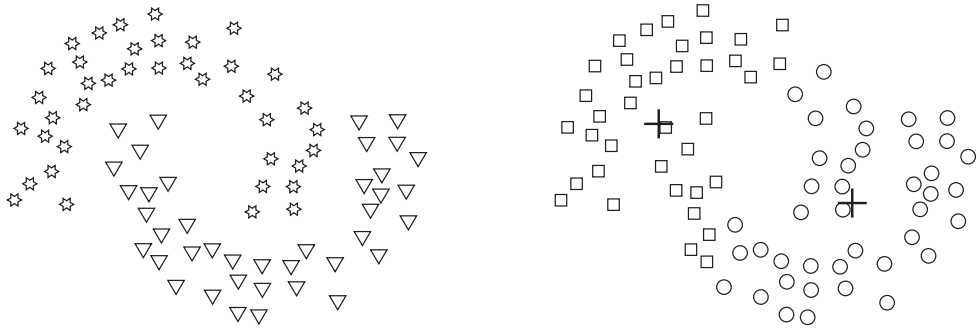
(a) Original points. (b) Three K-means clusters.

Figure 5.9. K-means with clusters of different size.



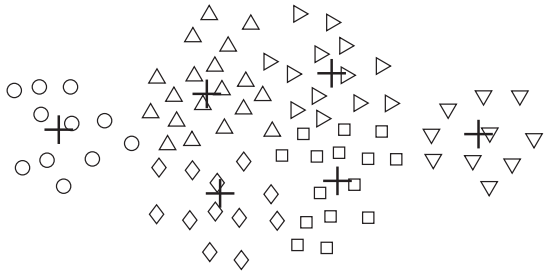
(a) Original points. (b) Three K-means clusters.

Figure 5.10. K-means with clusters of different density.

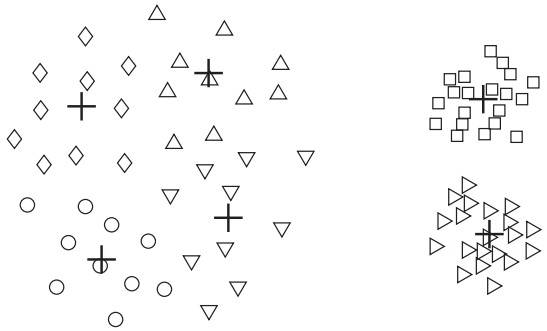


(a) Original points. (b) Two K-means clusters.

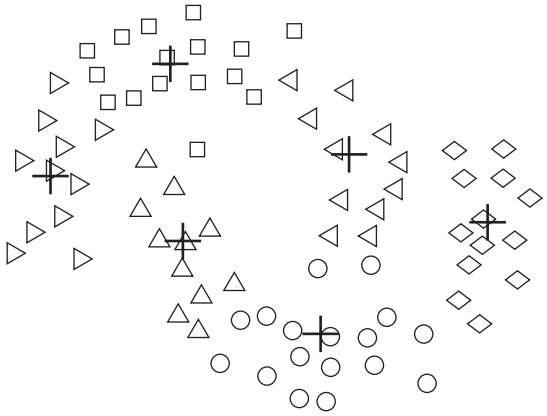
Figure 5.11. K-means with non-globular clusters.



(a) Unequal sizes.



(b) Unequal densities.



(c) Non-spherical shapes.

Figure 5.12. Using K-means to find clusters that are subclusters of the natural clusters.

problem—to find a global optimum—is to enumerate all possible ways of dividing the points into clusters and then choose the set of clusters that best satisfies the objective function, e.g., that minimizes the total SSE. Of course, this exhaustive strategy is computationally infeasible and as a result, a more practical approach is needed, even if such an approach finds solutions that are not guaranteed to be optimal. One technique, which is known as **gradient descent**, is based on picking an initial solution and then repeating the following two steps: compute the change to the solution that best optimizes the objective function and then update the solution.

We assume that the data is one-dimensional, i.e., $dist(x, y) = (x - y)^2$. This does not change anything essential, but greatly simplifies the notation.

Derivation of K-means as an Algorithm to Minimize the SSE

In this section, we show how the centroid for the K-means algorithm can be mathematically derived when the proximity function is Euclidean distance and the objective is to minimize the SSE. Specifically, we investigate how we can best update a cluster centroid so that the cluster SSE is minimized. In mathematical terms, we seek to minimize Equation 5.1, which we repeat here, specialized for one-dimensional data.

$$\text{SSE} = \sum_{i=1}^{K} \sum_{x \in C_i} (c_i - x)^2 \tag{5.4}$$

Here, C_i is the ith cluster, x is a point in C_i, and c_i is the mean of the ith cluster. See Table 5.1 for a complete list of notation.

We can solve for the kth centroid c_k, which minimizes Equation 5.4, by differentiating the SSE, setting it equal to 0, and solving, as indicated below.

$$\begin{aligned} \frac{\partial}{\partial c_k}\text{SSE} &= \frac{\partial}{\partial c_k} \sum_{i=1}^{K} \sum_{x \in C_i} (c_i - x)^2 \\ &= \sum_{i=1}^{K} \sum_{x \in C_i} \frac{\partial}{\partial c_k} (c_i - x)^2 \\ &= \sum_{x \in C_k} 2 \times (c_k - x_k) = 0 \end{aligned}$$

$$\sum_{x \in C_k} 2 \times (c_k - x_k) = 0 \Rightarrow m_k c_k = \sum_{x \in C_k} x_k \Rightarrow c_k = \frac{1}{m_k} \sum_{x \in C_k} x_k$$

Thus, as previously indicated, the best centroid for minimizing the SSE of a cluster is the mean of the points in the cluster.

Derivation of K-means for SAE

To demonstrate that the K-means algorithm can be applied to a variety of different objective functions, we consider how to partition the data into K clusters such that the sum of the Manhattan (L_1) distances of points from the center of their clusters is minimized. We are seeking to minimize the sum of the L_1 absolute errors (SAE) as given by the following equation, where $dist_{L_1}$ is the L_1 distance. Again, for notational simplicity, we use one-dimensional data, i.e., $dist_{L_1} = |c_i - x|$.

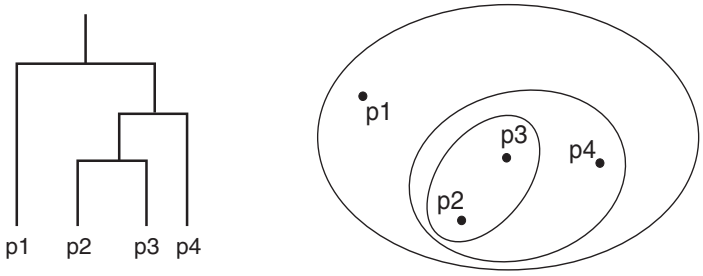
$$\text{SAE} = \sum_{i=1}^{K} \sum_{x \in C_i} dist_{L_1}(c_i, x) \tag{5.5}$$

We can solve for the kth centroid c_k, which minimizes Equation 5.5, by differentiating the SAE, setting it equal to 0, and solving.

$$\begin{aligned} \frac{\partial}{\partial c_k}\text{SAE} &= \frac{\partial}{\partial c_k}\sum_{i=1}^{K}\sum_{x \in C_i} |c_i - x| \\ &= \sum_{i=1}^{K}\sum_{x \in C_i} \frac{\partial}{\partial c_k}|c_i - x| \\ &= \sum_{x \in C_k} \frac{\partial}{\partial c_k}|c_k - x| = 0 \end{aligned}$$

$$\sum_{x \in C_k} \frac{\partial}{\partial c_k}|c_k - x| = 0 \Rightarrow \sum_{x \in C_k} sign(x - c_k) = 0$$

If we solve for c_k, we find that $c_k = median\{x \in C_k\}$, the median of the points in the cluster. The median of a group of points is straightforward to compute and less susceptible to distortion by outliers.



(a) Dendrogram. (b) Nested cluster diagram.

Figure 5.13. A hierarchical clustering of four points shown as a dendrogram and as nested clusters.

5.3 Agglomerative Hierarchical Clustering

Hierarchical clustering techniques are a second important category of clustering methods. As with K-means, these approaches are relatively old compared to many clustering algorithms, but they still enjoy widespread use. There are two basic approaches for generating a hierarchical clustering:

Agglomerative: Start with the points as individual clusters and, at each step, merge the closest pair of clusters. This requires defining a notion of cluster proximity.

Divisive: Start with one, all-inclusive cluster and, at each step, split a cluster until only singleton clusters of individual points remain. In this case, we need to decide which cluster to split at each step and how to do the splitting.

Agglomerative hierarchical clustering techniques are by far the most common, and, in this section, we will focus exclusively on these methods. A divisive hierarchical clustering technique is described in Section 8.4.2.

A hierarchical clustering is often displayed graphically using a tree-like diagram called a **dendrogram**, which displays both the cluster-subcluster relationships and the order in which the clusters were merged (agglomerative view) or split (divisive view). For sets of two-dimensional points, such as those that we will use as examples, a hierarchical clustering can also be graphically represented using a nested cluster diagram. Figure 5.13 shows an example of these two types of figures for a set of four two-dimensional points. These points were clustered using the single-link technique that is described in Section 5.3.2.

5.5 Cluster Evaluation

In supervised classification, the evaluation of the resulting classification model is an integral part of the process of developing a classification model, and there are well-accepted evaluation measures and procedures, e.g., accuracy and cross-validation, respectively. However, because of its very nature, cluster evaluation is not a well-developed or commonly used part of cluster analysis. Nonetheless, cluster evaluation, or **cluster validation** as it is more traditionally called, is important, and this section will review some of the most common and easily applied approaches.

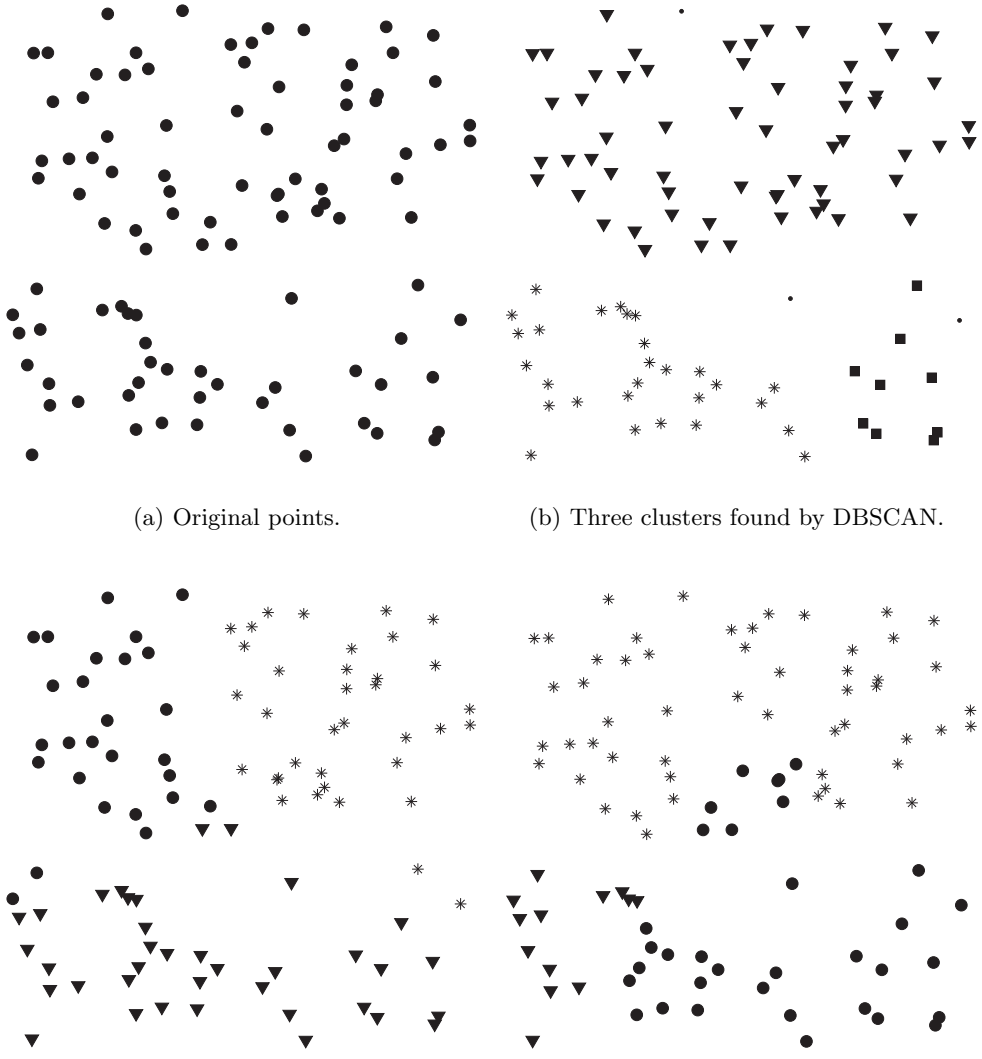
There might be some confusion as to why cluster evaluation is necessary. Many times, cluster analysis is conducted as a part of an exploratory data analysis. Hence, evaluation seems to be an unnecessarily complicated addition to what is supposed to be an informal process. Furthermore, because there are a number of different types of clusters—in some sense, each clustering algorithm defines its own type of cluster—it can seem that each situation might require a different evaluation measure. For instance, K-means clusters might be evaluated in terms of the SSE, but for density-based clusters, which need not be globular, SSE would not work well at all.

Nonetheless, cluster evaluation should be a part of any cluster analysis. A key motivation is that almost every clustering algorithm will find clusters in a data set, even if that data set has no natural cluster structure. For instance, consider Figure 5.26, which shows the result of clustering 100 points that are randomly (uniformly) distributed on the unit square. The original points are shown in Figure 5.26(a), while the clusters found by DBSCAN, K-means, and complete link are shown in Figures 5.26(b), 5.26(c), and 5.26(d), respectively. Since DBSCAN found three clusters (after we set *Eps* by looking at the distances of the fourth nearest neighbors), we set K-means and complete link to find three clusters as well. (In Figure 5.26(b) the noise is shown by the small markers.) However, the clusters do not look compelling for any of the three methods. In higher dimensions, such problems cannot be so easily detected.

5.5.1 Overview

Being able to distinguish whether there is non-random structure in the data is just one important aspect of cluster validation. The following is a list of several important issues for cluster validation.

1. Determining the **clustering tendency** of a set of data, i.e., distinguishing whether non-random structure actually exists in the data.



(a) Original points.

(b) Three clusters found by DBSCAN.

(c) Three clusters found by K-means.

(d) Three clusters found by complete link.

Figure 5.26. Clustering of 100 uniformly distributed points.

2. Determining the correct number of clusters.
3. Evaluating how well the results of a cluster analysis fit the data *without* reference to external information.
4. Comparing the results of a cluster analysis to externally known results, such as externally provided class labels.
5. Comparing two sets of clusters to determine which is better.

Notice that items 1, 2, and 3 do not make use of any external information—they are unsupervised techniques—while item 4 requires external information. Item 5 can be performed in either a supervised or an unsupervised manner. A further distinction can be made with respect to items 3, 4, and 5: Do we want to evaluate the entire clustering or just individual clusters?

While it is possible to develop various numerical measures to assess the different aspects of cluster validity mentioned above, there are a number of challenges. First, a measure of cluster validity is sometimes quite limited in the scope of its applicability. For example, most work on measures of clustering tendency has been done for two- or three-dimensional spatial data. Second, we need a framework to interpret any measure. If we obtain a value of 10 for a measure that evaluates how well cluster labels match externally provided class labels, does this value represent a good, fair, or poor match? The goodness of a match often can be measured by looking at the statistical distribution of this value, i.e., how likely it is that such a value occurs by chance. Finally, if a measure is too complicated to apply or to understand, then few will use it.

The evaluation measures, or indices, that are applied to judge various aspects of cluster validity are traditionally classified into the following three types.

Unsupervised. Measures the goodness of a clustering structure without respect to external information. An example of this is the SSE. Unsupervised measures of cluster validity are often further divided into two classes: measures of **cluster cohesion** (compactness, tightness), which determine how closely related the objects in a cluster are, and measures of **cluster separation** (isolation), which determine how distinct or well-separated a cluster is from other clusters. Unsupervised measures are often called **internal indices** because they use only information present in the data set.

Supervised. Measures the extent to which the clustering structure discovered by a clustering algorithm matches some external structure. An example of a supervised index is entropy, which measures how well cluster

labels match externally supplied class labels. Supervised measures are often called **external indices** because they use information not present in the data set.

Relative. Compares different clusterings or clusters. A relative cluster evaluation measure is a supervised or unsupervised evaluation measure that is used for the purpose of comparison. Thus, relative measures are not actually a separate type of cluster evaluation measure, but are instead a specific use of such measures. As an example, two K-means clusterings can be compared using either the SSE or entropy.

In the remainder of this section, we provide specific details concerning cluster validity. We first describe topics related to unsupervised cluster evaluation, beginning with (1) measures based on cohesion and separation, and (2) two techniques based on the proximity matrix. Since these approaches are useful only for partitional sets of clusters, we also describe the popular cophenetic correlation coefficient, which can be used for the unsupervised evaluation of a hierarchical clustering. We end our discussion of unsupervised evaluation with brief discussions about finding the correct number of clusters and evaluating clustering tendency. We then consider supervised approaches to cluster validity, such as entropy, purity, and the Jaccard measure. We conclude this section with a short discussion of how to interpret the values of (unsupervised or supervised) validity measures.

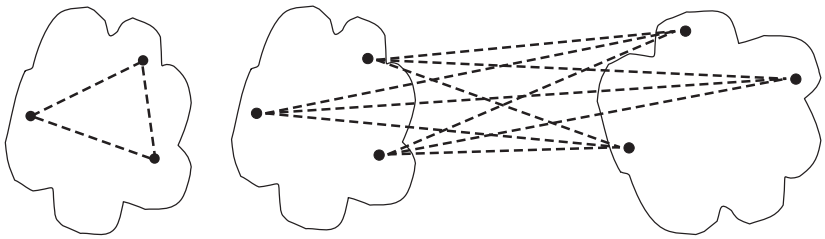
5.5.2 Unsupervised Cluster Evaluation Using Cohesion and Separation

Many internal measures of cluster validity for partitional clustering schemes are based on the notions of cohesion or separation. In this section, we use cluster validity measures for prototype- and graph-based clustering techniques to explore these notions in some detail. In the process, we will also see some interesting relationships between prototype- and graph-based measures.

In general, we can consider expressing overall cluster validity for a set of K clusters as a weighted sum of the validity of individual clusters,

$$\textit{overall validity} = \sum_{i=1}^{K} w_i \; \textit{validity}(C_i). \tag{5.8}$$

The *validity* function can be cohesion, separation, or some combination of these quantities. The weights will vary depending on the cluster validity measure. In some cases, the weights are simply 1 or the size of the cluster, while



(a) Cohesion. (b) Separation.

Figure 5.27. Graph-based view of cluster cohesion and separation.

in other cases to be discussed a bit later, they reflect a more complicated property of the cluster.

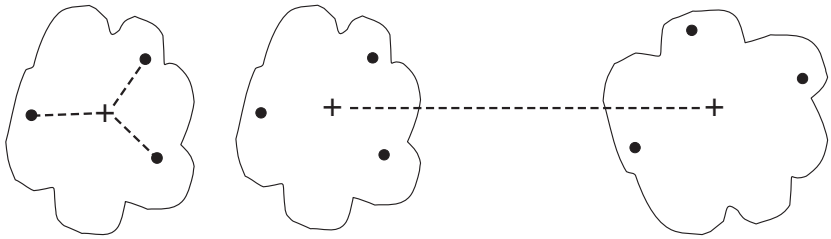
Graph-Based View of Cohesion and Separation

From a graph-based view, the cohesion of a cluster can be defined as the sum of the weights of the links in the proximity graph that connect points within the cluster. See Figure 5.27(a). (Recall that the proximity graph has data objects as nodes, a link between each pair of data objects, and a weight assigned to each link that is the proximity between the two data objects connected by the link.) Likewise, the separation between two clusters can be measured by the sum of the weights of the links from points in one cluster to points in the other cluster. This is illustrated in Figure 5.27(b).

Most simply, the cohesion and separation for a graph-based cluster can be expressed using Equations 5.9 and 5.10, respectively. The *proximity* function can be a similarity or a dissimilarity. For similarity, as in Table 5.6, higher values are better for cohesion while lower values are better for separation. For dissimilarity, the opposite is true, i.e., lower values are better for cohesion while higher values are better for separation. More complicated approaches are possible but typically embody the basic ideas of figure 5.27a and 5.27b.

$$cohesion(C_i) = \sum_{\substack{\mathbf{x} \in C_i \\ \mathbf{y} \in C_i}} proximity(\mathbf{x}, \mathbf{y}) \tag{5.9}$$

$$separation(C_i, C_j) = \sum_{\substack{\mathbf{x} \in C_i \\ \mathbf{y} \in C_j}} proximity(\mathbf{x}, \mathbf{y}) \tag{5.10}$$



(a) Cohesion. (b) Separation.

Figure 5.28. Prototype-based view of cluster cohesion and separation.

Prototype-Based View of Cohesion and Separation

For prototype-based clusters, the cohesion of a cluster can be defined as the sum of the proximities with respect to the prototype (centroid or medoid) of the cluster. Similarly, the separation between two clusters can be measured by the proximity of the two cluster prototypes. This is illustrated in Figure 5.28, where the centroid of a cluster is indicated by a "+".

Cohesion for a prototype-based cluster is given in Equation 5.11, while two measures for separation are given in Equations 5.12 and 5.13, respectively, where $\mathbf{c}_i$ is the prototype (centroid) of cluster C_i and $\mathbf{c}$ is the overall prototype (centroid). There are two measures for separation because, as we will see in the next section, the separation of cluster prototypes from an overall prototype is sometimes directly related to the separation of cluster prototypes from one another. (This is true, for example, for Euclidean distance.) Note that Equation 5.11 is the cluster SSE if we let proximity be the squared Euclidean distance.

$$cohesion(C_i) = \sum_{\mathbf{x} \in C_i} proximity(\mathbf{x}, \mathbf{c}_i) \tag{5.11}$$

$$separation(C_i, C_j) = proximity(\mathbf{c}_i, \mathbf{c}_j) \tag{5.12}$$

$$separation(C_i) = proximity(\mathbf{c}_i, \mathbf{c}) \tag{5.13}$$

Relationship between Prototype-Based Cohesion and Graph-Based Cohesion

While the graph-based and prototype-based approaches to measuring the cohesion and separation of a cluster seem distinct, for some proximity measures they are equivalent. For instance, for the SSE and points in Euclidean space, it can be shown (Equation 5.14) that the average pairwise distance between the points in a cluster is equivalent to the SSE of the cluster. See Exercise 27 on page 392.

$$\text{Cluster SSE} = \sum_{\mathbf{x} \in C_i} dist(\mathbf{c}_i, \mathbf{x})^2 = \frac{1}{2m_i} \sum_{\mathbf{x} \in C_i} \sum_{\mathbf{y} \in C_i} dist(\mathbf{x}, \mathbf{y})^2 \tag{5.14}$$

Relationship of the Two Approaches to Prototype-Based Separation

When proximity is measured by Euclidean distance, the traditional measure of separation between clusters is the between group sum of squares (SSB), which is the sum of the squared distance of a cluster centroid, $\mathbf{c}_i$, to the overall mean, $\mathbf{c}$, of all the data points. The SSB is given by Equation 5.15, where $\mathbf{c}_i$ is the mean of the ith cluster and $\mathbf{c}$ is the overall mean. The higher the total SSB of a clustering, the more separated the clusters are from one another.

$$\text{Total SSB} = \sum_{i=1}^{K} m_i \; dist(\mathbf{c}_i, \mathbf{c})^2 \tag{5.15}$$

It is straightforward to show that the total SSB is directly related to the pairwise distances between the centroids. In particular, if the cluster sizes are equal, i.e., $m_i = m/K$, then this relationship takes the simple form given by Equation 5.16. (See Exercise 28 on page 392.) It is this type of equivalence that motivates the definition of prototype separation in terms of both Equations 5.12 and 5.13.

$$\text{Total SSB} = \frac{1}{2K} \sum_{i=1}^{K} \sum_{j=1}^{K} \frac{m}{K} \; dist(\mathbf{c}_i, \mathbf{c}_j)^2 \tag{5.16}$$

Relationship between Cohesion and Separation

For some validity measures, there is also a strong relationship between cohesion and separation. Specifically, it is possible to show that the sum of the total SSE and the total SSB is a constant; i.e., that it is equal to the total sum of squares (TSS), which is the sum of squares of the distance of each point to

the overall mean of the data. The importance of this result is that minimizing SSE (cohesion) is equivalent to maximizing SSB (separation).

We provide the proof of this fact below, since the approach illustrates techniques that are also applicable to proving the relationships stated in the last two sections. To simplify the notation, we assume that the data is one-dimensional, i.e., $dist(x, y) = (x-y)^2$. Also, we use the fact that the cross-term $\sum_{i=1}^{K} \sum_{x \in C_i} (x - c_i)(c - c_i)$ is 0. (See Exercise 29 on page 392.)

$$\begin{aligned}
\text{TSS} &= \sum_{i=1}^{K} \sum_{x \in C_i} (x - c)^2 \\
&= \sum_{i=1}^{K} \sum_{x \in C_i} ((x - c_i) - (c - c_i))^2 \\
&= \sum_{i=1}^{K} \sum_{x \in C_i} (x - c_i)^2 - 2 \sum_{i=1}^{K} \sum_{x \in C_i} (x - c_i)(c - c_i) + \sum_{i=1}^{K} \sum_{x \in C_i} (c - c_i)^2 \\
&= \sum_{i=1}^{K} \sum_{x \in C_i} (x - c_i)^2 + \sum_{i=1}^{K} \sum_{x \in C_i} (c - c_i)^2 \\
&= \sum_{i=1}^{K} \sum_{x \in C_i} (x - c_i)^2 + \sum_{i=1}^{K} |C_i|(c - c_i)^2 \\
&= \text{SSE} + \text{SSB}
\end{aligned}$$

Relationship between Graph- and Centroid-Based Cohesion

It can also be shown that there is a relationship between graph- and centroid-based cohesion measures for Euclidean distance. For simplicity, we once again assume one-dimensional data. Recall that $c_i = 1/m_i \sum_{y \in C_i} y$.

$$
\begin{aligned}
m_i^2 cohesion(C_i) &= m_i^2 \sum_{x \in C_i} proximity(x, c_i) \\
&= \sum_{x \in C_i} m_i^2 (x - c_i)^2 \\
&= \sum_{x \in C_i} (m_i x - m_i c_i)^2 \\
&= \sum_{x \in C_i} (m_i x - m_i (1/m_i \sum_{y \in C_i} y))^2 \\
&= \sum_{x \in C_i} \sum_{y \in C_i} (x - y)^2 \\
&= \sum_{\substack{x \in C_i \\ y \in C_i}} (x - y)^2 \\
&= \sum_{\substack{x \in C_i \\ y \in C_i}} proximity(x, y)
\end{aligned}
$$

More generally, in cases where a centroid makes sense for the data, the simple graph or centroid-based measures of cluster validity we presented are often related.

Overall Measures of Cohesion and Separation

The previous definitions of cluster cohesion and separation gave us some simple and well-defined measures of individual cluster validity that can be combined into an overall measure of cluster validity by using a weighted sum, as indicated in Equation 5.8. However, we need to decide what weights to use. Not surprisingly, the weights used can vary widely. Often, but not always, they are either a function of cluster size or 1, which treats all clusters equally regardless of size.

The CLUstering TOolkit (CLUTO) (see the Bibliographic Notes) uses the cluster evaluation measures described in Table 5.6, as well as some other evaluation measures not mentioned here. Only similarity measures are used: cosine, correlation, Jaccard, and the inverse of Euclidean distance. $\mathcal{I}_1$ is a measure of cohesion in terms of the pairwise similarity of objects in the cluster. $\mathcal{I}_2$ is a measure of cohesion that can be expressed either in terms of the sum of the similarities of objects in the cluster to the cluster centroid or in terms of the pairwise similarities of objects in the cluster. $\mathcal{E}_1$ is a measure of separation.

It can be defined in terms of the similarity of a cluster centroid to the overall centroid or in terms of the pairwise similarities of object in the cluster to objects in other clusters. (Although $\mathcal{E}_1$ is a measure of separation, the second definition shows that it also uses cluster cohesion, albeit in the cluster weight.) $\mathcal{G}_1$, which is a cluster validity measure based on both cohesion and separation, is the sum of the pairwise similarities of all objects in the cluster with all objects outside the cluster—the total weight of the edges of the similarity graph that must be cut to separate the cluster from all other clusters—divided by the sum of the pairwise similarities of objects in the cluster.

Table 5.6. Table of graph-based cluster evaluation measures.

Name	Cluster Measure	Cluster Weight	Type
$\mathcal{I}_1$	$\sum_{\substack{\mathbf{x}\in C_i \\ \mathbf{y}\in C_i}} sim(\mathbf{x},\mathbf{y})$	$\frac{1}{m_i}$	graph-based cohesion
$\mathcal{I}_2$	$\sum_{\mathbf{x}\in C_i} sim(\mathbf{x},\mathbf{c}_i)$		prototype-based cohesion
$\mathcal{I}_2$	$\sqrt{\sum_{\substack{\mathbf{x}\in C_i \\ \mathbf{y}\in C_i}} sim(\mathbf{x},\mathbf{y})}$		prototype-based cohesion
$\mathcal{E}_1$	$sim(\mathbf{c}_i,\mathbf{c})$	m_i	prototype-based separation
$\mathcal{E}_1$	$\sum_{j=1}^{k}\sum_{\substack{\mathbf{x}\in C_i \\ \mathbf{y}\in C_j}} sim(\mathbf{x},\mathbf{y})$	$\frac{m_i}{\sqrt{(\sum_{\substack{\mathbf{x}\in C_i \\ \mathbf{y}\in C_i}} sim(\mathbf{x},\mathbf{y}))}}$	graph-based separation
$\mathcal{G}_1$	$\sum_{\substack{j=1 \\ j\neq i}}^{k}\sum_{\substack{\mathbf{x}\in C_i \\ \mathbf{y}\in C_j}} sim(\mathbf{x},\mathbf{y})$	$\frac{1}{\sum_{\substack{\mathbf{x}\in C_i \\ \mathbf{y}\in C_i}} sim(\mathbf{x},\mathbf{y})}$	graph-based separation and cohesion

Note that any unsupervised measure of cluster validity potentially can be used as an objective function for a clustering algorithm and vice versa. CLUTO

takes this approach by using an algorithm that is similar to the incremental K-means algorithm discussed in Section 5.2.2. Specifically, each point is assigned to the cluster that produces the best value for the cluster evaluation function. The cluster evaluation measure $\mathcal{I}_2$ corresponds to traditional K-means and produces clusters that have good SSE values. The other measures produce clusters that are not as good with respect to SSE, but that are more optimal with respect to the specified cluster validity measure.

Evaluating Individual Clusters and Objects

So far, we have focused on using cohesion and separation in the overall evaluation of a group of clusters. Most of these measures of cluster validity also can be used to evaluate individual clusters and objects. For example, we can rank individual clusters according to their specific value of cluster validity, i.e., cluster cohesion or separation. A cluster that has a high value of cohesion may be considered better than a cluster that has a lower value. This information often can be used to improve the quality of a clustering. If, for example, a cluster is not very cohesive, then we may want to split it into several subclusters. On the other hand, if two clusters are relatively cohesive, but not well separated, we may want to merge them into a single cluster.

We can also evaluate the objects within a cluster in terms of their contribution to the overall cohesion or separation of the cluster. Objects that contribute more to the cohesion and separation are near the "interior" of the cluster. Those objects for which the opposite is true are probably near the "edge" of the cluster. In the following section, we consider a cluster evaluation measure that uses an approach based on these ideas to evaluate points, clusters, and the entire set of clusters.

The Silhouette Coefficient

The popular method of silhouette coefficients combines both cohesion and separation. The following steps explain how to compute the silhouette coefficient for an individual point, a process that consists of the following three steps. We use distances, but an analogous approach can be used for similarities.

1. For the ith object, calculate its average distance to all other objects in its cluster. Call this value a_i.
2. For the ith object and any cluster not containing the object, calculate the object's average distance to all the objects in the given cluster. Find the minimum such value with respect to all clusters; call this value b_i.

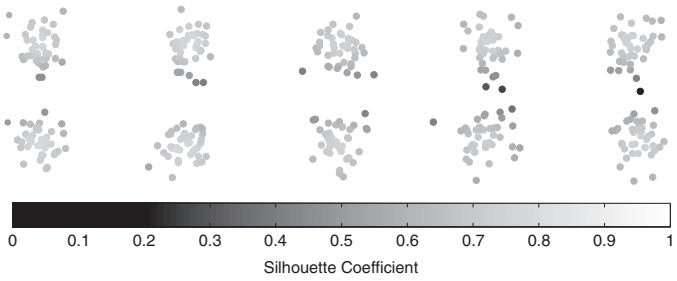


Figure 5.29. Silhouette coefficients for points in ten clusters.

3. For the ith object, the silhouette coefficient is $s_i = (b_i - a_i)/\max(a_i, b_i)$.

The value of the silhouette coefficient can vary between -1 and 1. A negative value is undesirable because this corresponds to a case in which a_i, the average distance to points in the cluster, is greater than b_i, the minimum average distance to points in another cluster. We want the silhouette coefficient to be positive ($a_i < b_i$), and for a_i to be as close to 0 as possible, since the coefficient assumes its maximum value of 1 when $a_i = 0$.

We can compute the average silhouette coefficient of a cluster by simply taking the average of the silhouette coefficients of points belonging to the cluster. An overall measure of the goodness of a clustering can be obtained by computing the average silhouette coefficient of all points.

Example 5.8 (Silhouette Coefficient)**.** Figure 5.29 shows a plot of the silhouette coefficients for points in 10 clusters. Darker shades indicate lower silhouette coefficients. ■

5.5.3 Unsupervised Cluster Evaluation Using the Proximity Matrix

In this section, we examine a couple of unsupervised approaches for assessing cluster validity that are based on the proximity matrix. The first compares an actual and idealized proximity matrix, while the second uses visualization.

General Comments on Unsupervised Cluster Evaluation Measures

In addition to the measures presented above, a large number of other measures have been proposed as unsupervised cluster validity measures. Almost all these measures, including the measures presented above are intended for partitional, center-based clusters. In particular, none of them does well for continuity- or

density-based clusters. Indeed, a recent evaluation—see Bibliographic Notes—of a dozen such measures found that although a number of them did well in terms of handling issues such as noise and differing sizes and density, none of them except a relatively recently proposed measure, Clustering Validation index based on Nearest Neighbors (CVNN), did well on handling arbitrary shapes. The silhouette index, however, did well on all other issues examined except for that.

Most unsupervised cluster evaluation measures, such as the silhouette coefficient, incorporate both cohesion (compactness) and separation. When used with a partitional clustering algorithm such as K-means, these measures will tend to decrease until the "natural" set of clusters is found and start increasing once clusters are being split "too finely" since separation will suffer and cohesion will not improve much. Thus, these measures can provide a way to determine the number of clusters. However, if the definition of a cluster used by the clustering algorithm, differs from that of the cluster evaluation measure, then the set of clusters identified as optimal by the algorithm and validation measure can differ. For instance, CLUTO uses the measures described in Table 5.6 to drive its clustering, and thus, the clustering produced will not usually match the optimal clusters according to the silhouette coefficient. Likewise for standard K-means and SSE. In addition, if there actually are subclusters that are not separated very well from one another, then methods that incorporate both may provide only a coarse view of the cluster structure of the data. Another consideration is that when clustering for summarization, we are not interested in the "natural" cluster structure of the data, but rather want to achieve a certain level of approximation, e.g., want to reduce SSE to a certain level.

More generally, if there are not too many clusters, then it can be better to examine cluster cohesion and separation independently. This can give a more comprehensive view of how cohesive each cluster is and how well each pair of clusters is separated from one another. For instance, given a centroid based clustering, we could compute the pairwise similarity or distance of the centroids, i.e., compute the distance or similarity matrix of the centroids. The approach just outlined is similar to looking at the confusion matrix for a classification problem instead of classification measures, such as accuracy or the F-measure.

Measuring Cluster Validity via Correlation

If we are given the similarity matrix for a data set and the cluster labels from a cluster analysis of the data set, then we can evaluate the "goodness" of

the clustering by looking at the correlation between the similarity matrix and an ideal version of the similarity matrix based on the cluster labels. (With minor changes, the following applies to proximity matrices, but for simplicity, we discuss only similarity matrices.) More specifically, an ideal cluster is one whose points have a similarity of 1 to all points in the cluster, and a similarity of 0 to all points in other clusters. Thus, if we sort the rows and columns of the similarity matrix so that all objects belonging to the same cluster are together, then an ideal cluster similarity matrix has a **block diagonal** structure. In other words, the similarity is non-zero, i.e., 1, inside the blocks of the similarity matrix whose entries represent intra-cluster similarity, and 0 elsewhere. The ideal cluster similarity matrix is constructed by creating a matrix that has one row and one column for each data point—just like an actual similarity matrix—and assigning a 1 to an entry if the associated pair of points belongs to the same cluster. All other entries are 0.

High correlation between the ideal and actual similarity matrices indicates that the points that belong to the same cluster are close to each other, while low correlation indicates the opposite. (Because the actual and ideal similarity matrices are symmetric, the correlation is calculated only among the $n(n-1)/2$ entries below or above the diagonal of the matrices.) Consequently, this is not a good measure for many density- or contiguity-based clusters, because they are not globular and can be closely intertwined with other clusters.

Example 5.9 (Correlation of Actual and Ideal Similarity Matrices)**.** To illustrate this measure, we calculated the correlation between the ideal and actual similarity matrices for the K-means clusters shown in Figure 5.26(c) (random data) and Figure 5.30(a) (data with three well-separated clusters). The correlations were 0.5810 and 0.9235, respectively, which reflects the expected result that the clusters found by K-means in the random data are worse than the clusters found by K-means in data with well-separated clusters. ■

Judging a Clustering Visually by Its Similarity Matrix

The previous technique suggests a more general, qualitative approach to judging a set of clusters: Order the similarity matrix with respect to cluster labels and then plot it. In theory, if we have well-separated clusters, then the similarity matrix should be roughly block-diagonal. If not, then the patterns displayed in the similarity matrix can reveal the relationships between clusters. Again, all of this can be applied to dissimilarity matrices, but for simplicity, we will only discuss similarity matrices.

Example 5.10 (Visualizing a Similarity Matrix)**.** Consider the points in Figure 5.30(a), which form three well-separated clusters. If we use K-means to group these points into three clusters, then we should have no trouble finding these clusters because they are well-separated. The separation of these clusters is illustrated by the reordered similarity matrix shown in Figure 5.30(b). (For uniformity, we have transformed the distances into similarities using the formula $s = 1-(d-min_d)/(max_d-min_d)$.) Figure 5.31 shows the reordered similarity matrices for clusters found in the random data set of Figure 5.26 by DBSCAN, K-means, and complete link.

The well-separated clusters in Figure 5.30 show a very strong, block-diagonal pattern in the reordered similarity matrix. However, there are also weak block diagonal patterns—see Figure 5.31—in the reordered similarity matrices of the clusterings found by K-means, DBSCAN, and complete link in the random data. Just as people can find patterns in clouds, data mining algorithms can find clusters in random data. While it is entertaining to find patterns in clouds, it is pointless and perhaps embarrassing to find clusters in noise. ■

This approach may seem hopelessly expensive for large data sets, since the computation of the proximity matrix takes $O(m^2)$ time, where m is the number of objects, but with sampling, this method can still be used. We can take a sample of data points from each cluster, compute the similarity between these points, and plot the result. It is sometimes necessary to oversample small clusters and undersample large ones to obtain an adequate representation of all clusters.

5.5.4 Unsupervised Evaluation of Hierarchical Clustering

The previous approaches to cluster evaluation are intended for partitional clusterings. Here we discuss the cophenetic correlation, a popular evaluation measure for hierarchical clusterings. The **cophenetic distance** between two objects is the proximity at which an agglomerative hierarchical clustering technique puts the objects in the same cluster for the first time. For example, if at some point in the agglomerative hierarchical clustering process, the smallest distance between the two clusters that are merged is 0.1, then all points in one cluster have a cophenetic distance of 0.1 with respect to the points in the other cluster. In a cophenetic distance matrix, the entries are the cophenetic distances between each pair of objects. The cophenetic distance is different for each hierarchical clustering of a set of points.

Table 5.7. Cophenetic distance matrix for single link and data in Table 2.14 on page 110.

Point	P1	P2	P3	P4	P5	P6
P1	0	0.222	0.222	0.222	0.222	0.222
P2	0.222	0	0.148	0.151	0.139	0.148
P3	0.222	0.148	0	0.151	0.148	0.110
P4	0.222	0.151	0.151	0	0.151	0.151
P5	0.222	0.139	0.148	0.151	0	0.148
P6	0.222	0.148	0.110	0.151	0.148	0

fits the data. One of the most common uses of this measure is to evaluate which type of hierarchical clustering is best for a particular type of data.

Example 5.12 (Cophenetic Correlation Coefficient)**.** We calculated the CPCC for the hierarchical clusterings shown in Figures 5.16–5.19. These values are shown in Table 5.8. The hierarchical clustering produced by the single link technique seems to fit the data less well than the clusterings produced by complete link, group average, and Ward's method.

Table 5.8. Cophenetic correlation coefficient for data of Table 2.14 and four agglomerative hierarchical clustering techniques.

Technique	CPCC
Single Link	0.44
Complete Link	0.63
Group Average	0.66
Ward's	0.64

■

5.5.5 Determining the Correct Number of Clusters

Various unsupervised cluster evaluation measures can be used to approximately determine the correct or natural number of clusters.

Example 5.13 (Number of Clusters)**.** The data set of Figure 5.29 has 10 natural clusters. Figure 5.32 shows a plot of the SSE versus the number of clusters for a (bisecting) K-means clustering of the data set, while Figure 5.33 shows the average silhouette coefficient versus the number of clusters for the same data. There is a distinct knee in the SSE and a distinct peak in the silhouette coefficient when the number of clusters is equal to 10. ■

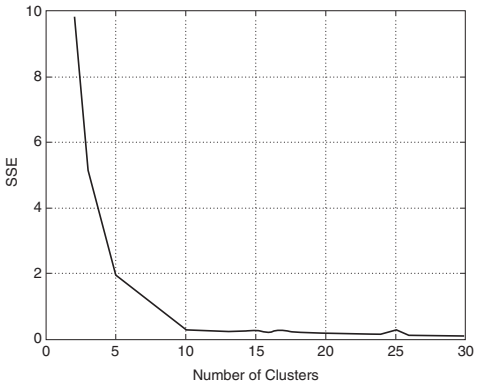


Figure 5.32. SSE versus number of clusters for the data of Figure 5.29 on page 364.

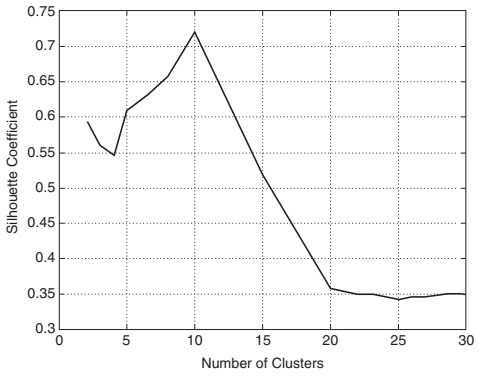


Figure 5.33. Average silhouette coefficient versus number of clusters for the data of Figure 5.29.

Thus, we can try to find the natural number of clusters in a data set by looking for the number of clusters at which there is a knee, peak, or dip in the plot of the evaluation measure when it is plotted against the number of clusters. Of course, such an approach does not always work well. Clusters can be considerably more intertwined or overlapping than those shown in Figure 5.29. Also, the data can consist of nested clusters. Actually, the clusters in Figure 5.29 are somewhat nested; i.e., there are five pairs of clusters since the clusters are closer top to bottom than they are left to right. There is a knee that indicates this in the SSE curve, but the silhouette coefficient curve is not as clear. In summary, while caution is needed, the technique we have just described can provide insight into the number of clusters in the data.

5.5.6 Clustering Tendency

One obvious way to determine if a data set has clusters is to try to cluster it. However, almost all clustering algorithms will dutifully find clusters when given data. To address this issue, we could evaluate the resulting clusters and only claim that a data set has clusters if at least some of the clusters are of good quality. However, this approach does not address the fact the clusters in the data can be of a different type than those sought by our clustering algorithm. To handle this additional problem, we could use multiple algorithms and again evaluate the quality of the resulting clusters. If the clusters are uniformly poor, then this may indeed indicate that there are no clusters in the data.

Alternatively, and this is the focus of measures of clustering tendency, we can try to evaluate whether a data set has clusters without clustering. The most common approach, especially for data in Euclidean space, has been to use statistical tests for spatial randomness. Unfortunately, choosing the correct model, estimating the parameters, and evaluating the statistical significance of the hypothesis that the data is non-random can be quite challenging. Nonetheless, many approaches have been developed, most of them for points in low-dimensional Euclidean space.

Example 5.14 (Hopkins Statistic)**.** For this approach, we generate p points that are randomly distributed across the data space and also sample p actual data points. For both sets of points, we find the distance to the nearest neighbor in the original data set. Let the u_i be the nearest neighbor distances of the artificially generated points, while the w_i are the nearest neighbor distances of the sample of points from the original data set. The Hopkins statistic H is then defined by Equation 5.17.

$$H = \frac{\sum_{i=1}^{p} w_i}{\sum_{i=1}^{p} u_i + \sum_{i=1}^{p} w_i} \tag{5.17}$$

If the randomly generated points and the sample of data points have roughly the same nearest neighbor distances, then H will be near 0.5. Values of H near 0 and 1 indicate, respectively, data that is highly clustered and data that is regularly distributed in the data space. To give an example, the Hopkins statistic for the data of Figure 5.26 was computed for $p = 20$ and 100 different trials. The average value of H was 0.56 with a standard deviation of 0.03. The same experiment was performed for the well-separated points of Figure 5.30. The average value of H was 0.95 with a standard deviation of 0.006. ∎

5.5.7 Supervised Measures of Cluster Validity

When we have external information about data, it is typically in the form of externally derived class labels for the data objects. In such cases, the usual procedure is to measure the degree of correspondence between the cluster labels and the class labels. But why is this of interest? After all, if we have the class labels, then what is the point in performing a cluster analysis? Motivations for such an analysis include the comparison of clustering techniques with the "ground truth" or the evaluation of the extent to which a manual classification process can be automatically produced by cluster analysis, e.g., the clustering of news articles. Another potential motivation

could be to evaluate whether objects in the same cluster tend to have the same label for semi-supervised learning techniques.

We consider two different kinds of approaches. The first set of techniques use measures from classification, such as entropy, purity, and the F-measure. These measures evaluate the extent to which a cluster contains objects of a single class. The second group of methods is related to the similarity measures for binary data, such as the Jaccard measure that we saw in Chapter 2. These approaches measure the extent to which two objects that are in the same class are in the same cluster and vice versa. For convenience, we will refer to these two types of measures as **classification-oriented** and **similarity-oriented**, respectively.

Classification-Oriented Measures of Cluster Validity

There are a number of measures that are commonly used to evaluate the performance of a classification model. In this section, we will discuss five: entropy, purity, precision, recall, and the F-measure. In the case of classification, we measure the degree to which predicted class labels correspond to actual class labels, but for the measures just mentioned, nothing fundamental is changed by using cluster labels instead of predicted class labels. Next, we quickly review the definitions of these measures in the context of clustering.

Entropy: The degree to which each cluster consists of objects of a single class. For each cluster, the class distribution of the data is calculated first, i.e., for cluster i we compute p_{ij}, the probability that a member of cluster i belongs to class j as $p_{ij} = m_{ij}/m_i$, where m_i is the number of objects in cluster i and m_{ij} is the number of objects of class j in cluster i. Using this class distribution, the entropy of each cluster i is calculated using the standard formula, $e_i = -\sum_{j=1}^{L} p_{ij} \log_2 p_{ij}$, where L is the number of classes. The total entropy for a set of clusters is calculated as the sum of the entropies of each cluster weighted by the size of each cluster, i.e., $e = \sum_{i=1}^{K} \frac{m_i}{m} e_i$, where K is the number of clusters and m is the total number of data points.

Purity: Another measure of the extent to which a cluster contains objects of a single class. Using the previous terminology, the purity of cluster i is $purity(i) = \max_j p_{ij}$, the overall purity of a clustering is $purity = \sum_{i=1}^{K} \frac{m_i}{m} purity(i)$.

Table 5.9. K-means clustering results for the *LA Times* document data set.

Cluster	Entertainment	Financial	Foreign	Metro	National	Sports	Entropy	Purity
1	3	5	40	506	96	27	1.2270	0.7474
2	4	7	280	29	39	2	1.1472	0.7756
3	1	1	1	7	4	671	0.1813	0.9796
4	10	162	3	119	73	2	1.7487	0.4390
5	331	22	5	70	13	23	1.3976	0.7134
6	5	358	12	212	48	13	1.5523	0.5525
Total	354	555	341	943	273	738	1.1450	0.7203

Precision: The fraction of a cluster that consists of objects of a specified class. The precision of cluster i with respect to class j is $precision(i, j) = p_{ij}$.

Recall: The extent to which a cluster contains all objects of a specified class. The recall of cluster i with respect to class j is $recall(i, j) = m_{ij}/m_j$, where m_j is the number of objects in class j.

F-measure A combination of both precision and recall that measures the extent to which a cluster contains *only* objects of a particular class and *all* objects of that class. The F-measure of cluster i with respect to class j is $F(i, j) = (2 \times precision(i, j) \times recall(i, j))/(precision(i, j) + recall(i, j))$. The F-measure of a set of clusters, partitional or hierarchical is presented on page 376 when we discuss cluster validity for hierarchical clusterings.

Example 5.15 (Supervised Evaluation Measures)**.** We present an example to illustrate these measures. Specifically, we use K-means with the cosine similarity measure to cluster 3204 newspaper articles from the *Los Angeles Times*. These articles come from six different classes: Entertainment, Financial, Foreign, Metro, National, and Sports. Table 5.9 shows the results of a K-means clustering to find six clusters. The first column indicates the cluster, while the next six columns together form the confusion matrix; i.e., these columns indicate how the documents of each category are distributed among the clusters. The last two columns are the entropy and purity of each cluster, respectively.

Ideally, each cluster will contain documents from only one class. In reality, each cluster contains documents from many classes. Nevertheless, many clusters contain documents primarily from just one class. In particular, cluster 3, which contains mostly documents from the Sports section, is exceptionally good, both in terms of purity and entropy. The purity and entropy of the other

clusters is not as good, but can typically be greatly improved if the data is partitioned into a larger number of clusters.

Precision, recall, and the F-measure can be calculated for each cluster. To give a concrete example, we consider cluster 1 and the Metro class of Table 5.9. The precision is $506/677 = 0.75$, recall is $506/943 = 0.26$, and hence, the F value is 0.39. In contrast, the F value for cluster 3 and Sports is 0.94. As in classification, the confusion matrix gives the most detailed information. ■

Similarity-Oriented Measures of Cluster Validity

The measures that we discuss in this section are all based on the premise that any two objects that are in the same cluster should be in the same class and vice versa. We can view this approach to cluster validity as involving the comparison of two matrices: (1) the **ideal cluster similarity matrix** discussed previously, which has a 1 in the ij^{th} entry if two objects, i and j, are in the same cluster and 0, otherwise, and (2) a **class similarity matrix** defined with respect to class labels, which has a 1 in the ij^{th} entry if two objects, i and j, belong to the same class, and a 0 otherwise. As before, we can take the correlation of these two matrices as the measure of cluster validity. This measure is known as Hubert's Γ statistic in the clustering validation literature.

Example 5.16 (Correlation between Cluster and Class Matrices)**.** To demonstrate this idea more concretely, we give an example involving five data points, $p_1, p_2, p_3, p_4, andp_5$, two clusters, $C_1 = \{p_1, p_2, p_3\}$ and $C_2 = \{p_4, p_5\}$, and two classes, $L_1 = \{p_1, p_2\}$ and $L2 = \{p_3, p_4, p_5\}$. The ideal cluster and class similarity matrices are given in Tables 5.10 and 5.11. The correlation between the entries of these two matrices is 0.359.

Table 5.10. Ideal cluster similarity matrix.

Point	p1	p2	p3	p4	p5
p1	1	1	1	0	0
p2	1	1	1	0	0
p3	1	1	1	0	0
p4	0	0	0	1	1
p5	0	0	0	1	1

Table 5.11. Class similarity matrix.

Point	p1	p2	p3	p4	p5
p1	1	1	0	0	0
p2	1	1	0	0	0
p3	0	0	1	1	1
p4	0	0	1	1	1
p5	0	0	1	1	1

■

More generally, we can use any of the measures for binary similarity that we saw in Section 2.4.5. (For example, we can convert these two matrices into binary vectors by appending the rows.) We repeat the definitions of the four quantities used to define those similarity measures, but modify our descriptive text to fit the current context. Specifically, we need to compute the following four quantities for all pairs of distinct objects. (There are $m(m-1)/2$ such pairs, if m is the number of objects.)

f_{00} = number of pairs of objects having a different class and a different cluster
f_{01} = number of pairs of objects having a different class and the same cluster
f_{10} = number of pairs of objects having the same class and a different cluster
f_{11} = number of pairs of objects having the same class and the same cluster

In particular, the simple matching coefficient, which is known as the Rand statistic in this context, and the Jaccard coefficient are two of the most frequently used cluster validity measures.

$$\text{Rand statistic} = \frac{f_{00} + f_{11}}{f_{00} + f_{01} + f_{10} + f_{11}} \tag{5.18}$$

$$\text{Jaccard coefficient} = \frac{f_{11}}{f_{01} + f_{10} + f_{11}} \tag{5.19}$$

Example 5.17 (Rand and Jaccard Measures)**.** Based on these formulas, we can readily compute the Rand statistic and Jaccard coefficient for the example based on Tables 5.10 and 5.11. Noting that $f_{00} = 4$, $f_{01} = 2$, $f_{10} = 2$, and $f_{11} = 2$, the Rand statistic $= (2 + 4)/10 = 0.6$ and the Jaccard coefficient $= 2/(2+2+2\ ) = 0.33$. ■

We also note that the four quantities, f_{00}, f_{01}, f_{10}, and f_{11}, define a *contingency* table as shown in Table 5.12.

Table 5.12. Two-way contingency table for determining whether pairs of objects are in the same class and same cluster.

	Same Cluster	Different Cluster
Same Class	f_{11}	f_{10}
Different Class	f_{01}	f_{00}

Previously, in the context of association analysis—see Section 4.7.1 on page 258—we presented an extensive discussion of measures of association that can

be used for this type of contingency table. (Compare Table 5.12 on page 375 with Table 4.6 on page 258.) Those measures can also be applied to cluster validity.

Cluster Validity for Hierarchical Clusterings

So far in this section, we have discussed supervised measures of cluster validity only for partitional clusterings. Supervised evaluation of a hierarchical clustering is more difficult for a variety of reasons, including the fact that a preexisting hierarchical structure often does not exist. In addition, although relatively pure clusters often exist at certain levels in the hierarchical clustering, as the clustering proceeds, the clusters will become impure. The key idea of the approach presented here, which is based on the F-measure, is to evaluate whether a hierarchical clustering contains, for each class, at least one cluster that is relatively pure and includes most of the objects of that class. To evaluate a hierarchical clustering with respect to this goal, we compute, for each class, the F-measure for each cluster in the cluster hierarchy, and then take the maximum F-measure attained for any cluster. Finally, we calculate an overall F-measure for the hierarchical clustering by computing the weighted average of all per-class F-measures, where the weights are based on the class sizes. More formally, this hierarchical F-measure is defined as follows:

$$F = \sum_j \frac{m_j}{m} \max_i F(i, j)$$

where the maximum is taken over all clusters i at all levels, m_j is the number of objects in class j, and m is the total number of objects. Note that this measure can also be applied for a partitional clustering without modification.

5.5.8 Assessing the Significance of Cluster Validity Measures

Cluster validity measures are intended to help us measure the goodness of the clusters that we have obtained. Indeed, they typically give us a single number as a measure of that goodness. However, we are then faced with the problem of interpreting the significance of this number, a task that might be even more difficult.

The minimum and maximum values of cluster evaluation measures can provide some guidance in many cases. For instance, by definition, a purity of 0 is bad, while a purity of 1 is good, at least if we trust our class labels and

want our cluster structure to reflect the class structure. Likewise, an entropy of 0 is good, as is an SSE of 0.

Sometimes, however, there is no minimum or maximum value, or the scale of the data might affect the interpretation. Also, even if there are minimum and maximum values with obvious interpretations, intermediate values still need to be interpreted. In some cases, we can use an absolute standard. If, for example, we are clustering for utility, we are often willing to tolerate only a certain level of error in the approximation of our points by a cluster centroid.

But if this is not the case, then we must do something else. A common approach is to interpret the value of our validity measure in statistical terms. Specifically, we attempt to judge how likely it is that our observed value was achieved by random chance. The value is good if it is unusual; i.e., if it is unlikely to be the result of random chance. The motivation for this approach is that we are interested only in clusters that reflect non-random structure in the data, and such structures should generate unusually high (low) values of our cluster validity measure, at least if the validity measures are designed to reflect the presence of strong cluster structure.

Example 5.18 (Significance of SSE)**.** To show how this works, we present an example based on K-means and the SSE. Suppose that we want a measure of how good the well-separated clusters of Figure 5.30 are with respect to random data. We generate many random sets of 100 points having the same range as the points in the three clusters, find three clusters in each data set using K-means, and accumulate the distribution of SSE values for these clusterings. By using this distribution of the SSE values, we can then estimate the probability of the SSE value for the original clusters. Figure 5.34 shows the histogram of the SSE from 500 random runs. The lowest SSE shown in Figure 5.34 is 0.0173. For the three clusters of Figure 5.30, the SSE is 0.0050. We could therefore conservatively claim that there is less than a 1% chance that a clustering such as that of Figure 5.30 could occur by chance. ∎

In the previous example, randomization was used to evaluate the statistical significance of a cluster validity measure. However, for some measures, such as Hubert's Γ statistic, the distribution is known and can be used to evaluate the measure. In addition, a normalized version of the measure can be computed by subtracting the mean and dividing by the standard deviation. See Bibliographic Notes for more details.

We stress that there is more to cluster evaluation (unsupervised or supervised) than obtaining a numerical measure of cluster validity. Unless this value has a natural interpretation based on the definition of the measure, we need to interpret this value in some way. If our cluster evaluation measure

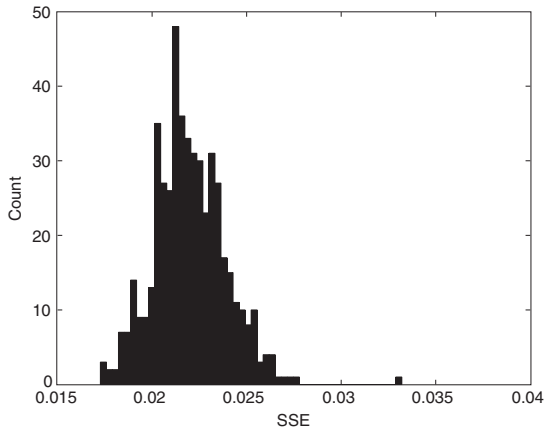


Figure 5.34. Histogram of SSE for 500 random data sets.

is defined such that lower (higher) values indicate stronger clusters, then we can use statistics to evaluate whether the value we have obtained is unusually low (high), provided we have a distribution for the evaluation measure. We have presented an example of how to find such a distribution, but there is considerably more to this topic, and we refer the reader to the Bibliographic Notes for more pointers.

Finally, even when an evaluation measure is used as a relative measure, i.e., to compare two clusterings, we still need to assess the significance in the difference between the evaluation measures of the two clusterings. Although one value will almost always be better than another, it can be difficult to determine if the difference is significant. Note that there are two aspects to this significance: whether the difference is statistically significant (repeatable) and whether the magnitude of the difference is meaningful with respect to the application. Many would not regard a difference of 0.1% as significant, even if it is consistently reproducible.

5.5.9 Choosing a Cluster Validity Measure

There are many more measures for evaluating cluster validity than have been discussed here. Various books and articles propose various measures as being superior to others. In this section, we offer some high-level guidance. First, it is important to distinguish whether the clustering is being performed for summarization or understanding. If summarization, typically class labels are not involved and the goal is maximum compression. This is often done by finding clusters that minimize the distance of objects to their closest cluster

centroid. Indeed, the clustering process is often driven by the objective of minimizing representation error. Measures such as SSE are more natural for this application.

If the clustering is being performed for understanding, then the situation is more complicated. For the unsupervised case, virtually all measures try to maximize cohesion and separation. Some measures obtain a "best" value for a particular value of K, the number of clusters. Although this might seem appealing, such measures typically only identify one "right" number of clusters, even when subclusters are present. (Recall that cohesion and separation continue to increase for K-means until there is one cluster per point.) More generally, if the number of clusters is not too large, it can be useful to manually examine the cluster cohesion of each cluster and the pairwise separation of clusters. However, note that very few cluster validity measures are appropriate to contiguity or density-based clusters that can have irregular and intertwined shapes.

For the supervised case, where clustering is almost always performed with a goal of generating understandable clusters, the ideal result of clustering is to produce clusters that match the underlying class structure. Evaluating the match between a set of clusters and classes is a non-trivial problem. The F-measure and hierarchical F-measure discussed earlier, are examples of how to evaluate such a match. Other examples can be found in the references to cluster evaluation provided in the Bibliographic Notes. In any case, when the number of clusters are relatively small, the confusion matrix can be more informative than any single measure of cluster validity since it an indicate which classes tend to be appear in clusters with which other classes. Note that supervised cluster evaluation indices are independent of whether the clusters are center-, contiguity-, or density-based.

In conclusion, it is important to realize that clustering is often used as an exploratory data technique whose goal is often not to provide a crisp answer, but rather to provide some insight into the underlying structure of the data. In this situation, cluster validity indices are useful to the extent they are useful to that end goal.

5.6 Bibliographic Notes

Discussion in this chapter has been most heavily influenced by the books on cluster analysis written by Jain and Dubes [370], Anderberg [343], and Kaufman and Rousseeuw [374], as well as the more recent book edited by and Aggarwal and Reddy [341]. Additional clustering books that may also

6

Classification: Alternative Techniques

Chapter 3 introduced the classification problem and presented a technique known as the decision tree classifier. Issues such as model overfitting and model evaluation were also discussed. This chapter presents alternative techniques for building classification models—from simple techniques such as rule-based and nearest neighbor classifiers to more sophisticated techniques such as artificial neural networks, deep learning, support vector machines, and ensemble methods. Other practical issues such as the class imbalance and multiclass problems are also discussed at the end of the chapter.

6.1 Types of Classifiers

Before presenting specific techniques, we first categorize the different types of classifiers available. One way to distinguish classifiers is by considering the characteristics of their output.

Binary versus Multiclass Binary classifiers assign each data instance to one of two possible labels, typically denoted as $+1$ and -1. The positive class usually refers to the category we are more interested in predicting correctly compared to the negative class (e.g., the `spam` category in email classification problems). If there are more than two possible labels available, then the technique is known as a multiclass classifier. As some classifiers were designed for binary classes only, they must be adapted to deal with multiclass problems. Techniques for transforming binary classifiers into multiclass classifiers are described in Section 6.12.

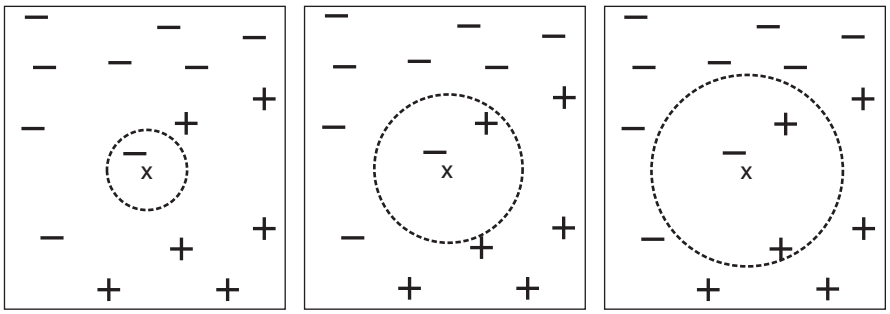
6.3 Nearest Neighbor Classifiers

The classification framework shown in Figure 3.3 involves a two-step process: (1) an inductive step for constructing a classification model from data, and (2) a deductive step for applying the model to test examples. Decision tree and rule-based classifiers are examples of **eager learners** because they are designed to learn a model that maps the input attributes to the class label as soon as the training data becomes available. An opposite strategy would be to delay the process of modeling the training data until it is needed to classify the test instances. Techniques that employ this strategy are known as **lazy learners**. An example of a lazy learner is the **Rote classifier**, which memorizes the entire training data and performs classification only if the attributes of a test instance match one of the training examples exactly. An obvious drawback of this approach is that some test instances may not be classified because they do not match any training example.

One way to make this approach more flexible is to find all the training examples that are relatively similar to the attributes of the test instances. These examples, which are known as **nearest neighbors**, can be used to determine the class label of the test instance. The justification for using nearest neighbors is best exemplified by the following saying: *"If it walks like a duck, quacks like a duck, and looks like a duck, then it's probably a duck."* A nearest neighbor classifier represents each example as a data point in a d-dimensional space, where d is the number of attributes. Given a test instance, we compute its proximity to the training instances according to one of the proximity measures described in Section 2.4 on page 91. The k-nearest neighbors of a given test instance z refer to the k training examples that are closest to z.

Figure 6.6 illustrates the 1-, 2-, and 3-nearest neighbors of a test instance located at the center of each circle. The instance is classified based on the class labels of its neighbors. In the case where the neighbors have more than one label, the test instance is assigned to the majority class of its nearest neighbors. In Figure 6.6(a), the 1-nearest neighbor of the instance is a negative example. Therefore the instance is assigned to the negative class. If the number of nearest neighbors is three, as shown in Figure 6.6(c), then the neighborhood contains two positive examples and one negative example. Using the majority voting scheme, the instance is assigned to the positive class. In the case where there is a tie between the classes (see Figure 6.6(b)), we may randomly choose one of them to classify the data point.

The preceding discussion underscores the importance of choosing the right value for k. If k is too small, then the nearest neighbor classifier may be



(a) 1-nearest neighbor (b) 2-nearest neighbor (c) 3-nearest neighbor

Figure 6.6. The 1-, 2-, and 3-nearest neighbors of an instance.

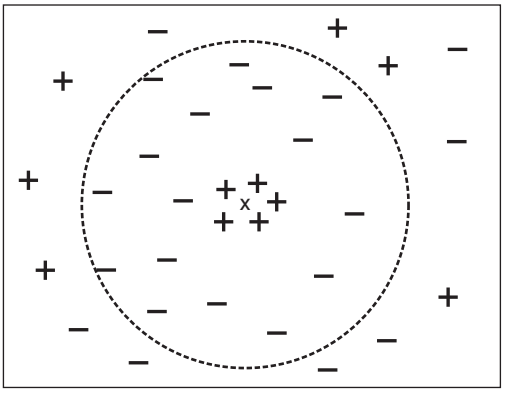


Figure 6.7. k-nearest neighbor classification with large k.

susceptible to overfitting due to noise, i.e., mislabeled examples in the training data. On the other hand, if k is too large, the nearest neighbor classifier may misclassify the test instance because its list of nearest neighbors includes training examples that are located far away from its neighborhood (see Figure 6.7).

6.3.1 Algorithm

A high-level summary of the nearest neighbor classification method is given in Algorithm 6.2. The algorithm computes the distance (or similarity) between each test instance $z = (\mathbf{x}', y')$ and all the training examples $(\mathbf{x}, y) \in D$ to determine its nearest neighbor list, D_z. Such computation can be costly if the number of training examples is large. However, efficient indexing techniques

Algorithm 6.2 The k-nearest neighbor classifier.

1: Let k be the number of nearest neighbors and D be the set of training examples.
2: **for** each test instance $z = (\mathbf{x}', y')$ **do**
3: Compute $d(\mathbf{x}', \mathbf{x})$, the distance between z and every example, $(\mathbf{x}, y) \in D$.
4: Select $D_z \subseteq D$, the set of k closest training examples to z.
5: $y' = \underset{v}{\text{argmax}} \sum_{(\mathbf{x}_i, y_i) \in D_z} I(v = y_i)$
6: **end for**

are available to reduce the computation needed to find the nearest neighbors of a test instance.

Once the nearest neighbor list is obtained, the test instance is classified based on the majority class of its nearest neighbors:

$$\text{Majority Voting: } y' = \underset{v}{\text{argmax}} \sum_{(\mathbf{x}_i, y_i) \in D_z} I(v = y_i), \tag{6.5}$$

where v is a class label, y_i is the class label for one of the nearest neighbors, and $I(\cdot)$ is an indicator function that returns the value 1 if its argument is true and 0 otherwise.

In the majority voting approach, every neighbor has the same impact on the classification. This makes the algorithm sensitive to the choice of k, as shown in Figure 6.6. One way to reduce the impact of k is to weight the influence of each nearest neighbor $\mathbf{x}_i$ according to its distance: $w_i = 1/d(\mathbf{x}', \mathbf{x}_i)^2$. As a result, training examples that are located far away from z have a weaker impact on the classification compared to those that are located close to z. Using the distance-weighted voting scheme, the class label can be determined as follows:

$$\text{Distance-Weighted Voting: } y' = \underset{v}{\text{argmax}} \sum_{(\mathbf{x}_i, y_i) \in D_z} w_i \times I(v = y_i). \tag{6.6}$$

6.3.2 Characteristics of Nearest Neighbor Classifiers

1. Nearest neighbor classification is part of a more general technique known as instance-based learning, which does not build a global model, but rather uses the training examples to make predictions for a test instance. (Thus, such classifiers are often said to be "model free.") Such algorithms require a proximity measure to determine the similarity or distance between instances and a classification function that returns the predicted class of a test instance based on its proximity to other instances.

2. Although lazy learners, such as nearest neighbor classifiers, do not require model building, classifying a test instance can be quite expensive because we need to compute the proximity values individually between the test and training examples. In contrast, eager learners often spend the bulk of their computing resources for model building. Once a model has been built, classifying a test instance is extremely fast.

3. Nearest neighbor classifiers make their predictions based on local information. (This is equivalent to building a local model for each test instance.) By contrast, decision tree and rule-based classifiers attempt to find a global model that fits the entire input space. Because the classification decisions are made locally, nearest neighbor classifiers (with small values of k) are quite susceptible to noise.

4. Nearest neighbor classifiers can produce decision boundaries of arbitrary shape. Such boundaries provide a more flexible model representation compared to decision tree and rule-based classifiers that are often constrained to rectilinear decision boundaries. The decision boundaries of nearest neighbor classifiers also have high variability because they depend on the composition of training examples in the local neighborhood. Increasing the number of nearest neighbors may reduce such variability.

5. Nearest neighbor classifiers have difficulty handling missing values in both the training and test sets since proximity computations normally require the presence of all attributes. Although, the subset of attributes present in two instances can be used to compute a proximity, such an approach may not produce good results since the proximity measures may be different for each pair of instances and thus hard to compare.

6. Nearest neighbor classifiers can handle the presence of interacting attributes, i.e., attributes that have more predictive power taken in combination then by themselves, by using appropriate proximity measures that can incorporate the effects of multiple attributes together.

7. The presence of irrelevant attributes can distort commonly used proximity measures, especially when the number of irrelevant attributes is large. Furthermore, if there are a large number of redundant attributes that are highly correlated with each other, then the proximity measure can be overly biased toward such attributes, resulting in improper estimates of distance. Hence, the presence of irrelevant and redundant attributes can adversely affect the performance of nearest neighbor classifiers.

8. Nearest neighbor classifiers can produce wrong predictions unless the appropriate proximity measure and data preprocessing steps are taken. For example, suppose we want to classify a group of people based on attributes such as height (measured in meters) and weight (measured in pounds). The height attribute has a low variability, ranging from 1.5 m to 1.85 m, whereas the weight attribute may vary from 90 lb. to 250 lb. If the scale of the attributes are not taken into consideration, the proximity measure may be dominated by differences in the weights of a person.

6.4 Naïve Bayes Classifier

Many classification problems involve uncertainty. First, the observed attributes and class labels may be unreliable due to imperfections in the measurement process, e.g., due to the limited preciseness of sensor devices. Second, the set of attributes chosen for classification may not be fully representative of the target class, resulting in uncertain predictions. To illustrate this, consider the problem of predicting a person's risk for heart disease based on a model that uses their diet and workout frequency as attributes. Although most people who eat healthily and exercise regularly have less chance of developing heart disease, they may still be at risk due to other *latent factors*, such as heredity, excessive smoking, and alcohol abuse, that are not captured in the model. Third, a classification model learned over a finite training set may not be able to fully capture the true relationships in the overall data, as discussed in the context of model overfitting in Chapter 3. Finally, uncertainty in predictions may arise due to the inherent *random nature* of real-world systems, such as those encountered in weather forecasting problems.

In the presence of uncertainty, there is a need to not only make predictions of class labels but also provide a measure of confidence associated with every prediction. Probability theory offers a systematic way for quantifying and manipulating uncertainty in data, and thus, is an appealing framework for assessing the confidence of predictions. Classification models that make use of probability theory to represent the relationship between attributes and class labels are known as **probabilistic classification models**. In this section, we present the naïve Bayes classifier, which is one of the simplest and most widely-used probabilistic classification models.

6.4.1 Basics of Probability Theory

Before we discuss how the naïve Bayes classifier works, we first introduce some basics of probability theory that will be useful in understanding the probabilistic classification models presented in this chapter. This involves defining the notion of probability and introducing some common approaches for manipulating probability values.

Consider a variable X, which can take any discrete value from the set $\{x_1, \ldots, x_k\}$. When we have multiple observations of that variable, such as in a data set where the variable describes some characteristic of data objects, then we can compute the relative frequency with which each value occurs. Specifically, suppose that X has the value x_i for n_i data objects. The relative frequency with which we observe the event $X = x_i$ is then n_i/N, where N denotes the total number of occurrences ($N = \sum_{i=1}^{k} n_i$). These relative frequencies characterize the uncertainty that we have with respect to what value X may take for an unseen observation and motivates the notion of probability.

More formally, the **probability** of an event e, e.g., $P(X = x_i)$, measures how likely it is for the event e to occur. The most traditional view of probability is based on relative frequency of events (frequentist), while the Bayesian viewpoint (described later) takes a more flexible view of probabilities. In either case, a probability is always a number between 0 and 1. Further, the sum of probability values of all possible events, e.g., outcomes of a variable X is equal to 1. Variables that have probabilities associated with each possible outcome (values) are known as **random variables**.

Now, let us consider two random variables, X and Y, that can each take k discrete values. Let n_{ij} be the number of times we observe $X = x_i$ and $Y = y_j$, out of a total number of N occurrences. The **joint probability** of observing $X = x_i$ and $Y = y_j$ together can be estimated as

$$P(X = x_i, Y = y_i) = \frac{n_{ij}}{N}. \tag{6.7}$$

(This is an estimate since we typically have only a finite subset of all possible observations.) Joint probabilities can be used to answer questions such as "what is the probability that there will be a surprise quiz today `AND` I will be late for the class." Joint probabilities are symmetric, i.e., $P(X = x, Y = y) = P(Y = y, X = x)$. For joint probabilities, it is to useful to consider their sum with respect to one of the random variables, as described in the following equation:

$$\sum_{j=1}^{k} P(X = x_i, Y = y_j) = \frac{\sum_{j=1}^{k} n_{ij}}{N} = \frac{n_i}{N} = P(X = x_i), \tag{6.8}$$

where n_i is the total number of times we observe $X = x_i$ irrespective of the value of Y. Notice that n_i/N is essentially the probability of observing $X = x_i$. Hence, by summing out the joint probabilities with respect to a random variable Y, we obtain the probability of observing the remaining variable X. This operation is called **marginalization** and the probability value $P(X = x_i)$ obtained by marginalizing out Y is sometimes called the **marginal probability** of X. As we will see later, joint probability and marginal probability form the basic building blocks of a number of probabilistic classification models discussed in this chapter.

Notice that in the previous discussions, we used $P(X = x_i)$ to denote the probability of a particular outcome of a random variable X. This notation can easily become cumbersome when a number of random variables are involved. Hence, in the remainder of this section, we will use $P(X)$ to denote the probability of any generic outcome of the random variable X, while $P(x_i)$ will be used to represent the probability of the specific outcome x_i.

Bayes Theorem

> *Suppose you have invited two of your friends Alex and Martha to a dinner party. You know that Alex attends* 40% *of the parties he is invited to. Further, if Alex is going to a party, there is an* 80% *chance of Martha coming along. On the other hand, if Alex is not going to the party, the chance of Martha coming to the party is reduced to* 30%*. If Martha has responded that she will be coming to your party, what is the probability that Alex will also be coming?*

Bayes theorem presents the statistical principle for answering questions like the previous one, where evidence from multiple sources has to be combined with prior beliefs to arrive at predictions. Bayes theorem can be briefly described as follows.

Let $P(Y|X)$ denote the **conditional probability** of observing the random variable Y whenever the random variable X takes a particular value. $P(Y|X)$ is often read as the probability of observing Y *conditioned* on the outcome of X. Conditional probabilities can be used for answering questions such as "given that it is going to rain today, what will be the probability that I will go to the class." Conditional probabilities of X and Y are related to their joint

probability in the following way:

$$P(Y|X) = \frac{P(X,Y)}{P(X)}, \quad \text{which implies} \tag{6.9}$$

$$\begin{aligned} P(X,Y) &= P(Y|X) \times P(X) \\ &= P(X|Y) \times P(Y). \end{aligned} \tag{6.10}$$

Rearranging the last two expressions in Equation 6.10 leads to Equation 6.11, which is known as **Bayes theorem**:

$$P(Y|X) = \frac{P(X|Y)P(Y)}{P(X)}. \tag{6.11}$$

Bayes theorem provides a relationship between the conditional probabilities $P(Y|X)$ and $P(X|Y)$. Note that the denominator in Equation 6.11 involves the marginal probability of X, which can also be represented as

$$P(X) = \sum_{i=1}^{k} P(X, y_i) = \sum_{i=1}^{k} P(X|y_i) \times P(y_i).$$

Using the previous expression for $P(X)$, we can obtain the following equation for $P(Y|X)$ solely in terms of $P(X|Y)$ and $P(Y)$:

$$P(Y|X) = \frac{P(X|Y)P(Y)}{\sum_{i=1}^{k} P(X|y_i)P(y_i)}. \tag{6.12}$$

Example 6.4. [Bayes Theorem] Bayes theorem can be used to solve a number of inferential questions about random variables. For example, consider the problem stated at the beginning on inferring whether Alex will come to the party. Let $P(A = 1)$ denote the probability of Alex going to a party, while $P(A = 0)$ denotes the probability of him not going to a party. We know that

$$\begin{aligned} & P(A = 1) = 0.4, \\ \text{and} \quad & P(A = 0) = 1 - P(A = 1) = 0.6. \end{aligned}$$

Further, let $P(M = 1|A)$ denote the conditional probability of Martha going to a party conditioned on whether Alex is going to the party. $P(M = 1|A)$ takes the following values:

$$\begin{aligned} & P(M = 1|A = 1) = 0.8, \\ \text{and} \quad & P(M = 1|A = 0) = 0.3. \end{aligned}$$

We can use the above values of $P(M|A)$ and $P(A)$ to compute the probability of Alex going to the party given Martha is going to the party, $P(A = 1|M = 1)$, as follows:

$$\begin{aligned} P(A=1|M=1) &= \frac{P(M=1|A=1)P(A=1)}{P(M=1|A=0)P(A=0)+P(M=1|A=1)P(A=1)}, \\ &= \frac{0.8 \times 0.4}{0.3 \times 0.6 + 0.8 \times 0.4} \\ &= \frac{0.32}{0.5} = 0.64. \end{aligned} \tag{6.13}$$

Notice that even though the prior probability $P(A)$ of Alex going to the party is low, the observation that Martha is going, $M = 1$, affects the conditional probability $P(A = 1|M = 1)$. This shows the value of Bayes theorem in combining prior assumptions with observed outcomes to make predictions. Since $P(A = 1|M = 1) > 0.5$, it is more likely for Alex to join if Martha is going to the party.

■

Using Bayes Theorem for Classification

For the purpose of classification, we are interested in computing the probability of observing a class label y for a data instance given its set of attribute values $\mathbf{x}$. This can be represented as $P(y|\mathbf{x})$, which is known as the **posterior probability** of the target class. Using the Bayes Theorem, we can represent the posterior probability as

$$P(y|\mathbf{x}) = \frac{P(\mathbf{x}|y)P(y)}{P(\mathbf{x})} \tag{6.14}$$

Note that the numerator of the previous equation involves two terms, $P(\mathbf{x}|y)$ and $P(y)$, both of which contribute to the posterior probability $P(y|\mathbf{x})$. We describe both of these terms in the following.

The first term $P(\mathbf{x}|y)$ is known as the **class-conditional** probability of the attributes given the class label. $P(\mathbf{x}|y)$ measures the likelihood of observing $\mathbf{x}$ from the distribution of instances belonging to y. If $\mathbf{x}$ indeed belongs to class y, then we should expect $P(\mathbf{x}|y)$ to be high. From this point of view, the use of class-conditional probabilities attempts to capture the process from which the data instances were generated. Because of this interpretation, probabilistic

classification models that involve computing class-conditional probabilities are known as **generative classification models**. Apart from their use in computing posterior probabilities and making predictions, class-conditional probabilities also provide insights about the underlying mechanism behind the generation of attribute values.

The second term in the numerator of Equation 6.14 is the **prior probability** $P(y)$. The prior probability captures our prior beliefs about the distribution of class labels, independent of the observed attribute values. (This is the Bayesian viewpoint.) For example, we may have a prior belief that the likelihood of any person to suffer from a heart disease is α, irrespective of their diagnosis reports. The prior probability can either be obtained using expert knowledge, or inferred from historical distribution of class labels.

The denominator in Equation 6.14 involves the probability of evidence, $P(\mathbf{x})$. Note that this term does not depend on the class label and thus can be treated as a normalization constant in the computation of posterior probabilities. Further, the value of $P(\mathbf{x})$ can be calculated as $P(\mathbf{x}) = \sum_i P(\mathbf{x}|y_i)P(y_i)$.

Bayes theorem provides a convenient way to combine our prior beliefs with the likelihood of obtaining the observed attribute values. During the training phase, we are required to learn the parameters for $P(y)$ and $P(\mathbf{x}|y)$. The prior probability $P(y)$ can be easily estimated from the training set by computing the fraction of training instances that belong to each class. To compute the class-conditional probabilities, one approach is to consider the fraction of training instances of a given class for every possible combination of attribute values. For example, suppose that there are two attributes X_1 and X_2 that can each take a discrete value from c_1 to c_k. Let n^0 denote the number of training instances belonging to class 0, out of which n^0_{ij} number of training instances have $X_1 = c_i$ and $X_2 = c_j$. The class-conditional probability can then be given as

$$P(X_1 = c_i, X_2 = c_j|Y = 0) = \frac{n^0_{ij}}{n^0}.$$

This approach can easily become computationally prohibitive as the number of attributes increase, due to the exponential growth in the number of attribute value combinations. For example, if every attribute can take k discrete values, then the number of attribute value combinations is equal to k^d, where d is the number of attributes. The large number of attribute value combinations can also result in poor estimates of class-conditional probabilities, since every combination will have fewer training instances when the size of training set is small.

In the following, we present the naïve Bayes classifier, which makes a simplifying assumption about the class-conditional probabilities, known as the

naïve Bayes assumption. The use of this assumption significantly helps in obtaining reliable estimates of class-conditional probabilities, even when the number of attributes are large.

6.4.2 Naïve Bayes Assumption

The naïve Bayes classifier assumes that the class-conditional probability of all attributes $\mathbf{x}$ can be factored as a product of class-conditional probabilities of every attribute x_i, as described in the following equation:

$$P(\mathbf{x}|y) = \prod_{i=1}^{d} P(x_i|y), \tag{6.15}$$

where every data instance $\mathbf{x}$ consists of d attributes, $\{x_1, x_2, \ldots, x_d\}$. The basic assumption behind the previous equation is that the attribute values x_i are **conditionally independent** of each other, given the class label y. This means that the attributes are influenced only by the target class and if we know the class label, then we can consider the attributes to be independent of each other. The concept of conditional independence can be formally stated as follows.

Conditional Independence

Let $\mathbf{X_1}$, $\mathbf{X_2}$, and $\mathbf{Y}$ denote three sets of random variables. The variables in $\mathbf{X_1}$ are said to be conditionally independent of $\mathbf{X_2}$, given $\mathbf{Y}$, if the following condition holds:

$$P(\mathbf{X_1}|\mathbf{X_2}, \mathbf{Y}) = P(\mathbf{X_1}|\mathbf{Y}). \tag{6.16}$$

This means that conditioned on $\mathbf{Y}$, the distribution of $\mathbf{X_1}$ is not influenced by the outcomes of $\mathbf{X_2}$, and hence is conditionally independent of X_2. To illustrate the notion of conditional independence, consider the relationship between a person's arm length ($\mathbf{X_1}$) and his or her reading skills ($\mathbf{X_2}$). One might observe that people with longer arms tend to have higher levels of reading skills, and thus consider $\mathbf{X_1}$ and $\mathbf{X_2}$ to be related to each other. However, this relationship can be explained by another factor, which is the age of the person ($\mathbf{Y}$). A young child tends to have short arms and lacks the reading skills of an adult. If the age of a person is fixed, then the observed relationship between arm length and reading skills disappears. Thus, we can conclude that arm length and reading skills are not directly related to each other and are conditionally independent when the age variable is fixed.

Another way of describing conditional independence is to consider the joint conditional probability, $P(\mathbf{X_1}, \mathbf{X_2}|\mathbf{Y})$, as follows:

$$\begin{aligned} P(\mathbf{X_1}, \mathbf{X_2}|\mathbf{Y}) &= \frac{P(\mathbf{X_1}, \mathbf{X_2}, \mathbf{Y})}{P(\mathbf{Y})} \\ &= \frac{P(\mathbf{X_1}, \mathbf{X_2}, \mathbf{Y})}{P(\mathbf{X_2}, \mathbf{Y})} \times \frac{P(\mathbf{X_2}, \mathbf{Y})}{P(\mathbf{Y})} \\ &= P(\mathbf{X_1}|\mathbf{X_2}, \mathbf{Y}) \times P(\mathbf{X_2}|\mathbf{Y}) \\ &= P(\mathbf{X_1}|\mathbf{Y}) \times P(\mathbf{X_2}|\mathbf{Y}), \end{aligned} \tag{6.17}$$

where Equation 6.16 was used to obtain the last line of Equation 6.17. The previous description of conditional independence is quite useful from an operational perspective. It states that the joint conditional probability of $\mathbf{X_1}$ and $\mathbf{X_2}$ given Y can be factored as the product of conditional probabilities of $\mathbf{X_1}$ and $\mathbf{X_2}$ considered separately. This forms the basis of the naïve Bayes assumption stated in Equation 6.15.

How a Naïve Bayes Classifier Works

Using the naïve Bayes assumption, we only need to estimate the conditional probability of each x_i given Y separately, instead of computing the class-conditional probability for every combination of attribute values. For example, if n_i^0 and n_j^0 denote the number of training instances belonging to class 0 with $X_1 = c_i$ and $X_2 = c_j$, respectively, then the class-conditional probability can be estimated as

$$P(X_1 = c_i, X_2 = c_j|Y = 0) = \frac{n_i^0}{n^0} \times \frac{n_j^0}{n^0}.$$

In the previous equation, we only need to count the number of training instances for every one of the k values of an attribute X, irrespective of the values of other attributes. Hence, the number of parameters needed to learn class-conditional probabilities is reduced from d^k to dk. This greatly simplifies the expression for the class-conditional probability and makes it more amenable to learning parameters and making predictions, even in high-dimensional settings.

The naïve Bayes classifier computes the posterior probability for a test instance $\mathbf{x}$ by using the following equation:

$$P(y|\mathbf{x}) = \frac{P(y) \prod_{i=1}^{d} P(x_i|y)}{P(\mathbf{x})}. \tag{6.18}$$

Since $P(\mathbf{x})$ is fixed for every y and only acts as a normalizing constant to ensure that $P(y|\mathbf{x}) \in [0, 1]$, we can write

$$P(y|\mathbf{x}) \propto P(y) \prod_{i=1}^{d} P(x_i|y).$$

Hence, it is sufficient to choose the class that maximizes $P(y) \prod_{i=1}^{d} P(x_i|y)$.

One of the useful properties of the naïve Bayes classifier is that it can easily work with incomplete information about data instances, when only a subset of attributes are observed at every instance. For example, if we only observe p out of d attributes at a data instance, then we can still compute $P(y) \prod_{i=1}^{p} P(x_i|y)$ using those p attributes and choose the class with the maximum value. The naïve Bayes classifier can thus naturally handle missing values in test instances. In fact, in the extreme case where no attributes are observed, we can still use the prior probability $P(y)$ as an estimate of the posterior probability. As we observe more attributes, we can keep refining the posterior probability to better reflect the likelihood of observing the data instance.

In the next two subsections, we describe several approaches for estimating the conditional probabilities $P(x_i|y)$ for categorical and continuous attributes from the training set.

Estimating Conditional Probabilities for Categorical Attributes

For a categorical attribute X_i, the conditional probability $P(X_i = c|y)$ is estimated according to the fraction of training instances in class y where X_i takes on a particular categorical value c.

$$P(X_i = c|y) = \frac{n_c}{n},$$

where n is the number of training instances belonging to class y, out of which n_c number of instances have $X_i = c$. For example, in the training set given in Figure 6.8, seven people have the class label `Defaulted Borrower=No`, out of which three people have `Home Owner=Yes` while the remaining four have `Home Owner=No`. As a result, the conditional probability for P(`Home Owner=Yes`| `Defaulted Borrower=No`) is equal to 3/7. Similarly, the conditional probability for defaulted borrowers with `Marital Status = Single` is given by P(`Marital Status = Single`|`Defaulted Borrower=Yes`) = 2/3. Note that the sum of conditional probabilities over all possible outcomes of X_i is equal to one, i.e., $\sum_c P(X_i = c|y) = 1$.

	binary	categorical	continuous	class
Tid	Home Owner	Marital Status	Annual Income	Defaulted Borrower
1	Yes	Single	125K	No
2	No	Married	100K	No
3	No	Single	70K	No
4	Yes	Married	120K	No
5	No	Divorced	95K	Yes
6	No	Married	60K	No
7	Yes	Divorced	220K	No
8	No	Single	85K	Yes
9	No	Married	75K	No
10	No	Single	90K	Yes

Figure 6.8. Training set for predicting the loan default problem.

Estimating Conditional Probabilities for Continuous Attributes

There are two ways to estimate the class-conditional probabilities for continuous attributes:

1. We can discretize each continuous attribute and then replace the continuous values with their corresponding discrete intervals. This approach transforms the continuous attributes into ordinal attributes, and the simple method described previously for computing the conditional probabilities of categorical attributes can be employed. Note that the estimation error of this method depends on the discretization strategy (as described in Section 2.3.6 on page 83), as well as the number of discrete intervals. If the number of intervals is too large, every interval may have an insufficient number of training instances to provide a reliable estimate of $P(X_i|Y)$. On the other hand, if the number of intervals is too small, then the discretization process may loose information about the true distribution of continuous values, and thus result in poor predictions.

2. We can assume a certain form of probability distribution for the continuous variable and estimate the parameters of the distribution using the training data. For example, we can use a Gaussian distribution to represent the conditional probability of continuous attributes. The Gaussian distribution is characterized by two parameters, the mean, μ, and the variance, σ^2. For each class y_j, the class-conditional probability

for attribute X_i is

$$P(X_i = x_i|Y = y_j) = \frac{1}{\sqrt{2\pi}\sigma_{ij}} \exp\left[- \frac{(x_i - \mu_{ij})^2}{2\sigma_{ij}^2} \right]. \tag{6.19}$$

The parameter μ_{ij} can be estimated using the sample mean of X_i ($\overline{x}$) for all training instances that belong to y_j. Similarly, σ_{ij}^2 can be estimated from the sample variance (s^2) of such training instances. For example, consider the annual income attribute shown in Figure 6.8. The sample mean and variance for this attribute with respect to the class `No` are

$$\overline{x} = \frac{125 + 100 + 70 + \ldots + 75}{7} = 110$$
$$s^2 = \frac{(125 - 110)^2 + (100 - 110)^2 + \ldots + (75 - 110)^2}{6} = 2975$$
$$s = \sqrt{2975} = 54.54.$$

Given a test instance with taxable income equal to \$120K, we can use the following value as its conditional probability given class `No`:

$$P(\texttt{Income=120}|\texttt{No}) = \frac{1}{\sqrt{2\pi}(54.54)} \exp^{-\frac{(120-110)^2}{2\times 2975}} = 0.0072.$$

Example 6.5. [Naïve Bayes Classifier] Consider the data set shown in Figure 6.9(a), where the target class is `Defaulted Borrower`, which can take two values `Yes` and `No`. We can compute the class-conditional probability for each categorical attribute and the sample mean and variance for the continuous attribute, as summarized in Figure 6.9(b).

We are interested in predicting the class label of a test instance $\mathbf{x}$ = (`Home Owner`=`No`, `Marital Status` = `Married`, `Annual Income` = \$120K). To do this, we first compute the prior probabilities by counting the number of training instances belonging to every class. We thus obtain P(`Yes`) = 0.3 and P(`No`) =

Tid	Home Owner	Marital Status	Annual Income	Defaulted Borrower
1	Yes	Single	125K	**No**
2	No	Married	100K	**No**
3	No	Single	70K	**No**
4	Yes	Married	120K	**No**
5	No	Divorced	95K	**Yes**
6	No	Married	60K	**No**
7	Yes	Divorced	220K	**No**
8	No	Single	85K	**Yes**
9	No	Married	75K	**No**
10	No	Single	90K	**Yes**

(a)

P(Home Owner=Yes|No) = 3/7
P(Home Owner=No|No) = 4/7
P(Home Owner=Yes|Yes) = 0
P(Home Owner=No|Yes) = 1
P(Marital Status=Single|No) = 2/7
P(Marital Status=Divorced|No) = 1/7
P(Marital Status=Married|No) = 4/7
P(Marital Status=Single|Yes) = 2/3
P(Marital Status=Divorced|Yes) = 1/3
P(Marital Status=Married|Yes) = 0

For Annual Income:
If class=No: sample mean=110
sample variance=2975
If class=Yes: sample mean=90
sample variance=25

(b)

Figure 6.9. The naïve Bayes classifier for the loan classification problem.

0.7. Next, we can compute the class-conditional probability as follows:

$$\begin{aligned} P(\mathbf{x}|\texttt{No}) &= P(\texttt{Home Owner} = \texttt{No}|\texttt{No}) \times P(\texttt{Status} = \texttt{Married}|\texttt{No}) \\ &\quad \times P(\texttt{Annual Income} = \$120\text{K}|\texttt{No}) \\ &= 4/7 \times 4/7 \times 0.0072 = 0.0024. \end{aligned}$$

$$\begin{aligned} P(\mathbf{x}|\texttt{Yes}) &= P(\texttt{Home Owner} = \texttt{No}|\texttt{Yes}) \times P(\texttt{Status} = \texttt{Married}|\texttt{Yes}) \\ &\quad \times P(\texttt{Annual Income} = \$120\text{K}|\texttt{Yes}) \\ &= 1 \times 0 \times 1.2 \times 10^{-9} = 0. \end{aligned}$$

Notice that the class-conditional probability for class `Yes` has become 0 because there are no instances belonging to class `Yes` with `Status` = `Married` in the training set. Using these class-conditional probabilities, we can estimate the posterior probabilities as

$$\begin{aligned} P(\texttt{No}|\mathbf{x}) &= \frac{0.7 \times 0.0024}{P(\mathbf{x})}. \\ &= 0.0016\alpha. \\ P(\texttt{Yes}|\mathbf{x}) &= \frac{0.3 \times 0}{P(\mathbf{x})}. \\ &= 0. \end{aligned}$$

where $\alpha = 1/P(\mathbf{X})$ is a normalizing constant. Since $P(\texttt{No}|\mathbf{x}) > P(\texttt{Yes}|\mathbf{x})$, the instance is classified as `No`. ■

Handling Zero Conditional Probabilities

The preceding example illustrates a potential problem with using the naïve Bayes assumption in estimating class-conditional probabilities. If the conditional probability for any of the attributes is zero, then the entire expression for the class-conditional probability becomes zero. Note that zero conditional probabilities arise when the number of training instances is small and the number of possible values of an attribute is large. In such cases, it may happen that a combination of attribute values and class labels are never observed, resulting in a zero conditional probability.

In a more extreme case, if the training instances do not cover some combinations of attribute values and class labels, then we may not be able to even classify some of the test instances. For example, if $P(\texttt{Marital Status} = \texttt{Divorced}|\texttt{No})$ is zero instead of $1/7$, then a data instance with attribute set $\mathbf{x} = (\texttt{Home Owner} = \texttt{Yes}, \texttt{Marital Status} = \texttt{Divorced}, \texttt{Income} = \$120\text{K})$ has the following class-conditional probabilities:

$$P(\mathbf{x}|\texttt{No}) = 3/7 \times 0 \times 0.0072 = 0.$$
$$P(\mathbf{x}|\texttt{Yes}) = 0 \times 1/3 \times 1.2 \times 10^{-9} = 0.$$

Since both the class-conditional probabilities are 0, the naïve Bayes classifier will not be able to classify the instance. To address this problem, it is important to adjust the conditional probability estimates so that they are not as brittle as simply using fractions of training instances. This can be achieved by using the following alternate estimates of conditional probability:

$$\text{Laplace estimate:} \quad P(X_i = c|y) = \frac{n_c + 1}{n + v}, \tag{6.20}$$

$$\text{m-estimate:} \quad P(X_i = c|y) = \frac{n_c + mp}{n + m}, \tag{6.21}$$

where n is the number of training instances belonging to class y, n_c is the number of training instances with $X_i = c$ and $Y = y$, v is the total number of attribute values that X_i can take, p is some initial estimate of $P(X_i = c|y)$ that is known a priori, and m is a hyper-parameter that indicates our confidence in using p when the fraction of training instances is too brittle. Note that even

if $n_c = 0$, both Laplace and m-estimate provide non-zero values of conditional probabilities. Hence, they avoid the problem of vanishing class-conditional probabilities and thus generally provide more robust estimates of posterior probabilities.

Characteristics of Naïve Bayes Classifiers

1. Naïve Bayes classifiers are probabilistic classification models that are able to quantify the uncertainty in predictions by providing posterior probability estimates. They are also generative classification models as they treat the target class as the causative factor for generating the data instances. Hence, apart from computing posterior probabilities, naïve Bayes classifiers also attempt to capture the underlying mechanism behind the generation of data instances belonging to every class. They are thus useful for gaining predictive as well as descriptive insights.

2. By using the naïve Bayes assumption, they can easily compute class-conditional probabilities even in high-dimensional settings, provided that the attributes are conditionally independent of each other given the class labels. This property makes naïve Bayes classifier a simple and effective classification technique that is commonly used in diverse application problems, such as text classification.

3. Naïve Bayes classifiers are robust to isolated noise points because such points are not able to significantly impact the conditional probability estimates, as they are often averaged out during training.

4. Naïve Bayes classifiers can handle missing values in the training set by ignoring the missing values of every attribute while computing its conditional probability estimates. Further, naïve Bayes classifiers can effectively handle missing values in a test instance, by using only the non-missing attribute values while computing posterior probabilities. If the frequency of missing values for a particular attribute value depends on class label, then this approach will not accurately estimate posterior probabilities.

5. Naïve Bayes classifiers are robust to irrelevant attributes. If X_i is an irrelevant attribute, then $P(X_i|Y)$ becomes almost uniformly distributed for every class y. The class-conditional probabilities for every class thus receive similar contributions of $P(X_i|Y)$, resulting in negligible impact on the posterior probability estimates.

6. Correlated attributes can degrade the performance of naïve Bayes classifiers because the naïve Bayes assumption of conditional independence no longer holds for such attributes. For example, consider the following probabilities:

$$P(A=0|Y=0)=0.4, \quad P(A=1|Y=0)=0.6,$$
$$P(A=0|Y=1)=0.6, \quad P(A=1|Y=1)=0.4,$$

where A is a binary attribute and Y is a binary class variable. Suppose there is another binary attribute B that is perfectly correlated with A when $Y=0$, but is independent of A when $Y=1$. For simplicity, assume that the conditional probabilities for B are the same as for A. Given an instance with attributes $A=0, B=0$, and assuming conditional independence, we can compute its posterior probabilities as follows:

$$\begin{aligned} P(Y=0|A=0,B=0) &= \frac{P(A=0|Y=0)P(B=0|Y=0)P(Y=0)}{P(A=0,B=0)} \\ &= \frac{0.16\times P(Y=0)}{P(A=0,B=0)}. \end{aligned}$$

$$\begin{aligned} P(Y=1|A=0,B=0) &= \frac{P(A=0|Y=1)P(B=0|Y=1)P(Y=1)}{P(A=0,B=0)} \\ &= \frac{0.36\times P(Y=1)}{P(A=0,B=0)}. \end{aligned}$$

If $P(Y=0)=P(Y=1)$, then the naïve Bayes classifier would assign the instance to class 1. However, the truth is,

$$P(A=0,B=0|Y=0)=P(A=0|Y=0)=0.4,$$

because A and B are perfectly correlated when $Y=0$. As a result, the posterior probability for $Y=0$ is

$$\begin{aligned} P(Y=0|A=0,B=0) &= \frac{P(A=0,B=0|Y=0)P(Y=0)}{P(A=0,B=0)} \\ &= \frac{0.4\times P(Y=0)}{P(A=0,B=0)}, \end{aligned}$$

which is larger than that for $Y=1$. The instance should have been classified as class 0. Hence, the naïve Bayes classifier can produce incorrect

results when the attributes are not conditionally independent given the class labels. Naïve Bayes classifiers are thus not well-suited for handling redundant or interacting attributes.

6.5 Bayesian Networks

The conditional independence assumption made by naïve Bayes classifiers may seem too rigid, especially for classification problems where the attributes are dependent on each other even after conditioning on the class labels. We thus need an approach to relax the naïve Bayes assumption so that we can capture more generic representations of conditional independence among attributes.

In this section, we present a flexible framework for modeling probabilistic relationships between attributes and class labels, known as **Bayesian Networks**. By building on concepts from probability theory and graph theory, Bayesian networks are able to capture more generic forms of conditional independence using simple schematic representations. They also provide the necessary computational structure to perform inferences over random variables in an efficient way. In the following, we first describe the basic representation of a Bayesian network, and then discuss methods for performing inference and learning model parameters in the context of classification.

6.5.1 Graphical Representation

Bayesian networks belong to a broader family of models for capturing probabilistic relationships among random variables, known as **probabilistic graphical models**. The basic concept behind these models is to use graphical representations where the nodes of the graph correspond to random variables and the edges between the nodes express probabilistic relationships. Figures 6.10(a) and 6.10(b) show examples of probabilistic graphical models using directed edges (with arrows) and undirected edges (without arrows), respectively. Directed graphical models are also known as Bayesian networks while undirected graphical models are known as **Markov random fields**. The two approaches use different semantics for expressing relationships among random variables and are thus useful in different contexts. In the following, we briefly describe Bayesian networks that are useful in the context of classification.

A Bayesian network (also referred to as a **belief network**) involves directed edges between nodes, where every edge represents a direction of influence among random variables. For example, Figure 6.10(a) shows a Bayesian network where variable C depends upon the values of variables A and B, as indicated by the arrows pointing toward C from A and B. Consequently,

require more training instances for effectively learning the probability tables than the naïve Bayes classifier.

7. Although the sum-product algorithm provides computationally efficient techniques for performing inference over tree-like graphs, the complexity of the approach increase significantly when dealing with generic graphs of large sizes. In situations where exact inference is computationally infeasible, it is quite common to use approximate inference techniques.

6.6 Logistic Regression

The naïve Bayes and the Bayesian network classifiers described in the previous sections provide different ways of estimating the conditional probability of an instance $\mathbf{x}$ given class y, $P(\mathbf{x}|y)$. Such models are known as **probabilistic generative models**. Note that the conditional probability $P(\mathbf{x}|y)$ essentially describes the behavior of instances in the attribute space that are *generated* from class y. However, for the purpose of making predictions, we are finally interested in computing the posterior probability $P(y|\mathbf{x})$. For example, computing the following ratio of posterior probabilities is sufficient for inferring class labels in a binary classification problem:

$$\frac{P(y=1|\mathbf{x})}{P(y=0|\mathbf{x})}$$

This ratio is known as the **odds**. If this ratio is greater than 1, then $\mathbf{x}$ is classified as $y = 1$. Otherwise, it is assigned to class $y = 0$. Hence, one may simply learn a model of the odds based on the attribute values of training instances, without having to compute $P(\mathbf{x}|y)$ as an intermediate quantity in the Bayes theorem.

Classification models that directly assign class labels without computing class-conditional probabilities are called **discriminative models**. In this section, we present a probabilistic discriminative model known as **logistic regression**, which directly estimates the odds of a data instance $\mathbf{x}$ using its attribute values. The basic idea of logistic regression is to use a linear predictor, $z = \mathbf{w}^T\mathbf{x} + b$, for representing the odds of $\mathbf{x}$ as follows:

$$\frac{P(y=1|\mathbf{x})}{P(y=0|\mathbf{x})} = e^z = e^{\mathbf{w}^T\mathbf{x}+b}, \tag{6.37}$$

where $\mathbf{w}$ and b are the parameters of the model and $\mathbf{a}^T$ denotes the transpose of a vector $\mathbf{a}$. Note that if $\mathbf{w}^T\mathbf{x} + b > 0$, then $\mathbf{x}$ belongs to class 1 since its odds is greater than 1. Otherwise, $\mathbf{x}$ belongs to class 0.

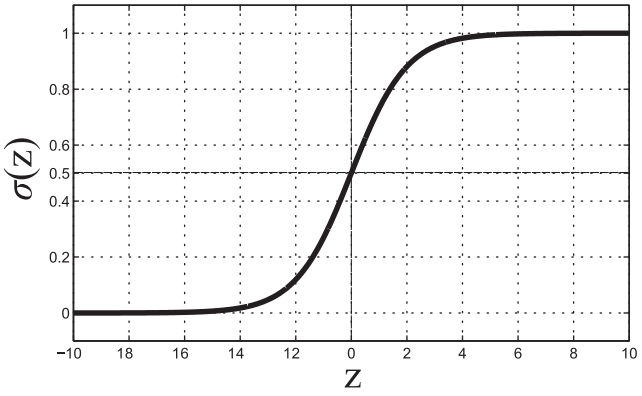


Figure 6.19. Plot of sigmoid (logistic) function, $\sigma(z)$.

Since $P(y = 0|\mathbf{x}) + P(y = 1|\mathbf{x}) = 1$, we can re-write Equation 6.37 as

$$\frac{P(y = 1|\mathbf{x})}{1 - P(y = 1|\mathbf{x})} = e^z.$$

This can be further simplified to express $P(y = 1|\mathbf{x})$ as a function of z.

$$P(y = 1|\mathbf{x}) = \frac{1}{1 + e^{-z}} = \sigma(z), \tag{6.38}$$

where the function $\sigma(.)$ is known as the logistic or sigmoid function. Figure 6.19 shows the behavior of the sigmoid function as we vary z. We can see that $\sigma(z) \geq 0.5$ only when $z \geq 0$. We can also derive $P(y = 0|\mathbf{x})$ using $\sigma(z)$ as follows:

$$P(y = 0|\mathbf{x}) = 1 - \sigma(z) = \frac{1}{1 + e^z} \tag{6.39}$$

Hence, if we have learned a suitable value of parameters $\mathbf{w}$ and b, we can use Equations 6.38 and 6.39 to estimate the posterior probabilities of any data instance $\mathbf{x}$ and determine its class label.

6.6.1 Logistic Regression as a Generalized Linear Model

Since the posterior probabilities are real-valued, their estimation using the previous equations can be viewed as solving a regression problem. In fact,

logistic regression belongs to a broader family of statistical regression models, known as **generalized linear models** (GLM). In these models, the target variable y is considered to be generated from a probability distribution $P(y|\mathbf{x})$, whose mean μ can be estimated using a link function $g(.)$ as follows:

$$g(\mu) = z = \mathbf{w}^T\mathbf{x} + b. \tag{6.40}$$

For binary classification using logistic regression, y follows a Bernoulli distribution (y can either be 0 or 1) and μ is equal to $P(y = 1|\mathbf{x})$. The link function $g(.)$ of logistic regression, called the logit function, can thus be represented as

$$g(\mu) = \log\left(\frac{\mu}{1-\mu}\right).$$

Depending on the choice of link function $g(.)$ and the form of probability distribution $P(y|\mathbf{x})$, GLMs are able to represent a broad family of regression models, such as linear regression and Poisson regression. They require different approaches for estimating their model parameters, $(\mathbf{w}, b)$. In this chapter, we will only discuss approaches for estimating the model parameters of logistic regression, although methods for estimating parameters of other types of GLMs are often similar (and sometimes even simpler). (See Bibliographic Notes for more details on GLMs.)

Note that even though logistic regression has relationships with regression models, it is a classification model since the computed posterior probabilities are eventually used to determine the class label of a data instance.

6.6.2 Learning Model Parameters

The parameters of logistic regression, $(\mathbf{w}, b)$, are estimated during training using a statistical approach known as the maximum likelihood estimation (MLE) method. This method involves computing the likelihood of observing the training data given $(\mathbf{w}, b)$, and then determining the model parameters $(\mathbf{w}^*, b^*)$ that yield maximum likelihood.

Let $D.train = \{(\mathbf{x_1}, y_1), (\mathbf{x_2}, y_2), \ldots, (\mathbf{x_n}, y_n)\}$ denote a set of n training instances, where y_i is a binary variable (0 or 1). For a given training instance $\mathbf{x_i}$, we can compute its posterior probabilities using Equations 6.38 and 6.39. We can then express the likelihood of observing y_i given $\mathbf{x_i}$, $\mathbf{w}$, and b as

$$\begin{aligned} P(y_i|\mathbf{x_i}, \mathbf{w}, b) &= P(y = 1|\mathbf{x_i})^{y_i} \times P(y = 0|\mathbf{x_i})^{1-y_i}, \\ &= (\sigma(z_i))^{y_i} \times (1 - \sigma(z_i))^{1-y_i}, \\ &= (\sigma(\mathbf{w}^T\mathbf{x_i} + b))^{y_i} \times (1 - \sigma(\mathbf{w}^T\mathbf{x_i} + b))^{1-y_i}, \end{aligned} \tag{6.41}$$

where $\sigma(.)$ is the sigmoid function as described above, Equation 6.41 basically means that the likelihood $P(y_i|\mathbf{x_i}, \mathbf{w}, b)$ is equal to $P(y = 1|\mathbf{x_i})$ when $y_i = 1$, and equal to $P(y = 0|\mathbf{x_i})$ when $y_i = 0$. The likelihood of all training instances, $\mathcal{L}(\mathbf{w}, b)$, can then be computed by taking the product of individual likelihoods (assuming independence among training instances) as follows:

$$\mathcal{L}(\mathbf{w}, b) = \prod_{i=1}^{n} P(y_i|\mathbf{x_i}, \mathbf{w}, b) = \prod_{i=1}^{n} P(y = 1|\mathbf{x_i})^{y_i} \times P(y = 0|\mathbf{x_i})^{1-y_i}. \quad (6.42)$$

The previous equation involves multiplying a large number of probability values, each of which are smaller than or equal to 1. Since this naïve computation can easily become numerically unstable when n is large, a more practical approach is to consider the negative logarithm (to base e) of the likelihood function, also known as the **cross entropy function**:

$$\begin{aligned} -\log \mathcal{L}(\mathbf{w}, b) &= -\sum_{i=1}^{n} y_i \log(P(y = 1|\mathbf{x_i})) + (1 - y_i) \log(P(y = 0|\mathbf{x_i})). \\ &= -\sum_{i=1}^{n} y_i \log(\sigma(\mathbf{w}^T \mathbf{x_i} + b)) + (1 - y_i) \log(1 - \sigma(\mathbf{w}^T \mathbf{x_i} + b)). \end{aligned}$$

The cross entropy is a loss function that measures how unlikely it is for the training data to be generated from the logistic regression model with parameters $(\mathbf{w}, b)$. Intuitively, we would like to find model parameters $(\mathbf{w}^*, b^*)$ that result in the lowest cross entropy, $-\log \mathcal{L}(\mathbf{w}^*, b^*)$.

$$\begin{aligned} (\mathbf{w}^*, b^*) &= \arg\min_{(\mathbf{w},b)} E(\mathbf{w}, b) \\ &= \arg\min_{(\mathbf{w},b)} -\log \mathcal{L}(\mathbf{w}, b), \end{aligned} \quad (6.43)$$

where $E(\mathbf{w}, b) = -\log \mathcal{L}(\mathbf{w}, b)$ is the loss function. It is worth emphasizing that $E(\mathbf{w}, b)$ is a convex function, i.e., any minima of $E(\mathbf{w}, b)$ will be a global minima. Hence, we can use any of the standard convex optimization techniques to solve Equation 6.43, which are mentioned in Appendix E. Here, we briefly describe the Newton-Raphson method that is commonly used for estimating the parameters of logistic regression. For ease of representation, we will use a single vector to describe $\tilde{\mathbf{w}} = (\mathbf{w}^T \ b)^T$, which is of size one greater than $\mathbf{w}$.

Similarly, we will consider the concatenated feature vector $\tilde{\mathbf{x}} = (\mathbf{x}^T\ 1)^T$, such that the linear predictor $z = \mathbf{w}^T\mathbf{x} + b$ can be succinctly written as $z = \tilde{\mathbf{w}}^T\tilde{\mathbf{x}}$. Also, the concatenation of all training labels, y_1 to y_n, will be represented as $\mathbf{y}$, the set consisting of $\sigma(z_1)$ to $\sigma(z_n)$ will be represented as $\boldsymbol{\sigma}$, and the concatenation of $\tilde{\mathbf{x}}_\mathbf{1}$ to $\tilde{\mathbf{x}}_\mathbf{n}$ will be represented as $\tilde{\mathbf{X}}$.

The Newton-Raphson is an iterative method for finding $\tilde{\mathbf{w}}^*$ that uses the following equation to update the model parameters at every iteration:

$$\tilde{\mathbf{w}}^{(new)} = \tilde{\mathbf{w}}^{(old)} - \mathbf{H}^{-1}\nabla E(\tilde{\mathbf{w}}), \tag{6.44}$$

where $\nabla E(\tilde{\mathbf{w}})$ and $\mathbf{H}$ are the first- and second-order derivatives of the loss function $E(\tilde{\mathbf{w}})$ with respect to $\tilde{\mathbf{w}}$, respectively. The key intuition behind Equation 6.44 is to move the model parameters in the direction of maximum gradient, such that $\tilde{\mathbf{w}}$ takes larger steps when $\nabla E(\tilde{\mathbf{w}})$ is large. When $\tilde{\mathbf{w}}$ arrives at a minima after some number of iterations, then $\nabla E(\tilde{\mathbf{w}})$ would become equal to 0 and thus result in convergence. Hence, we start with some initial values of $\tilde{\mathbf{w}}$ (either randomly assigned or set to 0) and use Equation 6.44 to iteratively update $\tilde{\mathbf{w}}$ till there are no significant changes in its value (beyond a certain threshold).

The first-order derivative of $E(\tilde{\mathbf{w}})$ is given by

$$\begin{aligned}
\nabla E(\tilde{\mathbf{w}}) &= -\sum_{i=1}^{n} y_i\tilde{\mathbf{x}}_\mathbf{i}(1-\sigma(\tilde{\mathbf{w}}^T\tilde{\mathbf{x}}_\mathbf{i})) - (1-y_i)\tilde{\mathbf{x}}_\mathbf{i}\sigma(\tilde{\mathbf{w}}^T\tilde{\mathbf{x}}_\mathbf{i}), \\
&= \sum_{i=1}^{n}(\sigma(\tilde{\mathbf{w}}^T\tilde{\mathbf{x}}_\mathbf{i}) - y_i)\tilde{\mathbf{x}}_\mathbf{i}, \\
&= \tilde{\mathbf{X}}(\boldsymbol{\sigma} - \mathbf{y}),
\end{aligned} \tag{6.45}$$

where we have used the fact that $d\sigma(z)/dz = \sigma(z)(1-\sigma(z))$. Using $\nabla E(\tilde{\mathbf{w}})$, we can compute the second-order derivative of $E(\tilde{\mathbf{w}})$ as

$$\begin{aligned}
\mathbf{H} = \nabla\nabla E(\tilde{\mathbf{w}}) &= \sum_{i=1}^{n}\sigma(\tilde{\mathbf{w}}^T\tilde{\mathbf{x}}_\mathbf{i})(1-\sigma(\tilde{\mathbf{w}}^T\tilde{\mathbf{x}}_\mathbf{i}))\tilde{\mathbf{x}}_\mathbf{i}\tilde{\mathbf{x}}_\mathbf{i}{}^T. \\
&= \tilde{\mathbf{X}}^T\mathbf{R}\tilde{\mathbf{X}},
\end{aligned} \tag{6.46}$$

where $\mathbf{R}$ is a diagonal matrix whose i^{th} diagonal element $\mathbf{R}_{ii} = \sigma_i(1-\sigma_i)$. We can now use the first- and second-order derivatives of $E(\tilde{\mathbf{w}})$ in Equation 6.44 to obtain the following update equation at the kth iteration:

$$\tilde{\mathbf{w}}^{(k+1)} = \tilde{\mathbf{w}}^{(k)} - (\tilde{\mathbf{X}}^T\mathbf{R}_k\tilde{\mathbf{X}})^{-1}\tilde{\mathbf{X}}^T(\boldsymbol{\sigma}_k - \mathbf{y}) \tag{6.47}$$

where the subscript k under $\mathbf{R}_k$ and $\boldsymbol{\sigma}_k$ refers to using $\tilde{\mathbf{w}}^{(k)}$ to compute both terms.

6.6.3 Characteristics of Logistic Regression

1. Logistic Regression is a discriminative model for classification that directly computes the poster probabilities without making any assumption about the class conditional probabilities. Hence, it is quite generic and can be applied in diverse applications. It can also be easily extended to multiclass classification, where it is known as **multinomial logistic regression**. However, its expressive power is limited to learning only linear decision boundaries.

2. Because there are different weights (parameters) for every attribute, the learned parameters of logistic regression can be analyzed to understand the relationships between attributes and class labels.

3. Because logistic regression does not involve computing densities and distances in the attribute space, it can work more robustly even in high-dimensional settings than distance-based methods such as nearest neighbor classifiers. However, the objective function of logistic regression does not involve any term relating to the complexity of the model. Hence, logistic regression does not provide a way to make a trade-off between model complexity and training performance, as compared to other classification models such as support vector machines. Nevertheless, variants of logistic regression can easily be developed to account for model complexity, by including appropriate terms in the objective function along with the cross entropy function.

4. Logistic regression can handle irrelevant attributes by learning weight parameters close to 0 for attributes that do not provide any gain in performance during training. It can also handle interacting attributes since the learning of model parameters is achieved in a joint fashion by considering the effects of all attributes together. Furthermore, if there are redundant attributes that are duplicates of each other, then logistic regression can learn equal weights for every redundant attribute, without degrading classification performance. However, the presence of a large number of irrelevant or redundant attributes in high-dimensional settings can make logistic regression susceptible to model overfitting.

5. Logistic regression cannot handle data instances with missing values, since the posterior probabilities are only computed by taking a weighted

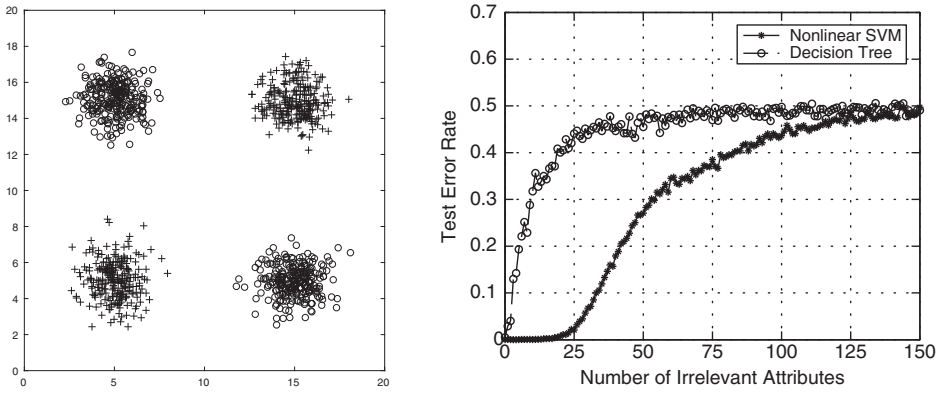
sum of all the attributes. If there are missing values in a training instance, it can be discarded from the training set. However, if there are missing values in a test instance, then logistic regression would fail to predict its class label.

6.7 Artificial Neural Network (ANN)

Artificial neural networks (ANN) are powerful classification models that are able to learn highly complex and nonlinear decision boundaries purely from the data. They have gained widespread acceptance in several applications such as vision, speech, and language processing, where they have been repeatedly shown to outperform other classification models (and in some cases even human performance). Historically, the study of artificial neural networks was inspired by attempts to emulate biological neural systems. The human brain consists primarily of nerve cells called **neurons**, linked together with other neurons via strands of fiber called **axons**. Whenever a neuron is stimulated (e.g., in response to a stimuli), it transmits nerve activations via axons to other neurons. The receptor neurons collect these nerve activations using structures called **dendrites**, which are extensions from the cell body of the neuron. The strength of the contact point between a dendrite and an axon, known as a **synapse**, determines the connectivity between neurons. Neuroscientists have discovered that the human brain learns by changing the strength of the synaptic connection between neurons upon repeated stimulation by the same impulse.

The human brain consists of approximately 100 billion neurons that are inter-connected in complex ways, making it possible for us to learn new tasks and perform regular activities. Note that a single neuron only performs a simple modular function, which is to respond to the nerve activations coming from sender neurons connected at its dendrite, and transmit its activation to receptor neurons via axons. However, it is the composition of these simple functions that together is able to express complex functions. This idea is at the basis of constructing artificial neural networks.

Analogous to the structure of a human brain, an artificial neural network is composed of a number of processing units, called nodes, that are connected with each other via directed links. The nodes correspond to neurons that perform the basic units of computation, while the directed links correspond to connections between neurons, consisting of axons and dendrites. Further, the weight of a directed link between two neurons represents the strength of the synaptic connection between neurons. As in biological neural systems, the



(a) Original data with two attributes.

(b) Test error rates after adding irrelevant attributes to the original data.

Figure 6.41. Comparing the effect of adding irrelevant attributes on the performance of nonlinear SVMs and decision trees.

4. SVM can be applied to categorical data by introducing dummy variables for each categorical attribute value present in the data. For example, if `Marital Status` has three values {`Single`, `Married`, `Divorced`}, we can introduce a binary variable for each of the attribute values.

5. The SVM formulation presented in this chapter is for binary class problems. However, multiclass extensions of SVM have also been proposed.

6. Although the training time of an SVM model can be large, the learned parameters can be succinctly represented with the help of a small number of support vectors, making the classification of test instances quite fast.

6.10 Ensemble Methods

This section presents techniques for improving classification accuracy by aggregating the predictions of multiple classifiers. These techniques are known as **ensemble** or **classifier combination** methods. An ensemble method constructs a set of **base classifiers** from training data and performs classification by taking a vote on the predictions made by each base classifier. This section explains why ensemble methods tend to perform better than any single classifier and presents techniques for constructing the classifier ensemble.

6.10.1 Rationale for Ensemble Method

The following example illustrates how an ensemble method can improve a classifier's performance.

Example 6.8. Consider an ensemble of 25 binary classifiers, each of which has an error rate of $\epsilon = 0.35$. The ensemble classifier predicts the class label of a test example by taking a majority vote on the predictions made by the base classifiers. If the base classifiers are identical, then all the base classifiers will commit the same mistakes. Thus, the error rate of the ensemble remains 0.35. On the other hand, if the base classifiers are independent—i.e., their errors are uncorrelated—then the ensemble makes a wrong prediction only if more than half of the base classifiers predict incorrectly. In this case, the error rate of the ensemble classifier is

$$e_{\text{ensemble}} = \sum_{i=13}^{25} \binom{25}{i} \epsilon^i (1-\epsilon)^{25-i} = 0.06, \tag{6.101}$$

which is considerably lower than the error rate of the base classifiers. ■

Figure 6.42 shows the error rate of an ensemble of 25 binary classifiers (e_{ensemble}) for different base classifier error rates (ϵ). The diagonal line represents the case in which the base classifiers are identical, while the solid line represents the case in which the base classifiers are independent. Observe that the ensemble classifier performs worse than the base classifiers when ϵ is larger than 0.5.

The preceding example illustrates two necessary conditions for an ensemble classifier to perform better than a single classifier: (1) the base classifiers should be independent of each other, and (2) the base classifiers should do better than a classifier that performs random guessing. In practice, it is difficult to ensure total independence among the base classifiers. Nevertheless, improvements in classification accuracies have been observed in ensemble methods in which the base classifiers are somewhat correlated.

6.10.2 Methods for Constructing an Ensemble Classifier

A logical view of the ensemble method is presented in Figure 6.43. The basic idea is to construct multiple classifiers from the original data and then aggregate their predictions when classifying unknown examples. The ensemble of classifiers can be constructed in many ways:

1. **By manipulating the training set.** In this approach, multiple training sets are created by resampling the original data according to some

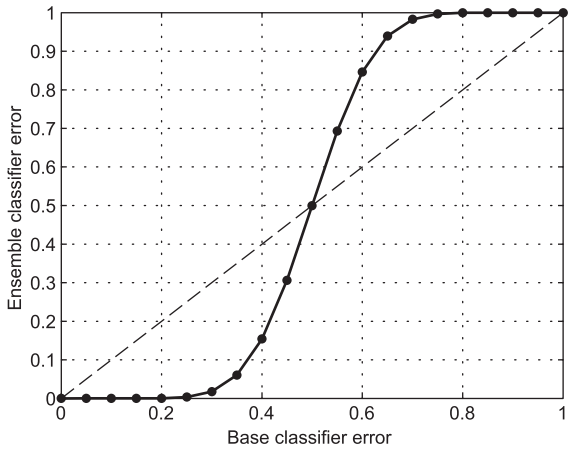


Figure 6.42. Comparison between errors of base classifiers and errors of the ensemble classifier.

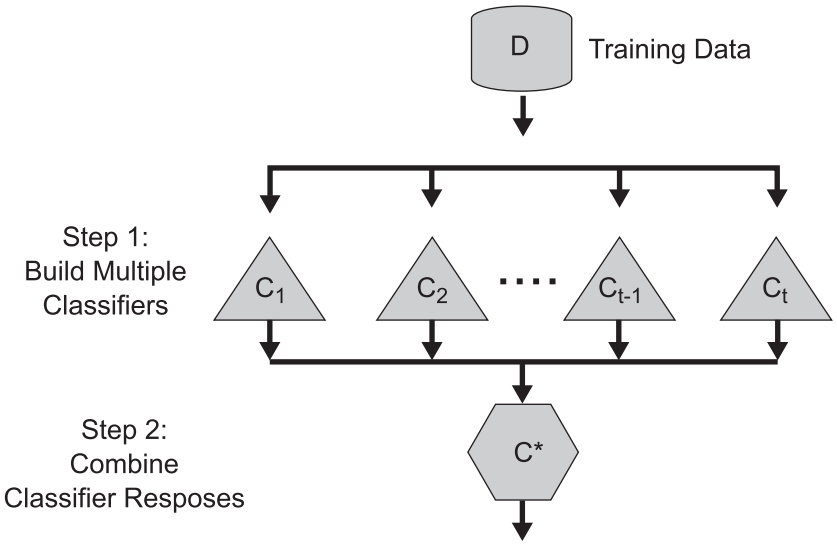


Figure 6.43. A logical view of the ensemble learning method.

sampling distribution and constructing a classifier from each training set. The sampling distribution determines how likely it is that an example will be selected for training, and it may vary from one trial to another. **Bagging** and **boosting** are two examples of ensemble methods that manipulate their training sets. These methods are described in further detail in Sections 6.10.4 and 6.10.5.

2. **By manipulating the input features.** In this approach, a subset of input features is chosen to form each training set. The subset can be either chosen randomly or based on the recommendation of domain experts. Some studies have shown that this approach works very well with data sets that contain highly redundant features. **Random forest**, which is described in Section 6.10.6, is an ensemble method that manipulates its input features and uses decision trees as its base classifiers.

3. **By manipulating the class labels.** This method can be used when the number of classes is sufficiently large. The training data is transformed into a binary class problem by randomly partitioning the class labels into two disjoint subsets, A_0 and A_1. Training examples whose class label belongs to the subset A_0 are assigned to class 0, while those that belong to the subset A_1 are assigned to class 1. The relabeled examples are then used to train a base classifier. By repeating this process multiple times, an ensemble of base classifiers is obtained. When a test example is presented, each base classifier C_i is used to predict its class label. If the test example is predicted as class 0, then all the classes that belong to A_0 will receive a vote. Conversely, if it is predicted to be class 1, then all the classes that belong to A_1 will receive a vote. The votes are tallied and the class that receives the highest vote is assigned to the test example. An example of this approach is the **error-correcting output coding** method described on page 533.

4. **By manipulating the learning algorithm.** Many learning algorithms can be manipulated in such a way that applying the algorithm several times on the same training data will result in the construction of different classifiers. For example, an artificial neural network can change its network topology or the initial weights of the links between neurons. Similarly, an ensemble of decision trees can be constructed by injecting randomness into the tree-growing procedure. For example, instead of choosing the best splitting attribute at each node, we can randomly choose one of the top k attributes for splitting.

The first three approaches are generic methods that are applicable to any classifier, whereas the fourth approach depends on the type of classifier used. The base classifiers for most of these approaches can be generated sequentially (one after another) or in parallel (all at once). Once an ensemble of classifiers has been learned, a test example $\mathbf{x}$ is classified by combining the predictions

made by the base classifiers $C_i(\mathbf{x})$:

$$C^*(\mathbf{x}) = f(C_1(\mathbf{x}), C_2(\mathbf{x}), \ldots, C_k(\mathbf{x})).$$

where f is the function that combines the ensemble responses. One simple approach for obtaining $C^*(\mathbf{x})$ is to take a majority vote of the individual predictions. An alternate approach is to take a *weighted* majority vote, where the weight of a base classifier denotes its accuracy or relevance.

Ensemble methods show the most improvement when used with **unstable classifiers**, i.e., base classifiers that are sensitive to minor perturbations in the training set, because of high model complexity. Although unstable classifiers may have a low *bias* in finding the optimal decision boundary, their predictions have a high *variance* for minor changes in the training set or model selection. This trade-off between bias and variance is discussed in detail in the next section. By aggregating the responses of multiple unstable classifiers, ensemble learning attempts to minimize their variance without worsening their bias.

6.10.3 Bias-Variance Decomposition

Bias-variance decomposition is a formal method for analyzing the generalization error of a predictive model. Although the analysis is slightly different for classification than regression, we first discuss the basic intuition of this decomposition by using an analogue of a regression problem.

Consider the illustrative task of reaching a target y by firing projectiles from a starting position $\mathbf{x}$, as shown in Figure 6.44. The target corresponds to the desired output at a test instance, while the starting position corresponds to its observed attributes. In this analogy, the projectile represents the model used for predicting the target using the observed attributes. Let $\hat{y}$ denote the

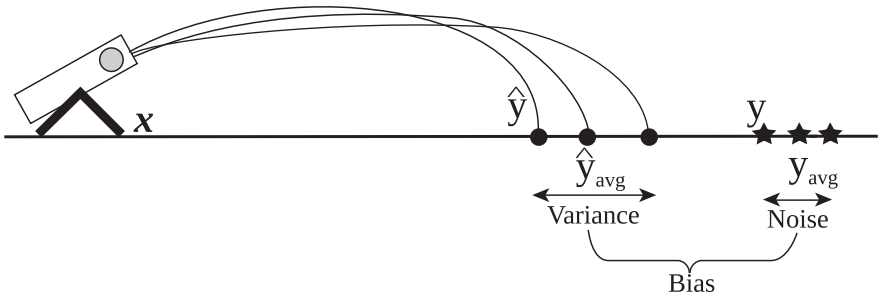
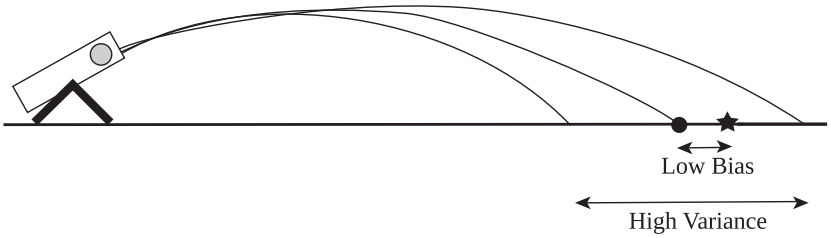
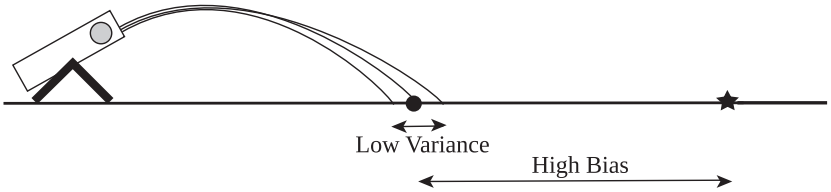


Figure 6.44. Bias-variance decomposition.



(a) Phenomena of Overfitting.



(b) Phenomena of Underfitting.

Figure 6.45. Plots showing the behavior of two-dimensional solutions with constant L_2 and L_1 norms.

The bias-variance trade-off can be used to explain why ensemble learning improves the generalization performance of unstable classifiers. If a base classifier show low bias but high variance, it can become susceptible to overfitting, as even a small change in the training set will result in different predictions. However, by combining the responses of multiple base classifiers, we can expect to reduce the overall variance. Hence, ensemble learning methods show better performance primarily by lowering the variance in the predictions, although they can even help in reducing the bias. One of the simplest approaches for combining predictions and reducing their variance is to compute their average. This forms the basis of the bagging method, described in the following subsection.

6.10.4 Bagging

Bagging, which is also known as bootstrap aggregating, is a technique that repeatedly samples (with replacement) from a data set according to a uniform probability distribution. Each bootstrap sample has the same size as the original data. Because the sampling is done with replacement, some instances may appear several times in the same training set, while others may be

omitted from the training set. On average, a bootstrap sample D_i contains approximately 63% of the original training data because each sample has a probability $1 - (1 - 1/N)^N$ of being selected in each D_i. If N is sufficiently large, this probability converges to $1 - 1/e \simeq 0.632$. The basic procedure for bagging is summarized in Algorithm 6.5. After training the k classifiers, a test instance is assigned to the class that receives the highest number of votes.

To illustrate how bagging works, consider the data set shown in Table 6.4. Let x denote a one-dimensional attribute and y denote the class label. Suppose we use only one-level binary decision trees, with a test condition $x \leq k$, where k is a split point chosen to minimize the entropy of the leaf nodes. Such a tree is also known as a **decision stump**.

Table 6.4. Example of data set used to construct an ensemble of bagging classifiers.

x	0.1	0.2	0.3	0.4	0.5	0.6	0.7	0.8	0.9	1
y	1	1	1	−1	−1	−1	−1	1	1	1

Without bagging, the best decision stump we can produce splits the instances at either $x \leq 0.35$ or $x \leq 0.75$. Either way, the accuracy of the tree is at most 70%. Suppose we apply the bagging procedure on the data set using 10 bootstrap samples. The examples chosen for training in each bagging round are shown in Figure 6.46. On the right-hand side of each table, we also describe the decision stump being used in each round.

We classify the entire data set given in Table 6.4 by taking a majority vote among the predictions made by each base classifier. The results of the predictions are shown in Figure 6.47. Since the class labels are either −1 or +1, taking the majority vote is equivalent to summing up the predicted values of y and examining the sign of the resulting sum (refer to the second to last

Algorithm 6.5 Bagging algorithm.

1: Let k be the number of bootstrap samples.
2: **for** $i = 1$ to k **do**
3: Create a bootstrap sample of size N, D_i.
4: Train a base classifier C_i on the bootstrap sample D_i.
5: **end for**
6: $C^*(x) = \underset{y}{\operatorname{argmax}} \sum_i \delta\big(C_i(x) = y\big)$.
$\{\delta(\cdot) = 1$ if its argument is true and 0 otherwise.$\}$

Bagging Round 1:

x	0.1	0.2	0.2	0.3	0.4	0.4	0.5	0.6	0.9	0.9
y	1	1	1	1	-1	-1	-1	-1	1	1

x <= 0.35 ==> y = 1
x > 0.35 ==> y = -1

Bagging Round 2:

x	0.1	0.2	0.3	0.4	0.5	0.8	0.9	1	1	1
y	1	1	1	-1	-1	1	1	1	1	1

x <= 0.65 ==> y = 1
x > 0.65 ==> y = 1

Bagging Round 3:

x	0.1	0.2	0.3	0.4	0.4	0.5	0.7	0.7	0.8	0.9
y	1	1	1	-1	-1	-1	-1	-1	1	1

x <= 0.35 ==> y = 1
x > 0.35 ==> y = -1

Bagging Round 4:

x	0.1	0.1	0.2	0.4	0.4	0.5	0.5	0.7	0.8	0.9
y	1	1	1	-1	-1	-1	-1	-1	1	1

x <= 0.3 ==> y = 1
x > 0.3 ==> y = -1

Bagging Round 5:

x	0.1	0.1	0.2	0.5	0.6	0.6	0.6	1	1	1
y	1	1	1	-1	-1	-1	-1	1	1	1

x <= 0.35 ==> y = 1
x > 0.35 ==> y = -1

Bagging Round 6:

x	0.2	0.4	0.5	0.6	0.7	0.7	0.7	0.8	0.9	1
y	1	-1	-1	-1	-1	-1	-1	1	1	1

x <= 0.75 ==> y = -1
x > 0.75 ==> y = 1

Bagging Round 7:

x	0.1	0.4	0.4	0.6	0.7	0.8	0.9	0.9	0.9	1
y	1	-1	-1	-1	-1	1	1	1	1	1

x <= 0.75 ==> y = -1
x > 0.75 ==> y = 1

Bagging Round 8:

x	0.1	0.2	0.5	0.5	0.5	0.7	0.7	0.8	0.9	1
y	1	1	-1	-1	-1	-1	-1	1	1	1

x <= 0.75 ==> y = -1
x > 0.75 ==> y = 1

Bagging Round 9:

x	0.1	0.3	0.4	0.4	0.6	0.7	0.7	0.8	1	1
y	1	1	-1	-1	-1	-1	-1	1	1	1

x <= 0.75 ==> y = -1
x > 0.75 ==> y = 1

Bagging Round 10:

x	0.1	0.1	0.1	0.1	0.3	0.3	0.8	0.8	0.9	0.9
y	1	1	1	1	1	1	1	1	1	1

x <= 0.05 ==> y = -1
x > 0.05 ==> y = 1

Figure 6.46. Example of bagging.

row in Figure 6.47). Notice that the ensemble classifier perfectly classifies all 10 examples in the original data.

The preceding example illustrates another advantage of using ensemble methods in terms of enhancing the representation of the target function. Even though each base classifier is a decision stump, combining the classifiers can lead to a decision boundary that mimics a decision tree of depth 2.

Bagging improves generalization error by reducing the variance of the base classifiers. The performance of bagging depends on the stability of the base

Round	x=0.1	x=0.2	x=0.3	x=0.4	x=0.5	x=0.6	x=0.7	x=0.8	x=0.9	x=1.0
1	1	1	1	-1	-1	-1	-1	-1	-1	-1
2	1	1	1	1	1	1	1	1	1	1
3	1	1	1	-1	-1	-1	-1	-1	-1	-1
4	1	1	1	-1	-1	-1	-1	-1	-1	-1
5	1	1	1	-1	-1	-1	-1	-1	-1	-1
6	-1	-1	-1	-1	-1	-1	-1	1	1	1
7	-1	-1	-1	-1	-1	-1	-1	1	1	1
8	-1	-1	-1	-1	-1	-1	-1	1	1	1
9	-1	-1	-1	-1	-1	-1	-1	1	1	1
10	1	1	1	1	1	1	1	1	1	1
Sum	2	2	2	-6	-6	-6	-6	2	2	2
Sign	1	1	1	-1	-1	-1	-1	1	1	1
True Class	1	1	1	-1	-1	-1	-1	1	1	1

Figure 6.47. Example of combining classifiers constructed using the bagging approach.

classifier. If a base classifier is unstable, bagging helps to reduce the errors associated with random fluctuations in the training data. If a base classifier is stable, i.e., robust to minor perturbations in the training set, then the error of the ensemble is primarily caused by bias in the base classifier. In this situation, bagging may not be able to improve the performance of the base classifiers significantly. It may even degrade the classifier's performance because the effective size of each training set is about 37% smaller than the original data.

6.10.5 Boosting

Boosting is an iterative procedure used to adaptively change the distribution of training examples for learning base classifiers so that they increasingly focus on examples that are hard to classify. Unlike bagging, boosting assigns a weight to each training example and may adaptively change the weight at the end of each boosting round. The weights assigned to the training examples can be used in the following ways:

1. They can be used to inform the sampling distribution used to draw a set of bootstrap samples from the original data.

2. They can be used to learn a model that is biased toward examples with higher weight.

This section describes an algorithm that uses weights of examples to determine the sampling distribution of its training set. Initially, the examples

are assigned equal weights, $1/N$, so that they are equally likely to be chosen for training. A sample is drawn according to the sampling distribution of the training examples to obtain a new training set. Next, a classifier is built from the training set and used to classify all the examples in the original data. The weights of the training examples are updated at the end of each boosting round. Examples that are classified incorrectly will have their weights increased, while those that are classified correctly will have their weights decreased. This forces the classifier to focus on examples that are difficult to classify in subsequent iterations.

The following table shows the examples chosen during each boosting round, when applied to the data shown in Table 6.4.

Boosting (Round 1):	7	3	2	8	7	9	4	10	6	3
Boosting (Round 2):	5	4	9	4	2	5	1	7	4	2
Boosting (Round 3):	4	4	8	10	4	5	4	6	3	4

Initially, all the examples are assigned the same weights. However, some examples may be chosen more than once, e.g., examples 3 and 7, because the sampling is done with replacement. A classifier built from the data is then used to classify all the examples. Suppose example 4 is difficult to classify. The weight for this example will be increased in future iterations as it gets misclassified repeatedly. Meanwhile, examples that were not chosen in the previous round, e.g., examples 1 and 5, also have a better chance of being selected in the next round since their predictions in the previous round were likely to be wrong. As the boosting rounds proceed, examples that are the hardest to classify tend to become even more prevalent. The final ensemble is obtained by aggregating the base classifiers obtained from each boosting round.

Over the years, several implementations of the boosting algorithm have been developed. These algorithms differ in terms of (1) how the weights of the training examples are updated at the end of each boosting round, and (2) how the predictions made by each classifier are combined. An implementation called AdaBoost is explored in the next section.

AdaBoost

Let $\{(\mathbf{x}_j, y_j) \mid j = 1, 2, \ldots, N\}$ denote a set of N training examples. In the AdaBoost algorithm, the importance of a base classifier C_i depends on its

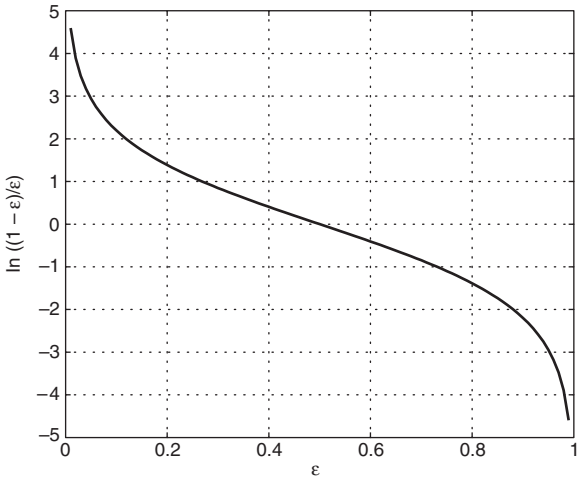


Figure 6.48. Plot of α as a function of training error ϵ.

error rate, which is defined as

$$\epsilon_i = \frac{1}{N}\left[\sum_{j=1}^{N} w_j \, I\Big(C_i(\mathbf{x}_j) \neq y_j\Big)\right], \tag{6.102}$$

where $I(p) = 1$ if the predicate p is true, and 0 otherwise. The importance of a classifier C_i is given by the following parameter,

$$\alpha_i = \frac{1}{2}\ln\left(\frac{1-\epsilon_i}{\epsilon_i}\right).$$

Note that α_i has a large positive value if the error rate is close to 0 and a large negative value if the error rate is close to 1, as shown in Figure 6.48.

The α_i parameter is also used to update the weight of the training examples. To illustrate, let $w_i^{(j)}$ denote the weight assigned to example $(\mathbf{x}_i, y_i)$ during the jth boosting round. The weight update mechanism for AdaBoost is given by the equation:

$$w_i^{(j+1)} = \frac{w_i^{(j)}}{Z_j} \times \begin{cases} e^{-\alpha_j} & \text{if } C_j(\mathbf{x_i}) = y_i \\ e^{\alpha_j} & \text{if } C_j(\mathbf{x_i}) \neq y_i \end{cases}, \tag{6.103}$$

where Z_j is the normalization factor used to ensure that $\sum_i w_i^{(j+1)} = 1$. The weight update formula given in Equation 6.103 increases the weights

of incorrectly classified examples and decreases the weights of those classified correctly.

Instead of using a majority voting scheme, the prediction made by each classifier C_j is weighted according to α_j. This approach allows AdaBoost to penalize models that have poor accuracy, e.g., those generated at the earlier boosting rounds. In addition, if any intermediate rounds produce an error rate higher than 50%, the weights are reverted back to their original uniform values, $w_i = 1/N$, and the resampling procedure is repeated. The AdaBoost algorithm is summarized in Algorithm 6.6.

Algorithm 6.6 AdaBoost algorithm.

1: $\mathbf{w} = \{w_j = 1/N \mid j = 1, 2, \ldots, N\}$. {Initialize the weights for all N examples.}
2: Let k be the number of boosting rounds.
3: **for** $i = 1$ to k **do**
4: Create training set D_i by sampling (with replacement) from D according to $\mathbf{w}$.
5: Train a base classifier C_i on D_i.
6: Apply C_i to all examples in the original training set, D.
7: $\epsilon_i = \frac{1}{N}\left[\sum_j w_j \, \delta\big(C_i(x_j) \neq y_j\big)\right]$ {Calculate the weighted error.}
8: **if** $\epsilon_i > 0.5$ **then**
9: $\mathbf{w} = \{w_j = 1/N \mid j = 1, 2, \ldots, N\}$. {Reset the weights for all N examples.}
10: Go back to Step 4.
11: **end if**
12: $\alpha_i = \frac{1}{2} \ln \frac{1-\epsilon_i}{\epsilon_i}$.
13: Update the weight of each example according to Equation 6.103.
14: **end for**
15: $C^*(\mathbf{x}) = \underset{y}{\operatorname{argmax}} \sum_{j=1}^{T} \alpha_j \delta(C_j(\mathbf{x}) = y)\big)$.

Let us examine how the boosting approach works on the data set shown in Table 6.4. Initially, all the examples have identical weights. After three boosting rounds, the examples chosen for training are shown in Figure 6.49(a). The weights for each example are updated at the end of each boosting round using Equation 6.103, as shown in Figure 6.50(b).

Without boosting, the accuracy of the decision stump is, at best, 70%. With AdaBoost, the results of the predictions are given in Figure 6.50(b). The final prediction of the ensemble classifier is obtained by taking a weighted average of the predictions made by each base classifier, which is shown in the last row of Figure 6.50(b). Notice that AdaBoost perfectly classifies all the examples in the training data.

Boosting Round 1:

x	0.1	0.4	0.5	0.6	0.6	0.7	0.7	0.7	0.8	1
y	1	-1	-1	-1	-1	-1	-1	-1	1	1

Boosting Round 2:

x	0.1	0.1	0.2	0.2	0.2	0.2	0.3	0.3	0.3	0.3
y	1	1	1	1	1	1	1	1	1	1

Boosting Round 3:

x	0.2	0.2	0.4	0.4	0.4	0.4	0.5	0.6	0.6	0.7
y	1	1	-1	-1	-1	-1	-1	-1	-1	-1

(a) Training records chosen during boosting.

Round	x=0.1	x=0.2	x=0.3	x=0.4	x=0.5	x=0.6	x=0.7	x=0.8	x=0.9	x=1.0
1	0.1	0.1	0.1	0.1	0.1	0.1	0.1	0.1	0.1	0.1
2	0.311	0.311	0.311	0.01	0.01	0.01	0.01	0.01	0.01	0.01
3	0.029	0.029	0.029	0.228	0.228	0.228	0.228	0.009	0.009	0.009

(b) Weights of training records.

Figure 6.49. Example of boosting.

An important analytical result of boosting shows that the training error of the ensemble is bounded by the following expression:

$$e_{\text{ensemble}} \leq \prod_i \left[\sqrt{\epsilon_i(1-\epsilon_i)} \right], \tag{6.104}$$

where ϵ_i is the error rate of each base classifier i. If the error rate of the base classifier is less than 50%, we can write $\epsilon_i = 0.5 - \gamma_i$, where γ_i measures how much better the classifier is than random guessing. The bound on the training error of the ensemble becomes

$$e_{\text{ensemble}} \leq \prod_i \sqrt{1-4\gamma_i^2} \leq \exp\left(-2 \sum_i \gamma_i^2 \right). \tag{6.105}$$

Hence, the training error of the ensemble decreases exponentially, which leads to the fast convergence of the algorithm. By focusing on examples that are difficult to classify by base classifiers, it is able to reduce the bias of the final predictions along with the variance. AdaBoost has been shown to provide significant improvements in performance over base classifiers on a range of data

Round	Split Point	Left Class	Right Class	α
1	0.75	-1	1	1.738
2	0.05	1	1	2.7784
3	0.3	1	-1	4.1195

(a)

Round	x=0.1	x=0.2	x=0.3	x=0.4	x=0.5	x=0.6	x=0.7	x=0.8	x=0.9	x=1.0
1	-1	-1	-1	-1	-1	-1	-1	1	1	1
2	1	1	1	1	1	1	1	1	1	1
3	1	1	1	-1	-1	-1	-1	-1	-1	-1
Sum	5.16	5.16	5.16	-3.08	-3.08	-3.08	-3.08	0.397	0.397	0.397
Sign	1	1	1	-1	-1	-1	-1	1	1	1

(b)

Figure 6.50. Example of combining classifiers constructed using the AdaBoost approach.

sets. Nevertheless, because of its tendency to focus on training examples that are wrongly classified, the boosting technique can be susceptible to overfitting, resulting in poor generalization performance in some scenarios.

6.10.6 Random Forests

Random forests attempt to improve the generalization performance by constructing an ensemble of *decorrelated* decision trees. Random forests build on the idea of bagging to use a different bootstrap sample of the training data for learning decision trees. However, a key distinguishing feature of random forests from bagging is that at every internal node of a tree, the best splitting criterion is chosen among a small set of randomly selected attributes. In this way, random forests construct ensembles of decision trees by not only manipulating training instances (by using bootstrap samples similar to bagging), but also the input attributes (by using different subsets of attributes at every internal node).

Given a training set D consisting of n instances and d attributes, the basic procedure of training a random forest classifier can be summarized using the following steps:

1. Construct a bootstrap sample D_i of the training set by randomly sampling n instances (with replacement) from D.

2. Use D_i to learn a decision tree T_i as follows. At every internal node of T_i, randomly sample a set of p attributes and choose an attribute from this subset that shows the maximum reduction in an impurity measure for splitting. Repeat this procedure till every leaf is pure, i.e., containing instances from the same class.

Once an ensemble of decision trees have been constructed, their average prediction (majority vote) on a test instance is used as the final prediction of the random forest. Note that the decision trees involved in a random forest are *unpruned* trees, as they are allowed to grow to their largest possible size till every leaf is pure. Hence, the base classifiers of random forest represent unstable classifiers that have low bias but high variance, because of their large size.

Another property of the base classifiers learned in random forests is the *lack of correlation* among their model parameters and test predictions. This can be attributed to the use of an independently sampled data set D_i for learning every decision tree T_i, similar to the bagging approach. However, random forests have the additional advantage of choosing a splitting criterion at every internal node using a different (and randomly selected) subset of attributes. This property significantly helps in breaking the correlation structure, if any, among the decision trees T_i.

To realize this, consider a training set involving a large number of attributes, where only a small subset of attributes are strong predictors of the target class, whereas other attributes are weak indicators. Given such a training set, even if we consider different bootstrap samples D_i for learning T_i, we would mostly be choosing the same attributes for splitting at internal nodes, because the weak attributes would be largely overlooked when compared with the strong predictors. This can result in a considerable correlation among the trees. However, if we restrict the choice of attributes at every internal node to a random subset of attributes, we can ensure the selection of both strong and weak predictors, thus promoting diversity among the trees. This principle is utilized by random forests for creating decorrelated decision trees.

By aggregating the predictions of an ensemble of strong and decorrelated decision trees, random forests are able to reduce the variance of the trees without negatively impacting their low bias. This makes random forests quite robust to overfitting. Additionally, because of their ability to consider only a small subset of attributes at every internal node, random forests are computationally fast and robust even in high-dimensional settings.

The number of attributes to be selected at every node, p, is a hyperparameter of the random forest classifier. A small value of p can reduce the

correlation among the classifiers but may also reduce their strength. A large value can improve their strength but may result in correlated trees similar to bagging. Although common suggestions for p in the literature include $\sqrt{d}$ and $\log_2 d+1$, a suitable value of p for a given training set can always be selected by tuning it over a validation set, as described in Chapter 3. However, there is an alternative way for selecting hyper-parameters in random forests, which does not require using a separate validation set. It involves computing a reliable estimate of the generalization error rate directly during training, known as the **out-of-bag (oob)** error estimate. The oob estimate can be computed for any generic ensemble learning method that builds independent base classifiers using bootstrap samples of the training set, e.g., bagging and random forests. The approach for computing oob estimate can be described as follows.

Consider an ensemble learning method that uses an independent base classifier T_i built on a bootstrap sample of the training set D_i. Since every training instance $\mathbf{x}$ will be used for training approximately 63% of base classifiers, we can call $\mathbf{x}$ as an **out-of-bag sample** for the remaining 27% of base classifiers that did not use it for training. If we use these remaining 27% classifiers to make predictions on $\mathbf{x}$, we can obtain the **oob error** on $\mathbf{x}$ by taking their majority vote and comparing it with its class label. Note that the oob error estimates the error of 27% classifiers on an instance that was not used for training those classifiers. Hence, the oob error can be considered as a reliable estimate of generalization error. By taking the average of oob errors of all training instances, we can compute the overall oob error estimate. This can be used as an alternative to the validation error rate for selecting hyper-parameters. Hence, random forests do not need to use a separate partition of the training set for validation, as it can simultaneously train the base classifiers and compute generalization error estimates on the same data set.

Random forests have been empirically found to provide significant improvements in generalization performance that are often comparable, if not superior, to the improvements provided by the AdaBoost algorithm. Random forests are also more robust to overfitting and run much faster than the AdaBoost algorithm.

6.10.7 Empirical Comparison among Ensemble Methods

Table 6.5 shows the empirical results obtained when comparing the performance of a decision tree classifier against bagging, boosting, and random forest. The base classifiers used in each ensemble method consist of 50 decision